Agenda

- ❑ Introduction
- ❑ Kubernetes Architecture
  - ➢ ETCD in Kubernetes
  - ➢ Kube-API Server
  - ➢ Controller Managers
  - ➢ Kube Scheduler
  - ➢ Kubelet
- ❑ Install minikube
- ❑ Kubernetes Resources/Objects
- ❑ Kubernetes Objects creation.
  - ➢ YAML file Basics
  - ➢ namespace
  - ➢ POD
  - ➢ Multi-Pod
  - ➢ Replicaset
  - ➢ Labels and selectors
  - ➢ Deployments
    - Rollout and versioning
    - Deployment strategy
    - Upgrade
  - ➢ Daemonsets
- ❑ Scheduling
- ❑ Services
  - ➢ NodePort
  - ➢ Clusterip
- ❑ Environment variables
  - ➢ ConfigMap
  - ➢ configMap on pod
  - ➢ Secrets

C2 General

Agenda

- ❑ Secrets on pod
- ❑ Volumes
  - ➢ Persistent Volume
  - ➢ Persistent Volume Claims
  - ➢ Storage Class
- ❑ Scheduling
  - ➢ Node selectore
  - ➢ Nodename
  - ➢ Affinity roles
  - ➢ Taint and toleration
- ❑ Authentication and Authorization
  - ➢ Authentication using Tokens
  - ➢ Authentication using Certificates
  - ➢ Authorization Using RBAC

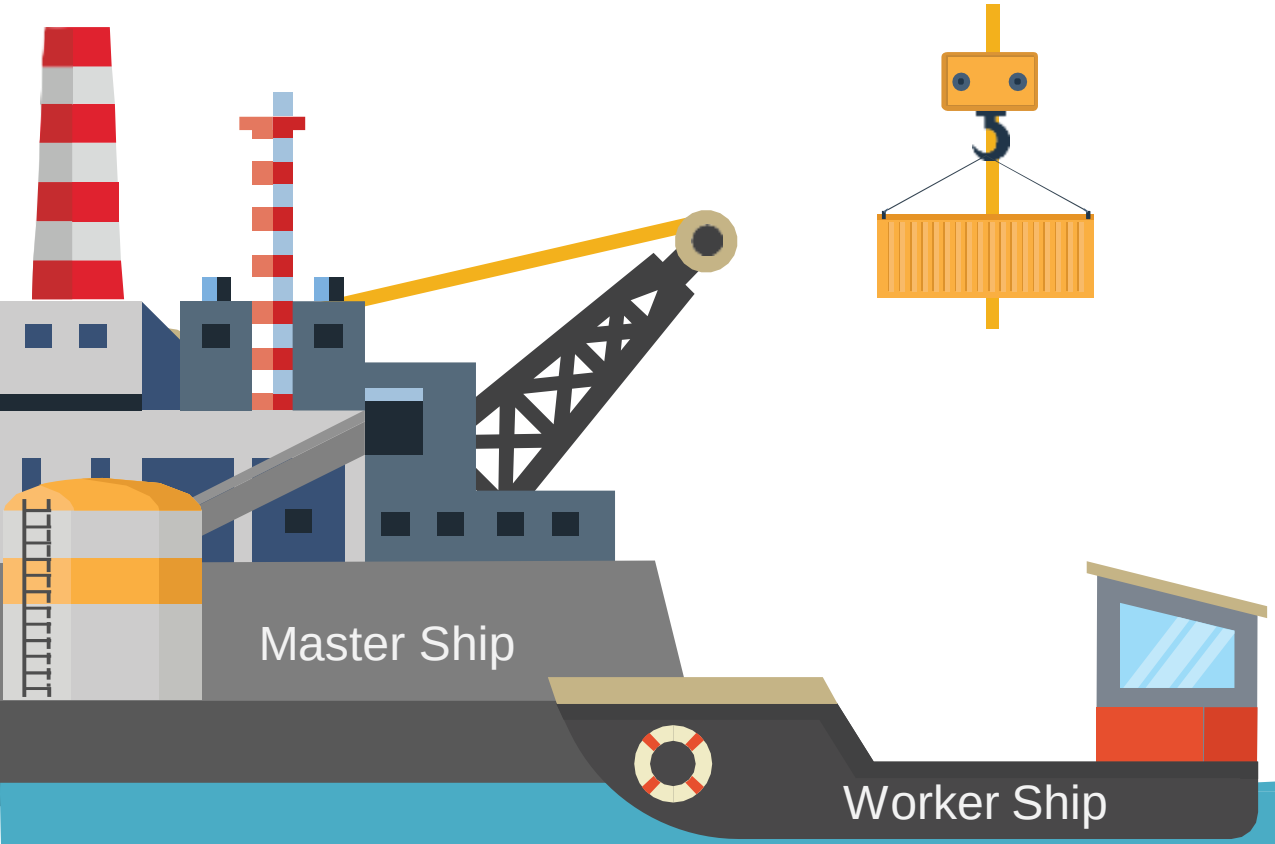
C2 General

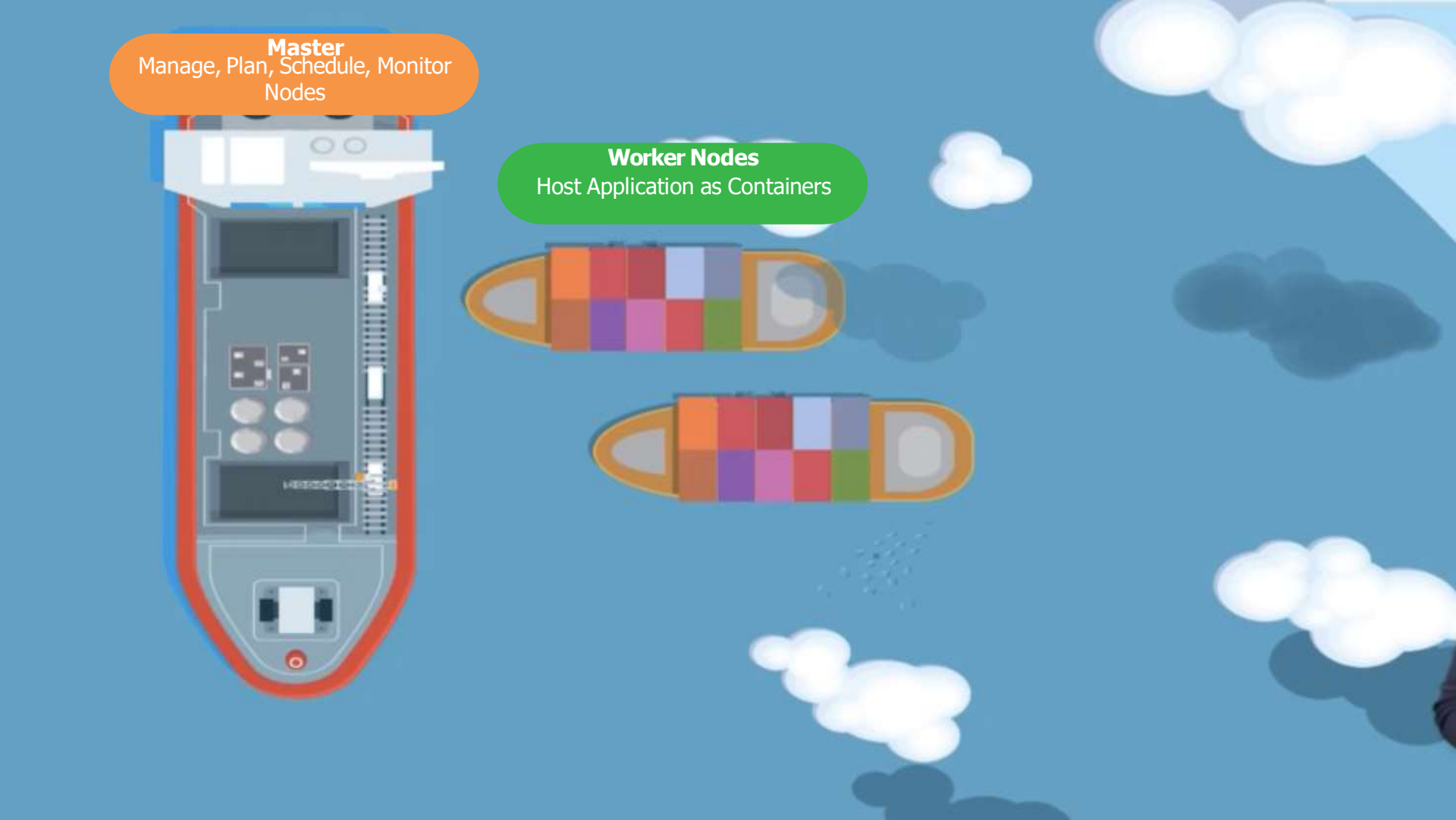
Cluster Architecture

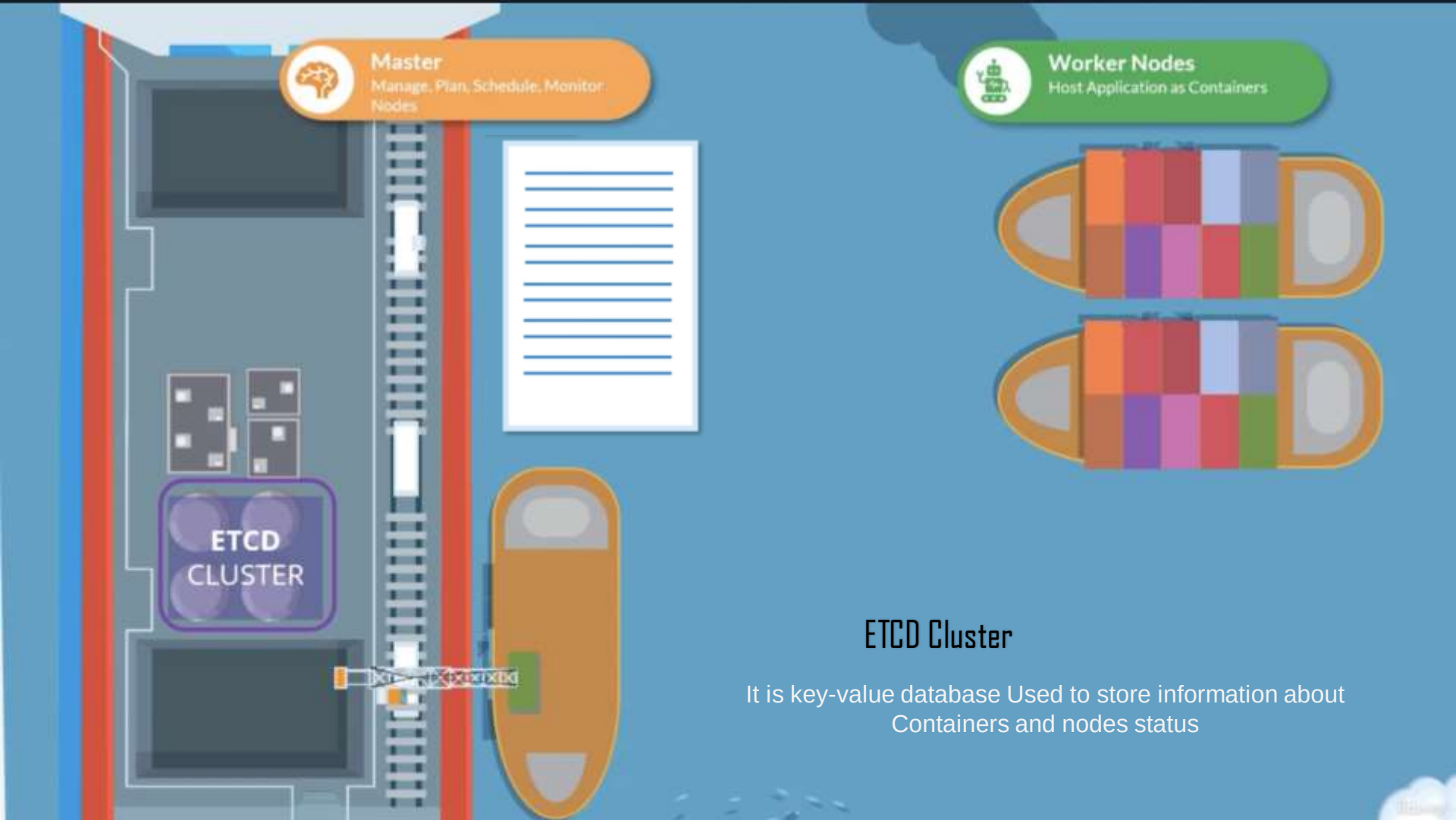
- Kubernetes Architecture
- ETCD in Kubernetes
- Kube-API Server
- Controller Managers
- Kube Scheduler
- Kubelet

C2 General

KUBERNETES ARCHITECTURE

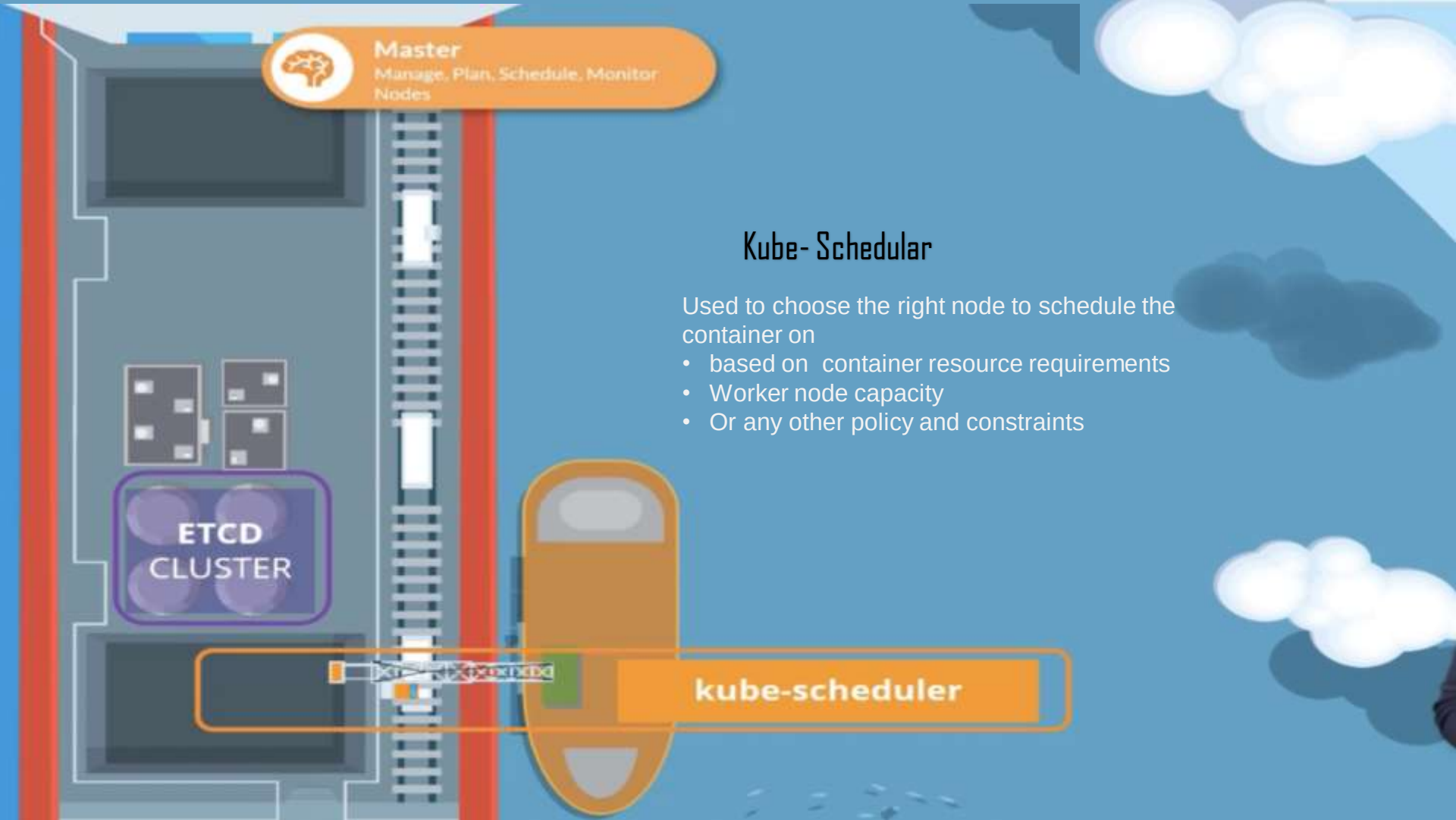

Master Ship
Worker Ship


Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers



ETCD Cluster

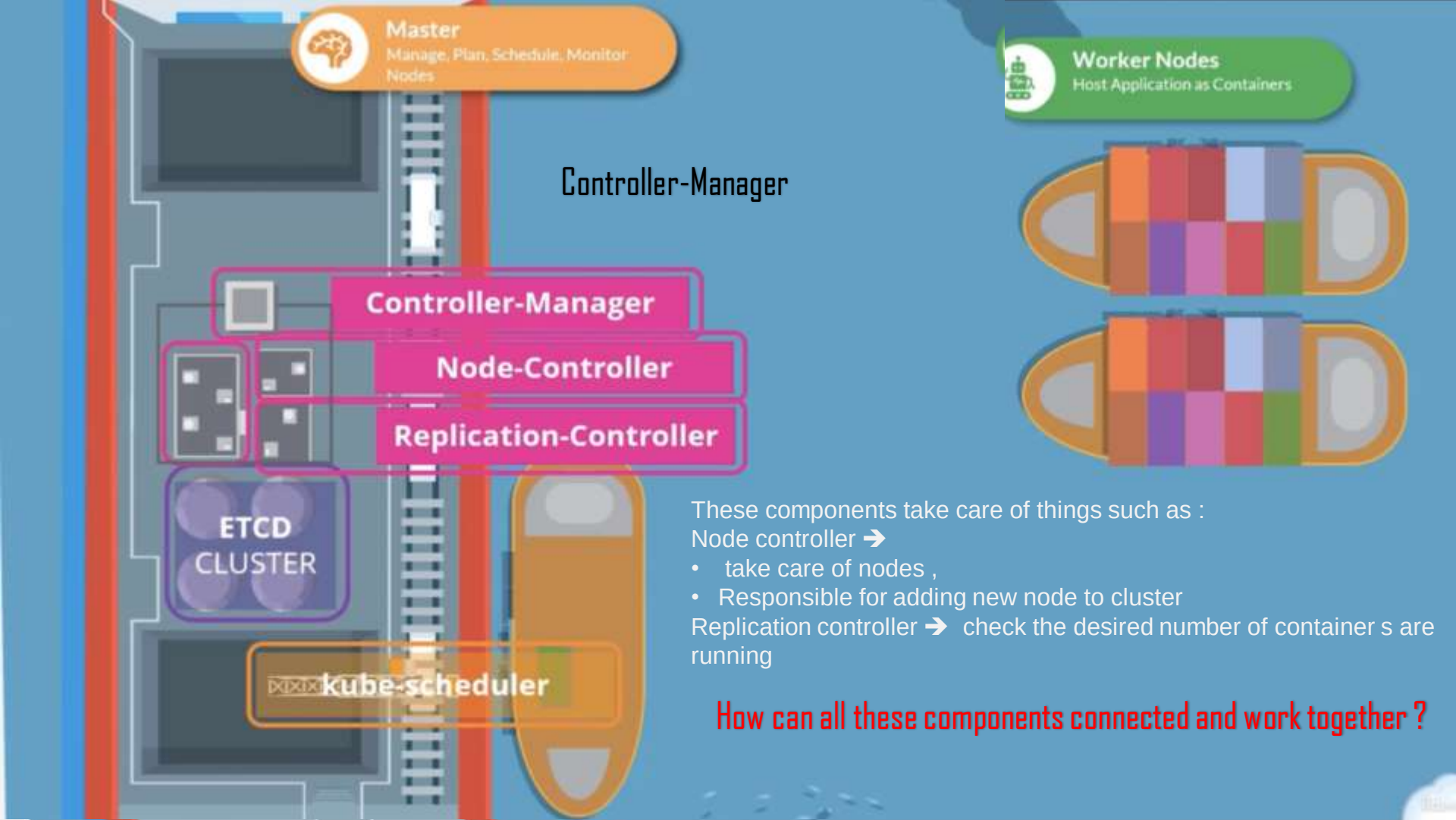
It is key-value database Used to store information about Containers and nodes status



Kube- Schedular

Used to choose the right node to schedule the container on

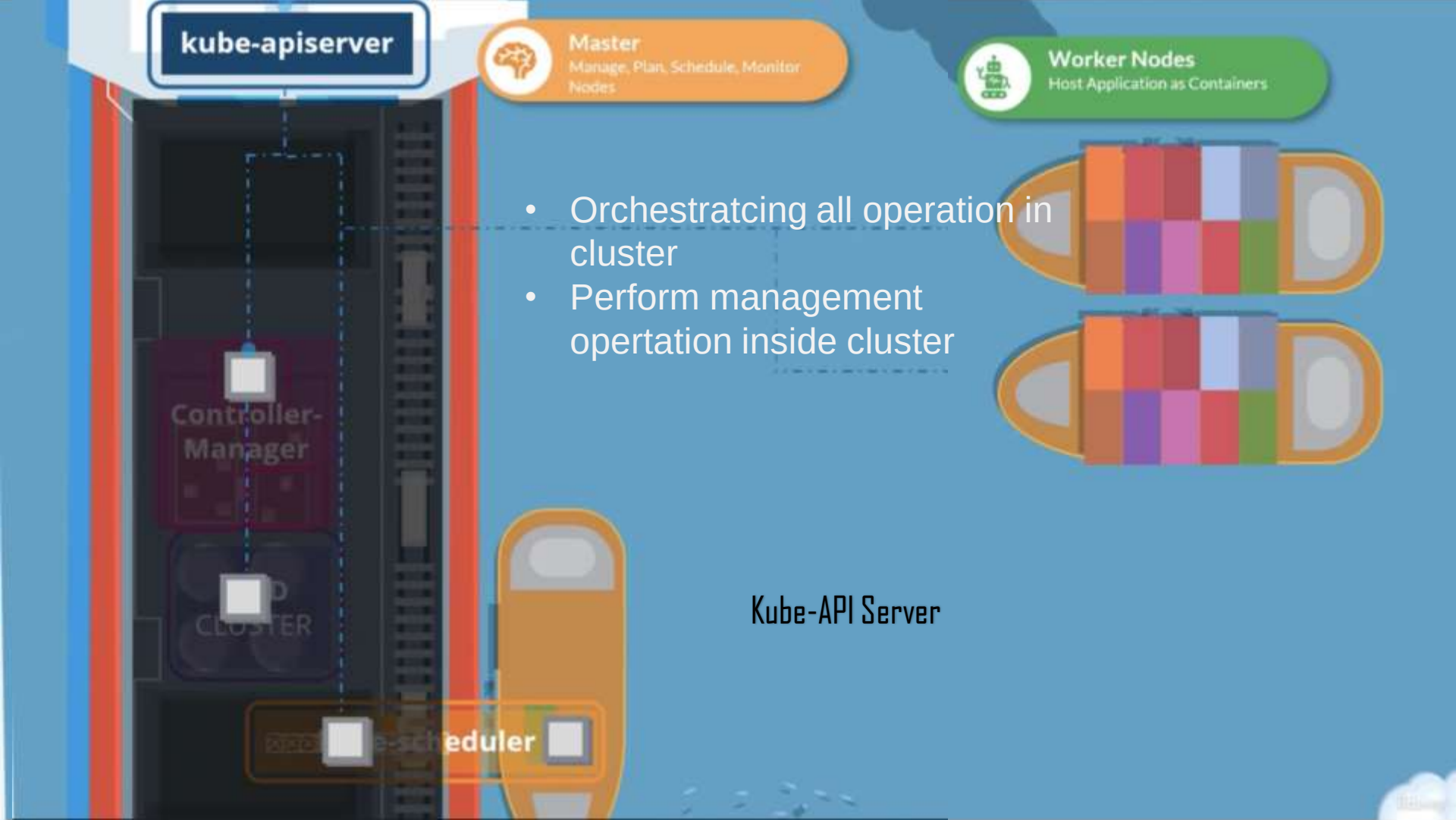
- based on container resource requirements
- Worker node capacity
- Or any other policy and constraints



These components take care of things such as :
Node controller ➔
- take care of nodes ,
- Responsible for adding new node to cluster

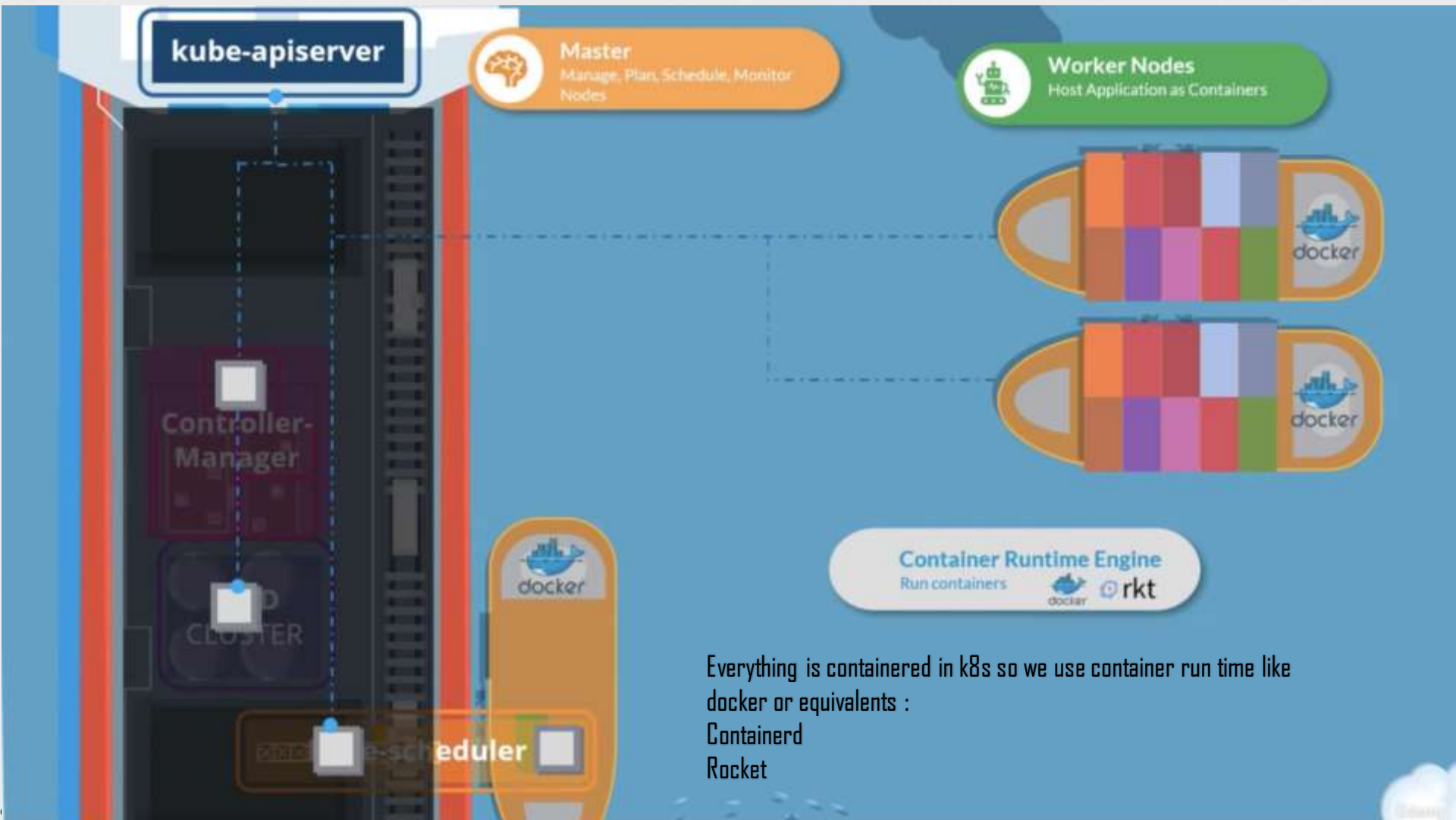
Replication controller ➔ check the desired number of container s are running

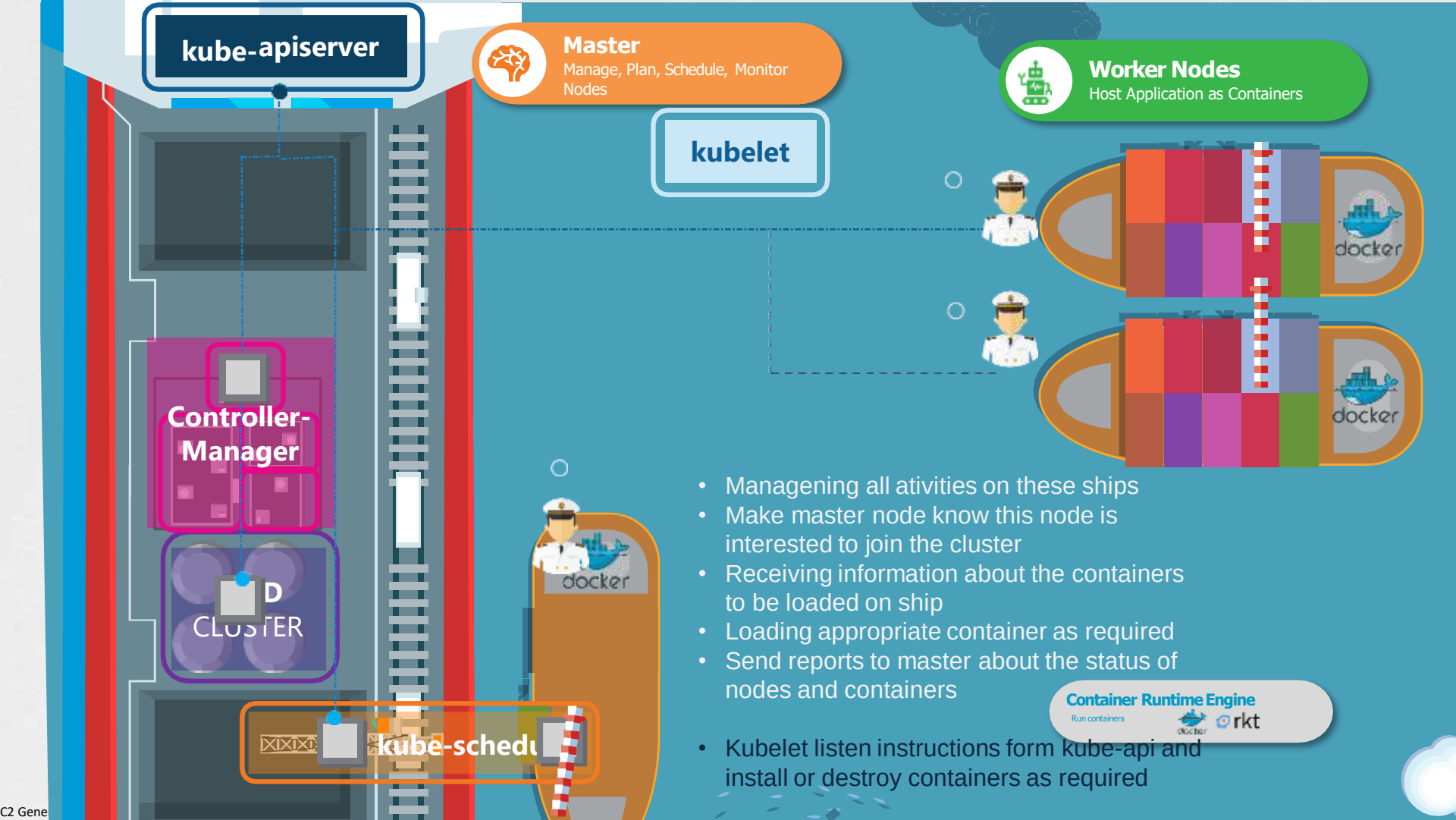
How can all these components connected and work together ?



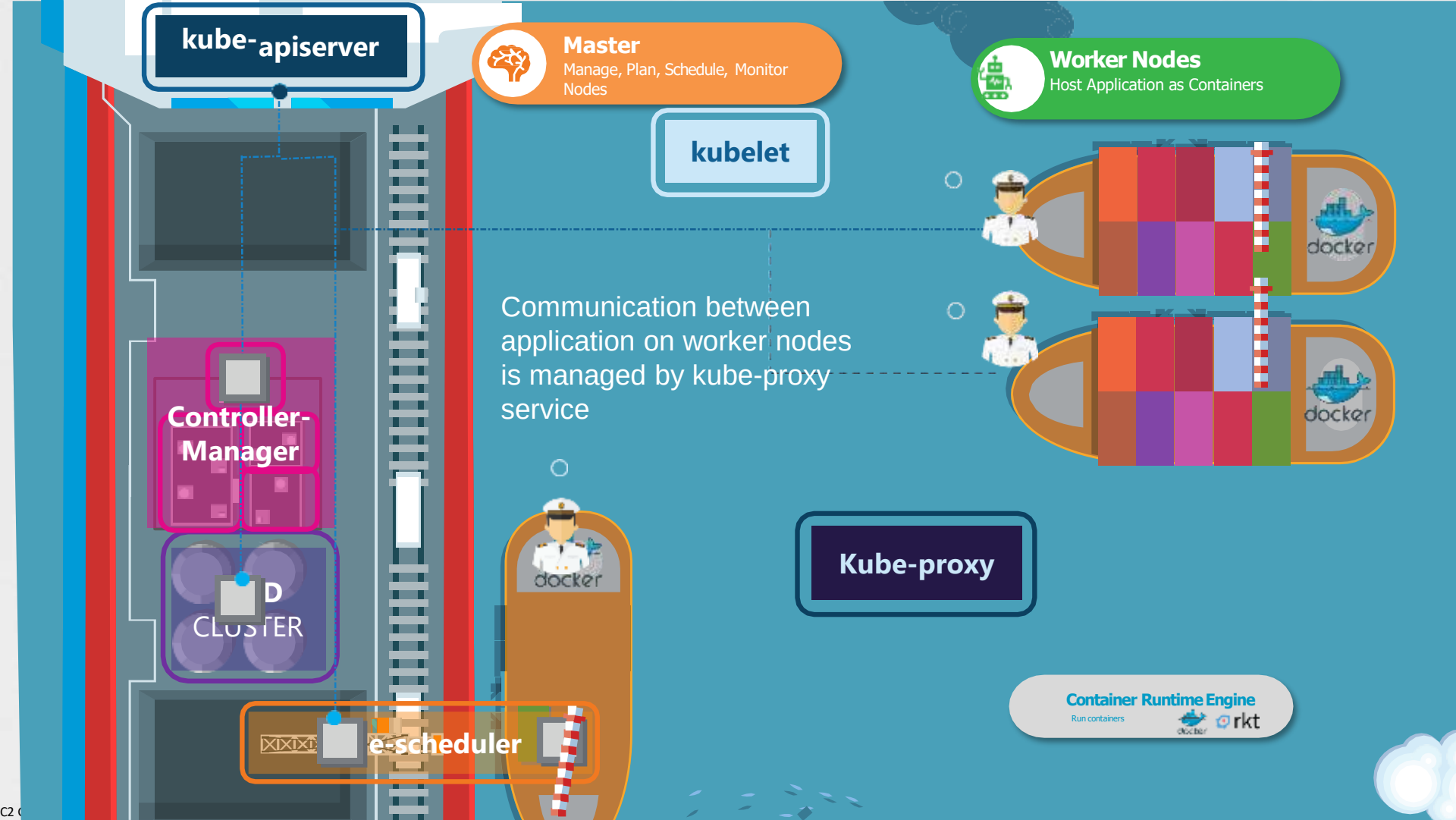
- Orchestratcing all operation in cluster
- Perform management opertation inside cluster

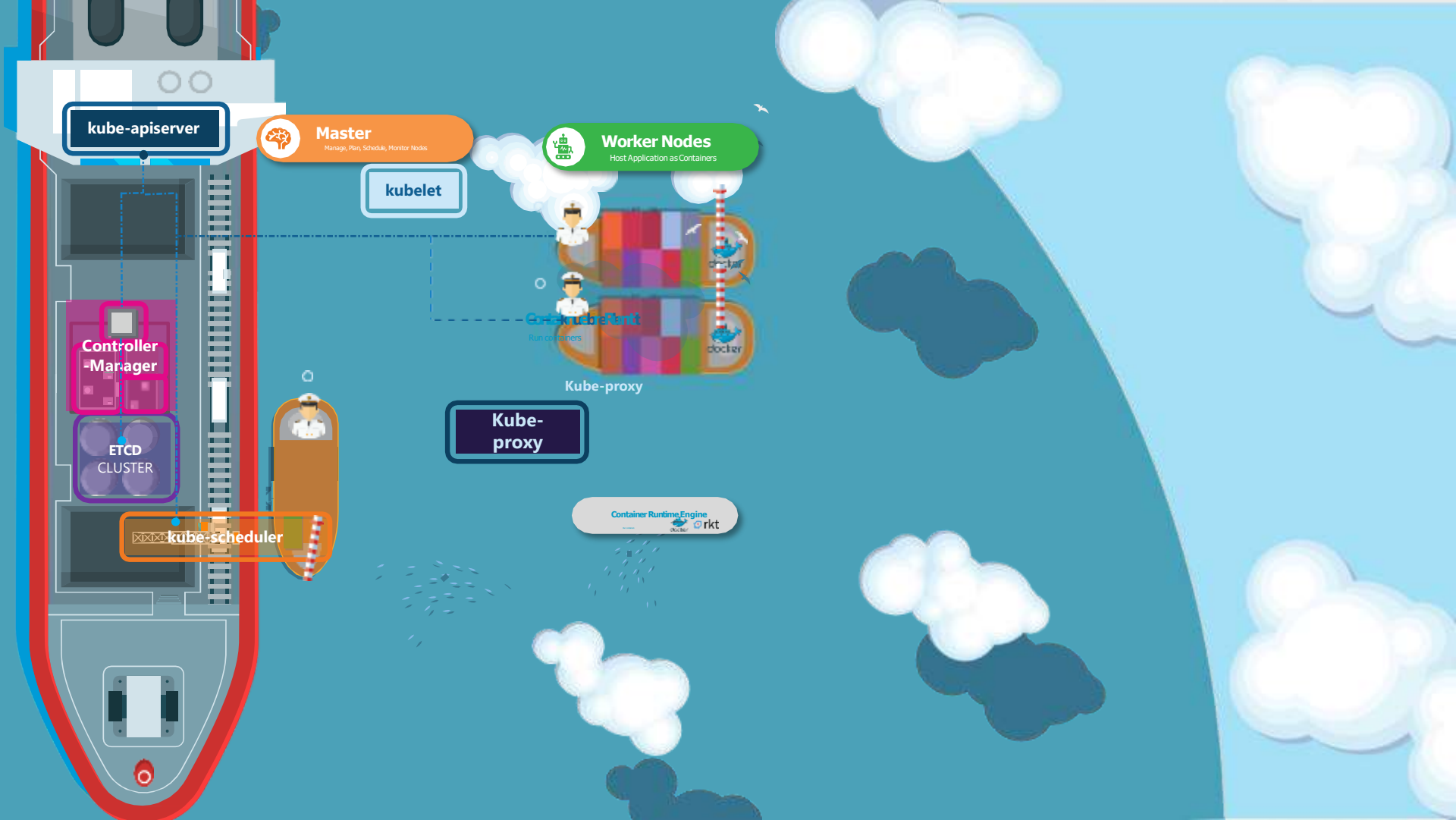
Kube-API Server


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
docker
docker
Controller-Manager
docker
Container Runtime Engine
Run containers
rkt
Everything is containered in k8s so we use container run time like docker or equivalents :
Containerd
Rocket

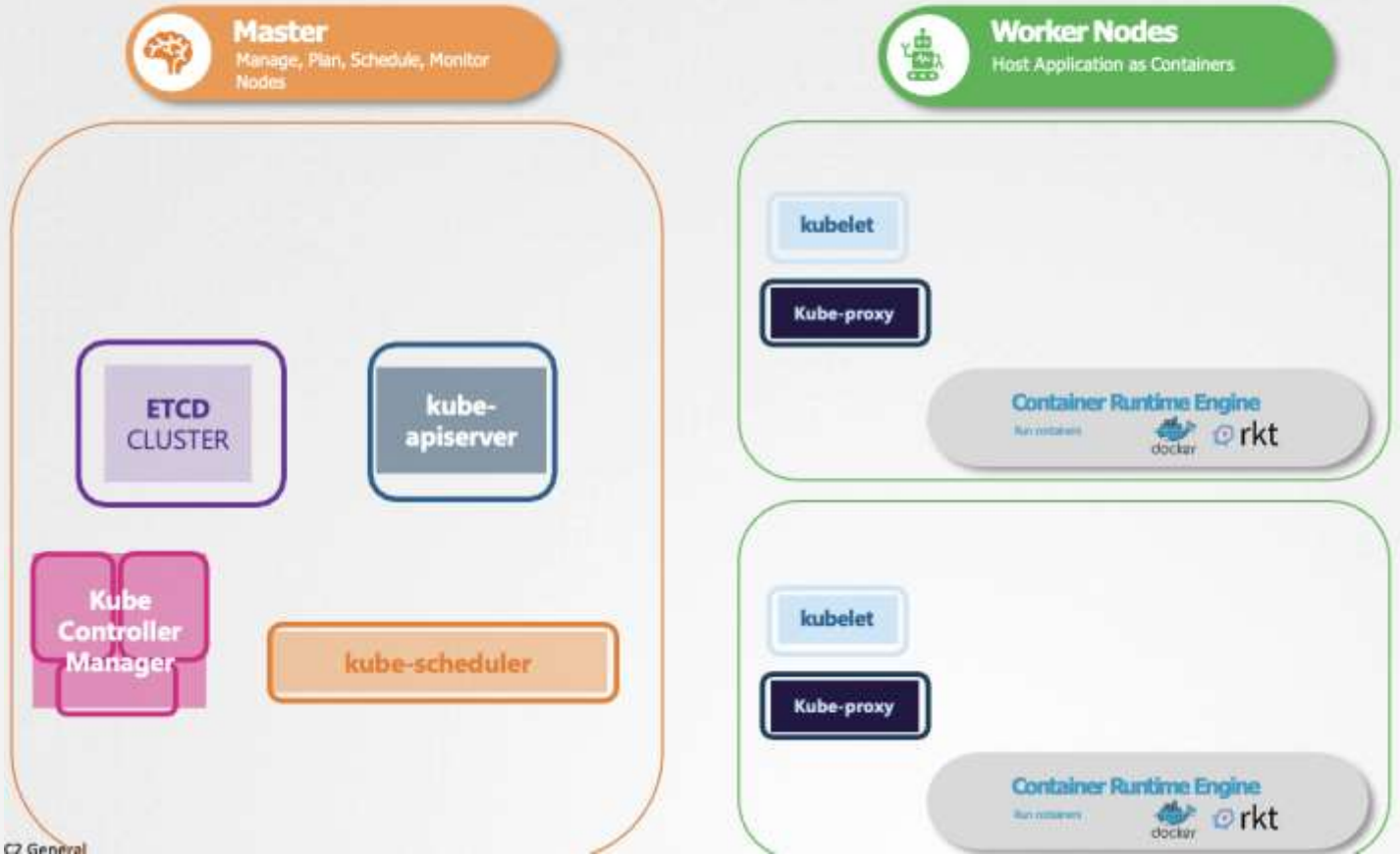

kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
kubelet
Worker Nodes
Host Application as Containers
docker
docker
Controller-Manager
D
CLUSTER
kube-schedu
docker
Container Runtime Engine
Run containers
rkt
• Managening all ativities on these ships
• Make master node know this node is interested to join the cluster
• Receiving information about the containers to be loaded on ship
• Loading appropriate container as required
• Send reports to master about the status of nodes and containers
• Kubelet listen instructions form kube-api and install or destroy containers as required

C2 Gene


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
kubelet
Communication between application on worker nodes is managed by kube-proxy service
Controller-Manager
D
CLUSTER
e-scheduler
Kube-proxy
Container Runtime Engine
Run containers
rkt
docker


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
kubelet
Worker Nodes
Host Application as Containers
Controller -Manager
ETCD CLUSTER
kube-scheduler
Run containers
Kube-proxy
Kube-proxy
Container Runtime Engine
rkt

Kubernetes Architecture



ETCD

FOR BEGINNERS


ETCD is a distributed reliable key-value store that is Simple, Secure & Fast

key-value store

Tabular/Relational Databases

Everytime a new information needed to be added the entire table is afected , and leeds to a lot of empty cells

Name	Age	Location	Salary	Grade
John Doe	45	New York	5000	
Dave Smith	34	New York	4000	
Aryan Kumar	10	New York		A
Lauren Rob	13	Bangalore		C
Lily Oliver	15	Bangalore		B

C2 General

key-value store

Key	Value
Name	John Doe
Age	45
Location	New York
Salary	5000

Key	Value
Name	Dave Smith
Age	34
Location	New York
Salary	4000
Organization	ACME

key-value store inforation on style of pages ,

each page contains specific information about individual

You can update each file indvidually withut update other documents or file

Key	Value
Name	Aryan Kumar
Age	10
Location	New York
Grade	A

Key	Value
Name	Lauren Rob
Age	13
Location	Bangalore
Grade	C

Key	Value
Name	Lily Oliver
Age	15
Location	Bangalore
Grade	B

C2 General

key-value store

```
{
  "name": "John Doe",
  "age": 45,
  "location": "New York",
  "salary": 5000
}
```

```
{
  "name": "Dave Smith",
  "age": 34,
  "location": "New York",
  "salary": 4000,
  "organization": "ACME"
}
```

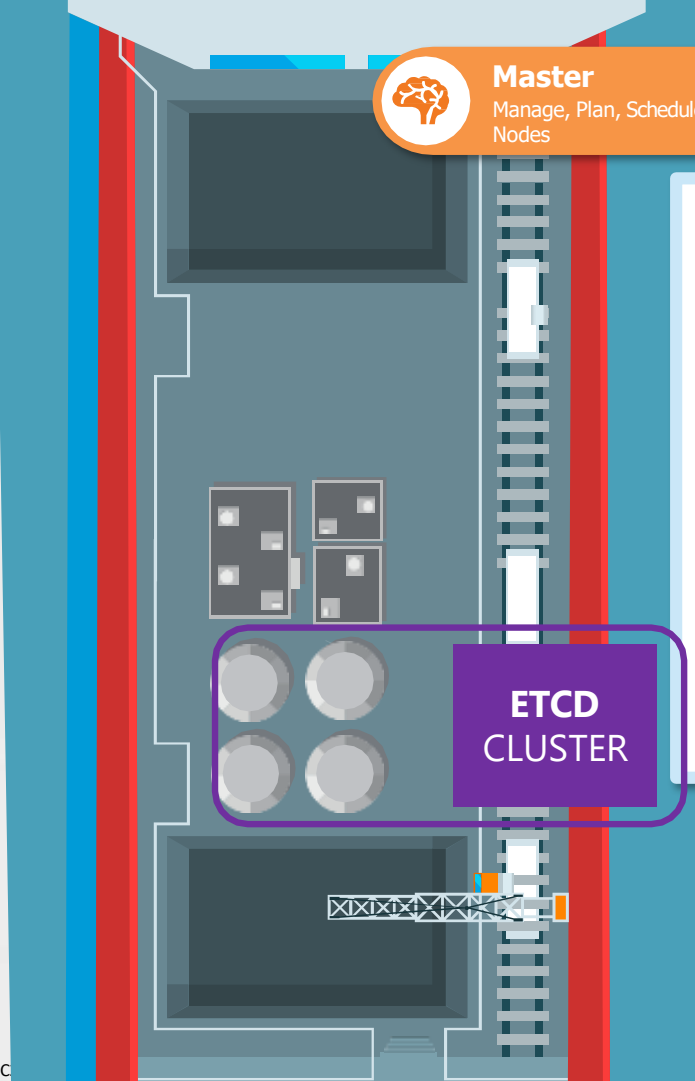
```
{
  "name": "Aryan Kumar",
  "age": 10,
  "location": "New York",
  "Grade": "A"
}
```

```
{
  "name": "Lily Oliver",
  "age": 15,
  "location": "Bangalore",
  "Grade": "B"
}
```

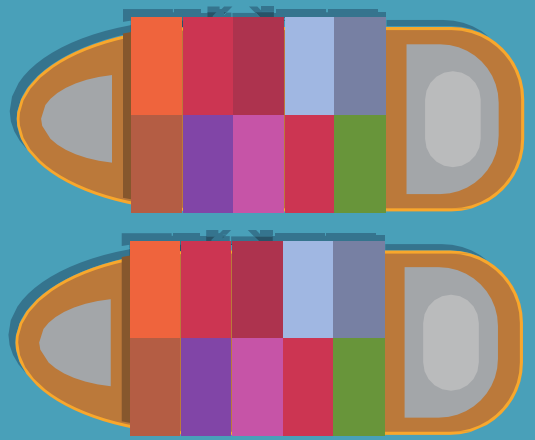
```
{
  "name": "Lauren Rob",
  "age": 13,
  "location": "Bangalore",
  "Grade": "C"
}
```

ETCD

In Kubernetes



- Nodes
- PODs
- Configs
- Secrets
- Accounts
- Roles
- Bindings
- Others



- Every output you see when you run Kubectl command is from ETCD database

- Every change in cluster such as deploying new pods , replicaset ,….are update in etcd

Setup - kubeadm

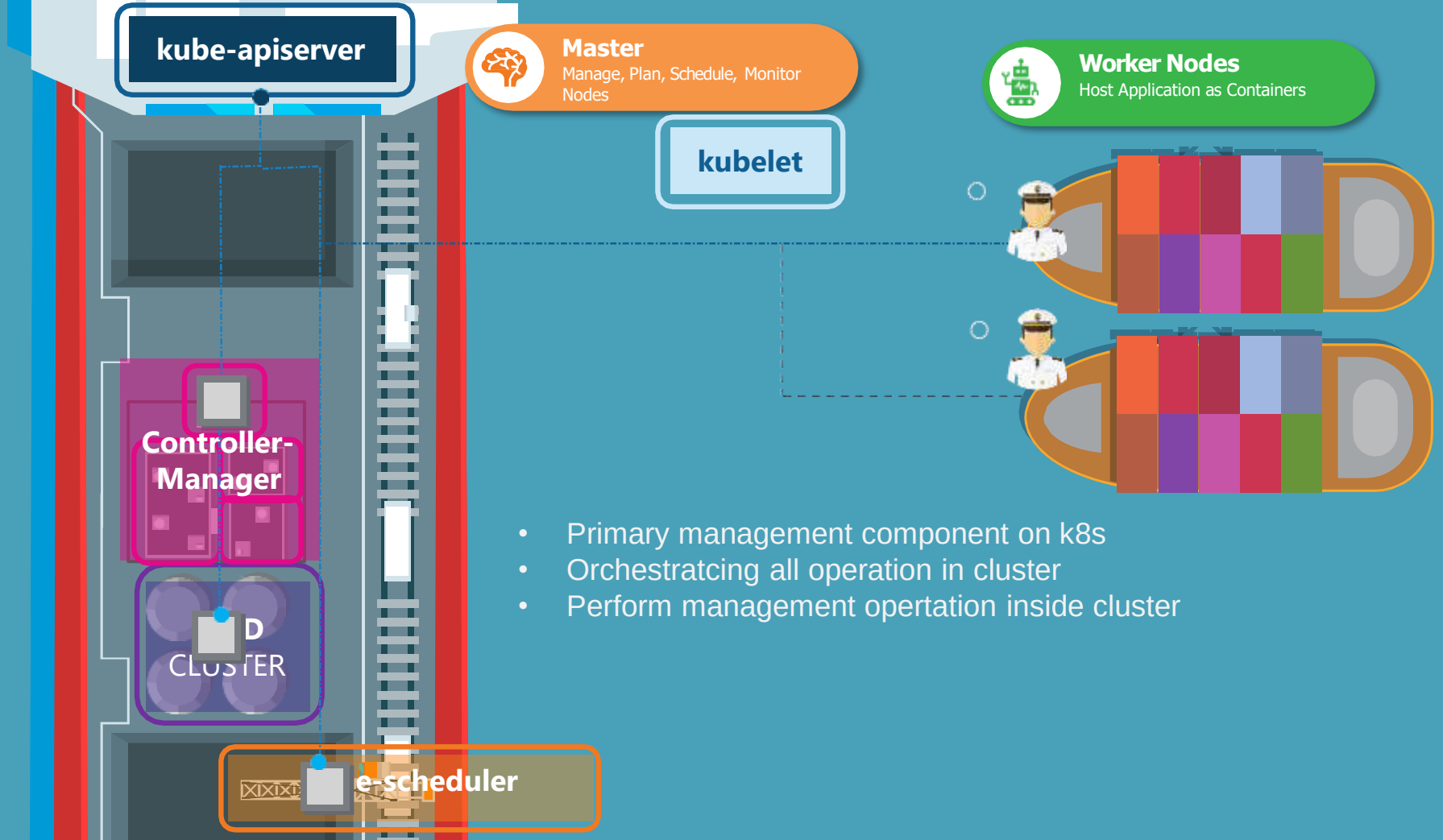
```
kubectl get pods -n kube-system
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-78fcdf6894-prwvl	1/1	Running	0	1h
kube-system	coredns-78fcdf6894-vqd9w	1/1	Running	0	1h
kube-system	etcd-master	1/1	Running	0	1h
kube-system	kube-apiserver-master	1/1	Running	0	1h
kube-system	kube-controller-manager-master	1/1	Running	0	1h
kube-system	kube-proxy-f6k26	1/1	Running	0	1h
kube-system	kube-proxy-hnzsw	1/1	Running	0	1h
kube-system	kube-scheduler-master	1/1	Running	0	1h
kube-system	weave-net-924k8	2/2	Running	1	1h
kube-system	weave-net-hzfcz	2/2	Running	1	1h

Etcd is run as pod inside master node

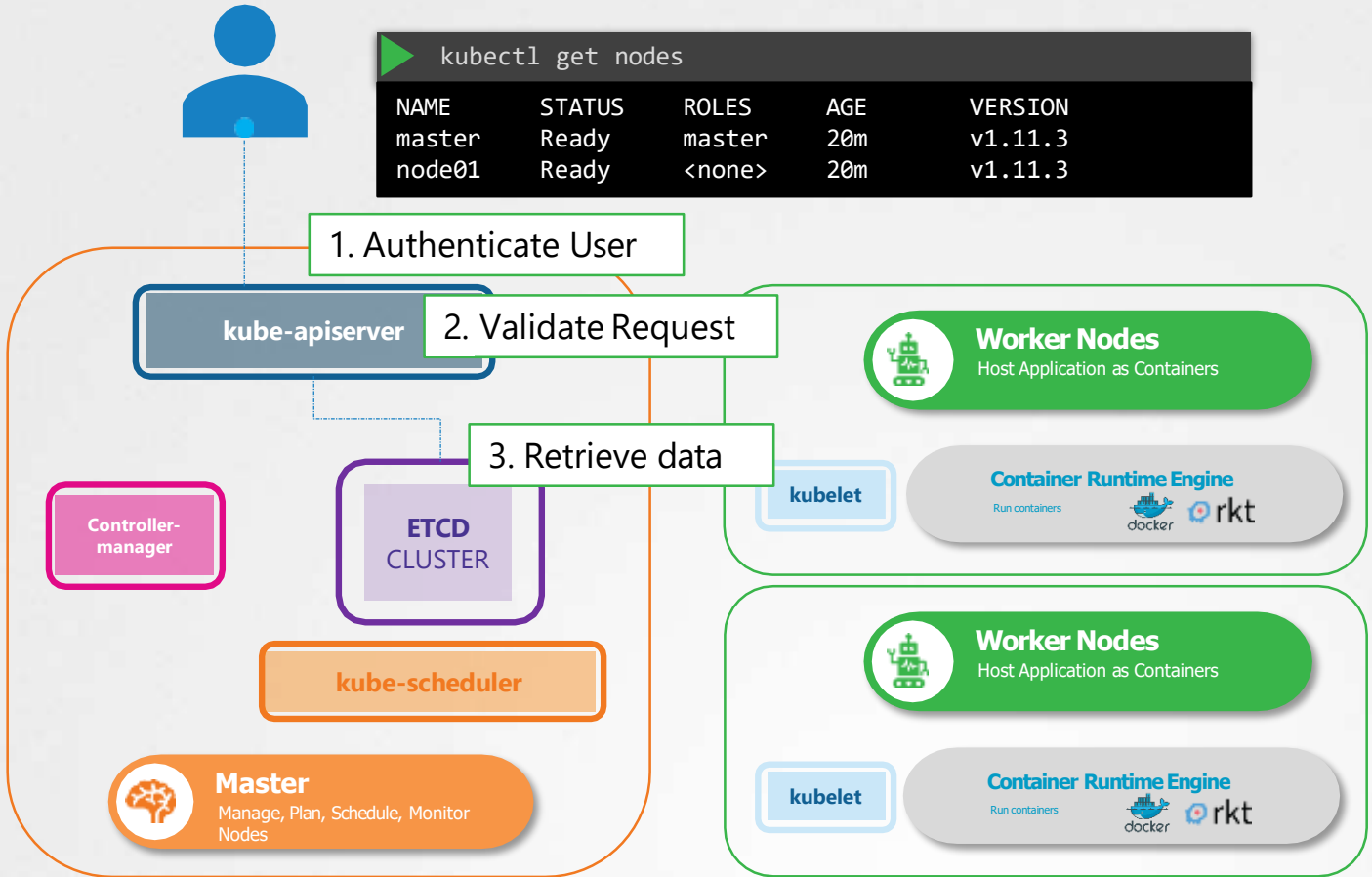
kube-api server





- Primary management component on k8s
- Orchestratcing all operation in cluster
- Perform management opertation inside cluster

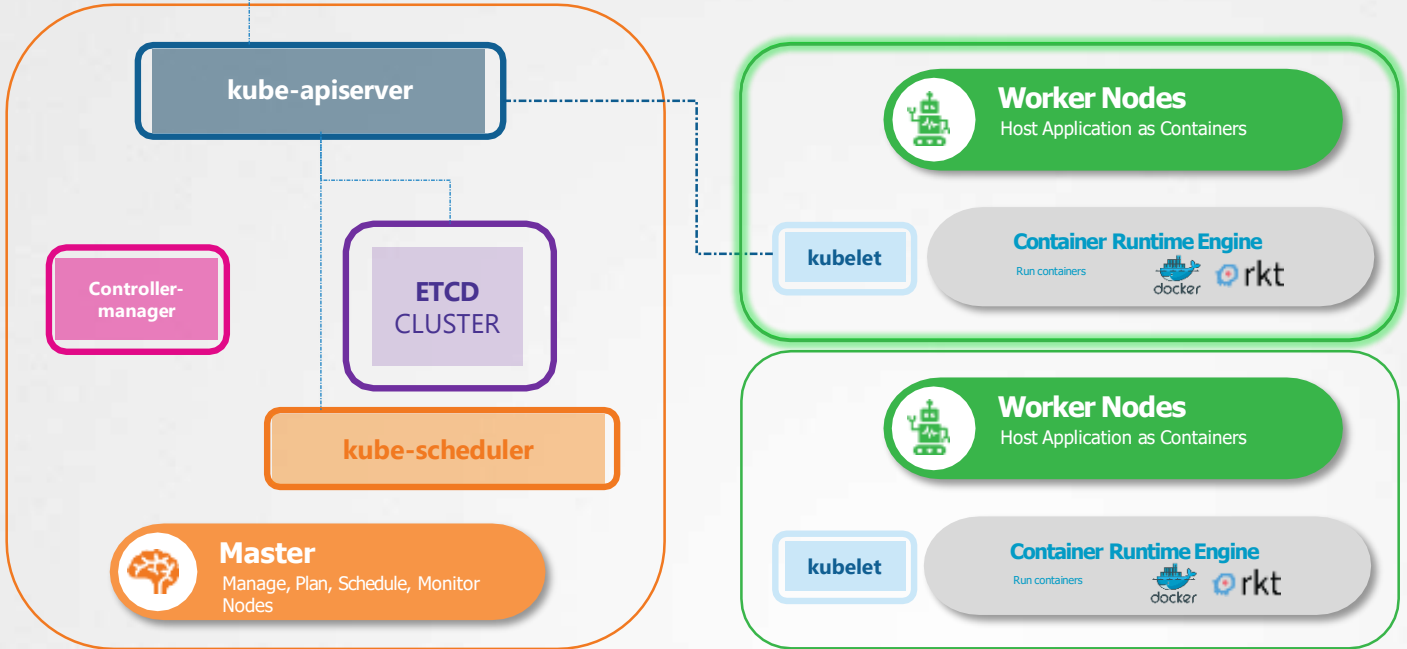
Kubernetes Architecture



Kubernetes Architecture



```
curl -X POST /api/v1/namespaces/default/pods ...[other]
Pod created!
```



1. Authenticate User
2. Validate Request
3. Retrieve data
4. Update ETCD
5. Scheduler
6. Kubelet

Kube-api is the center of different tasks

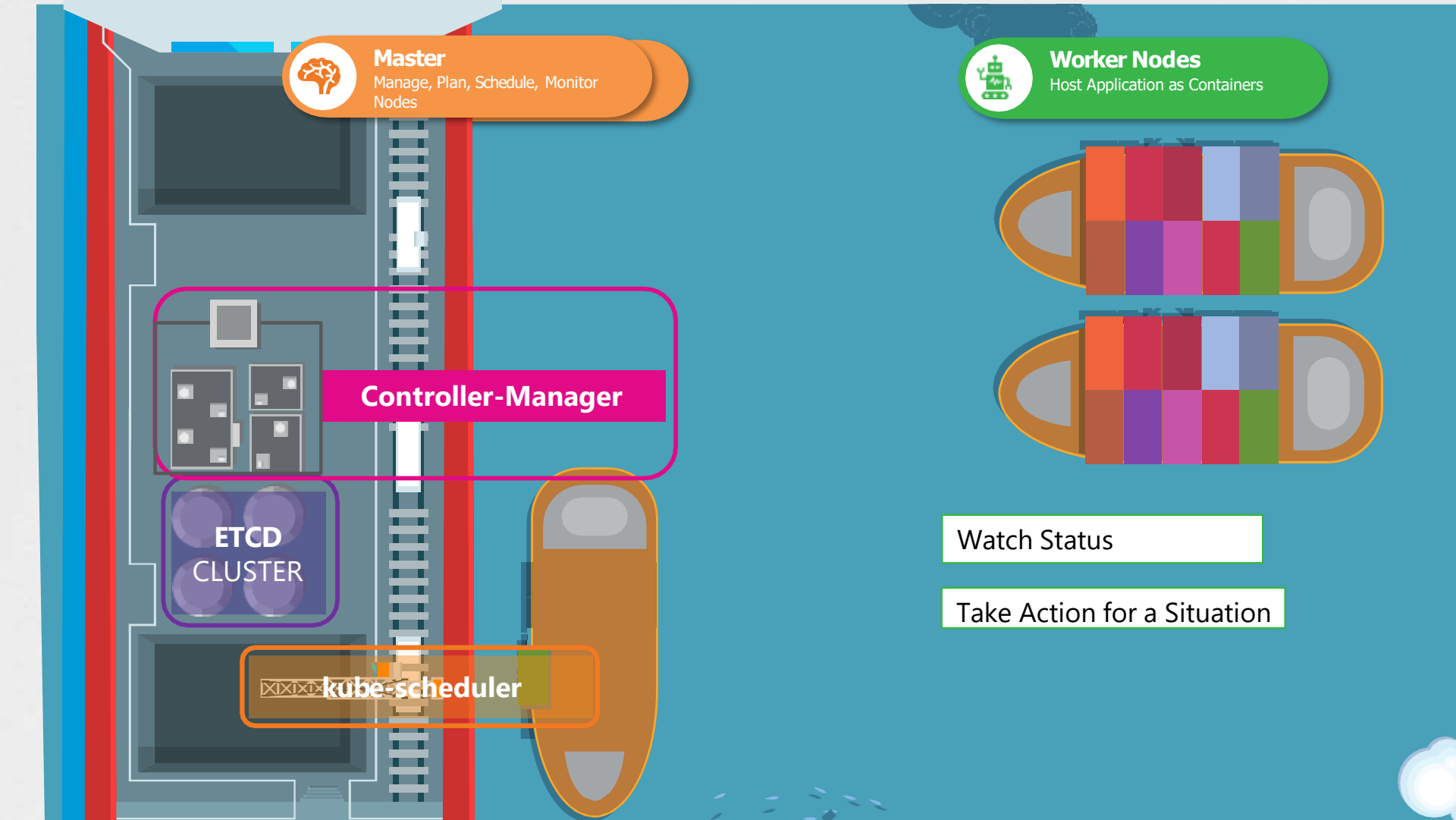
View api-server - kubeadm

```
kubectl get pods -n kube-system
```

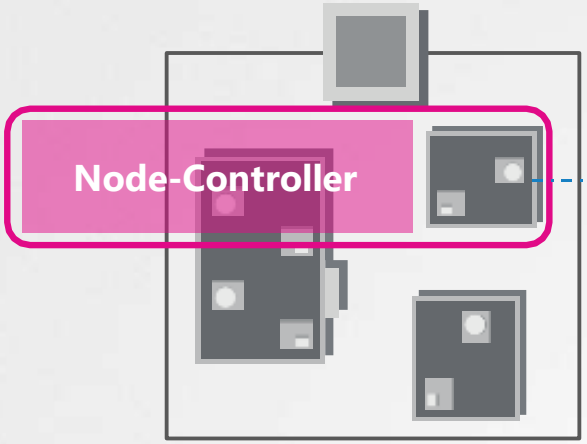
```
NAMESPACE     NAME                              READY     STATUS    RESTARTS   AGE
kube-system   coredns-78fcdf6894-hwrq9          1/1       Running   0          16m
kube-system   coredns-78fcdf6894-rzhjr          1/1       Running   0          16m
kube-system   etcd-master                       1/1       Running   0          15m
kube-system   kube-apiserver-master             1/1       Running   0          15m
kube-system   kube-controller-manager-master    1/1       Running   0          15m
kube-system   kube-proxy-lzt6f                  1/1       Running   0          16m
kube-system   kube-proxy-zm5qd                  1/1       Running   0          16m
kube-system   kube-scheduler-master             1/1       Running   0          15m
kube-system   weave-net-29z42                   2/2       Running   1          16m
kube-system   weave-net-snmdl                   2/2       Running   1          16m    -
```

Kube Controller Manager




Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
Controller-Manager
ETCD
CLUSTER
kube-scheduler
Watch Status
Take Action for a Situation

Controller



Watch Status

Remediate Situation

Status of nodes

Node Monitor Period = 5s

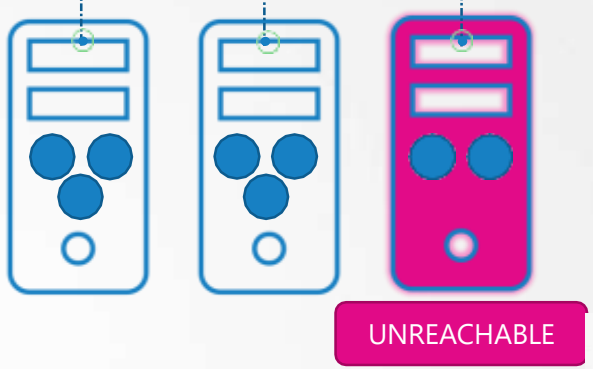
Wait 40 s before mark it unreachable

Node Monitor Grace Period = 40s

kube-apiserver

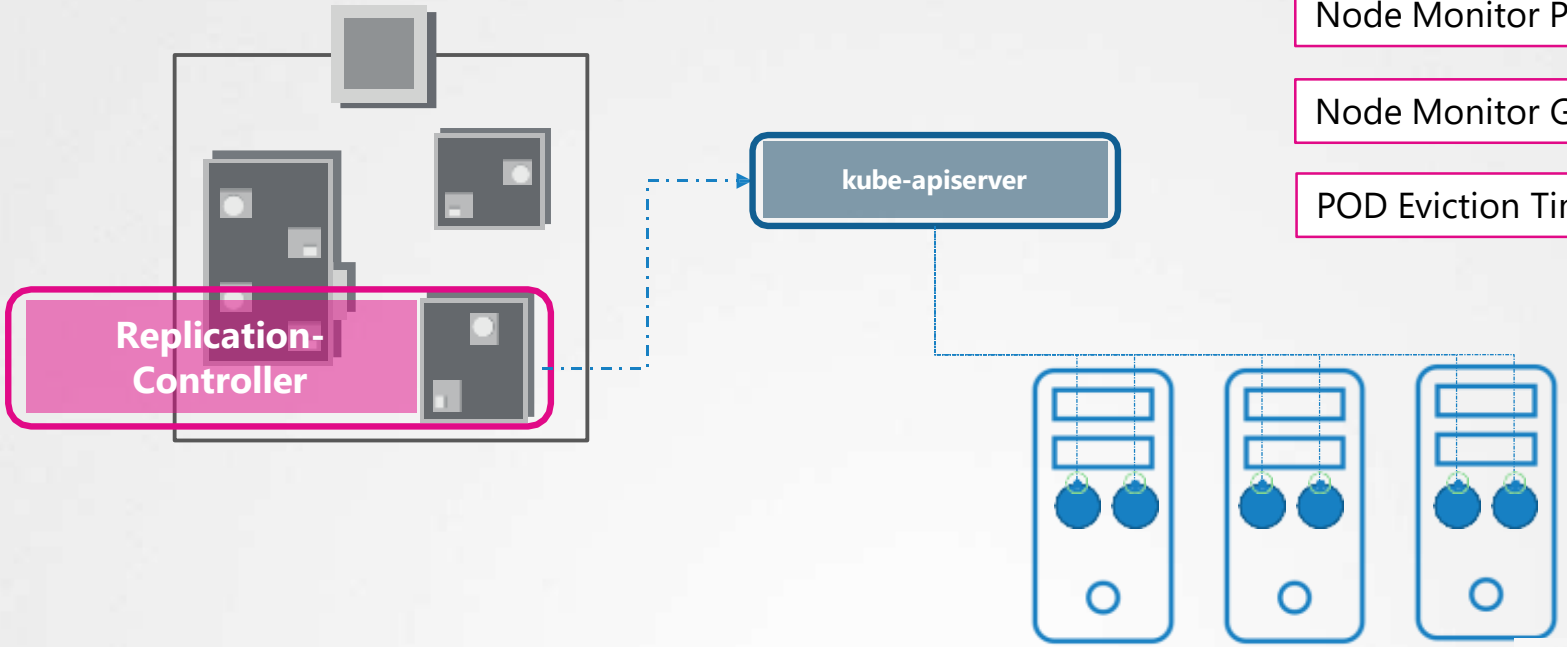
POD Eviction Timeout = 5m

after node is marked as not reachable
Give the node 5 m to be up if not will
recreate pods on a healthy node




```
kubectl get nodes
NAME       STATUS     ROLES    AGE   VERSION
worker-1   Ready      <none>   8d    v1.13.0
worker-2   NotReady   <none>   8d    v1.13.0
```

Controller



Watch Status

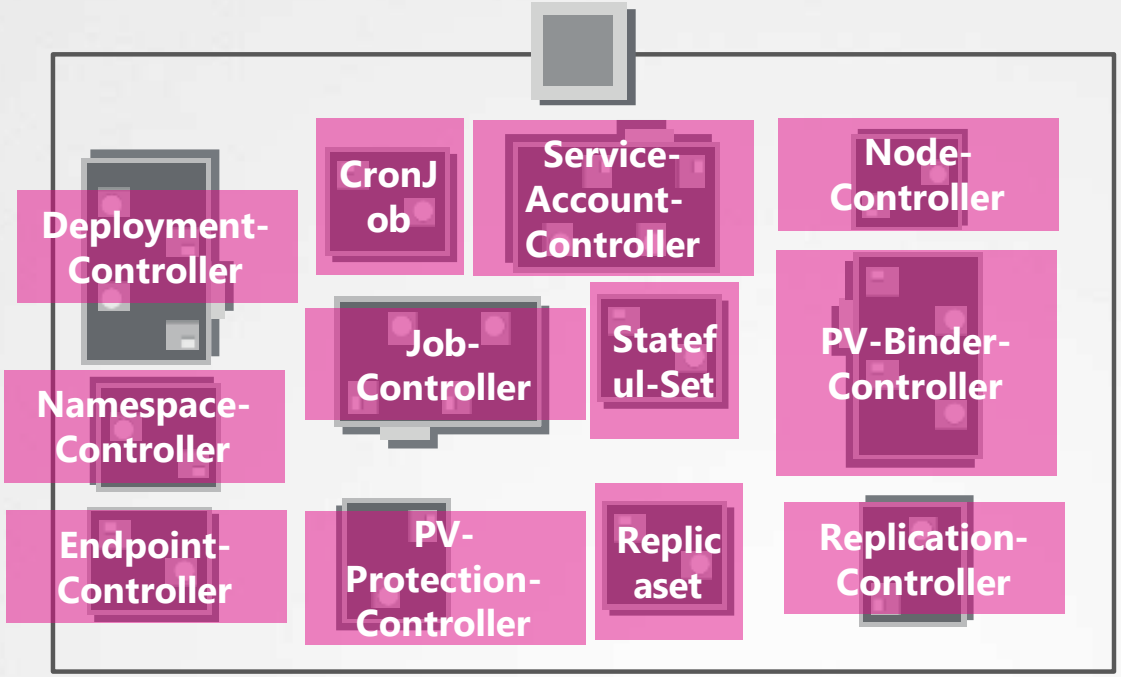
Remediate Situation

Node Monitor Period = 5s

Node Monitor Grace Period = 40s

POD Eviction Timeout = 5m

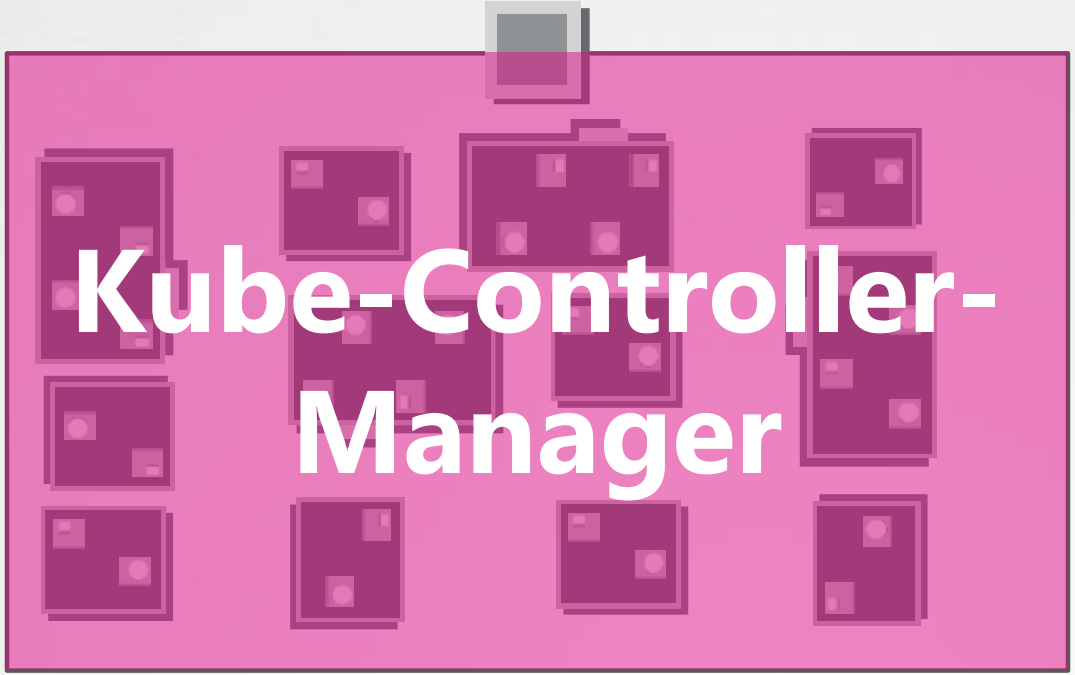
Controller



- Watch Status
- Remediate Situation
- Node Monitor Period = 5s
- Node Monitor Grace Period = 40s
- POD Eviction Timeout = 5m

Controller

All previous controllers are stored in one service called kube-controller-manager



- Watch Status
- Remediate Situation
- Node Monitor Period = 5s
- Node Monitor Grace Period = 40s
- POD Eviction Timeout = 5m

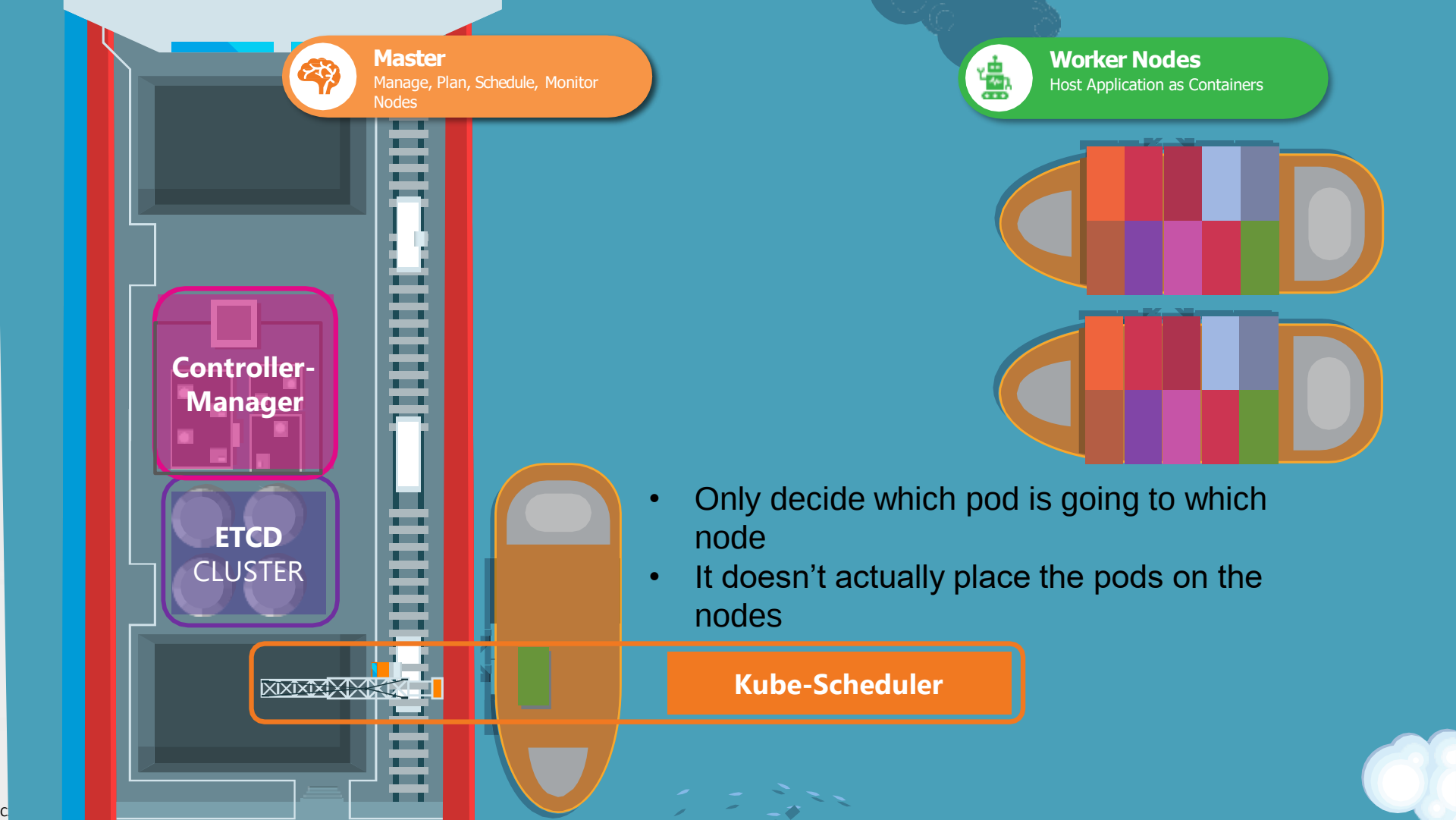
View kube-controller-manager - kubeadm

```
kubectl get pods -n kube-system
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-78fcdf6894-hwrq9	1/1	Running	0	16m
kube-system	coredns-78fcdf6894-rzhjr	1/1	Running	0	16m
kube-system	etcd-master	1/1	Running	0	15m
kube-system	kube-apiserver-master	1/1	Running	0	15m
kube-system	kube-controller-manager-master	1/1	Running	0	15m
kube-system	kube-proxy-lzt6f	1/1	Running	0	16m
kube-system	kube-proxy-zm5qd	1/1	Running	0	16m
kube-system	kube-scheduler-master	1/1	Running	0	15m
kube-system	weave-net-29z42	2/2	Running	1	16m
kube-system	weave-net-snmdl	2/2	Running	1	16m

Kube Scheduler





- Only decide which pod is going to which node
- It doesn't actually place the pods on the nodes

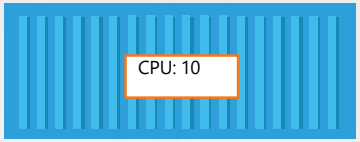
Kube-Scheduler

Have different ships , schedular choose which ship can have appropriate capacity
The ship must have required recourses to load the container



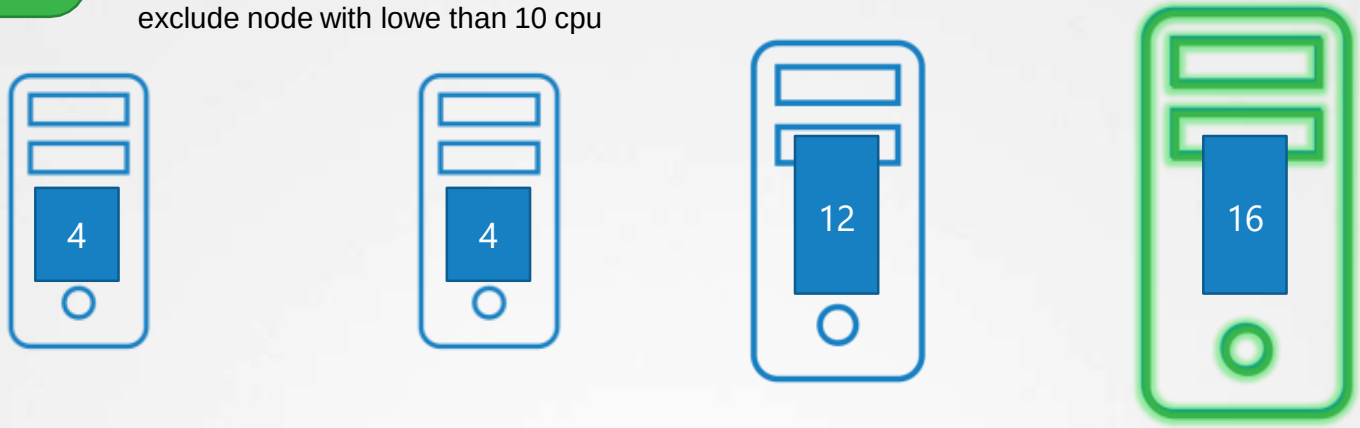
Kube-Scheduler

HOW Schedular works :



1. Filter Nodes

Filter node and exclude nodes with low critera for example above container neeed 10 cputhen it will exclude node with lowe than 10 cpu



2. Rank Nodes

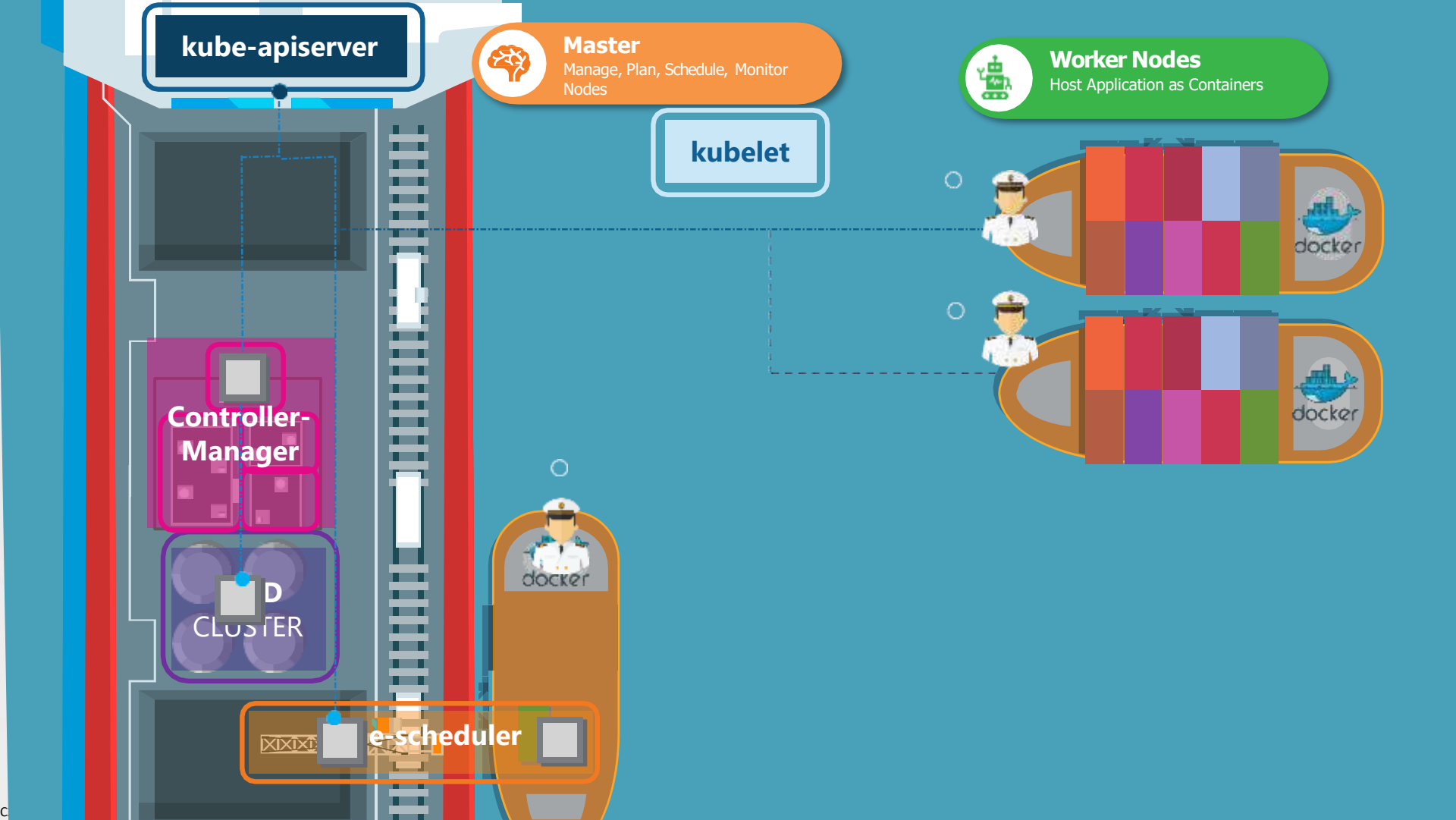
After exclude unusful nodes it make a rank for other for example :
The last 2 nodes will have 2 cpu and 6 cpu after we add the container so the last one will win

More Later…

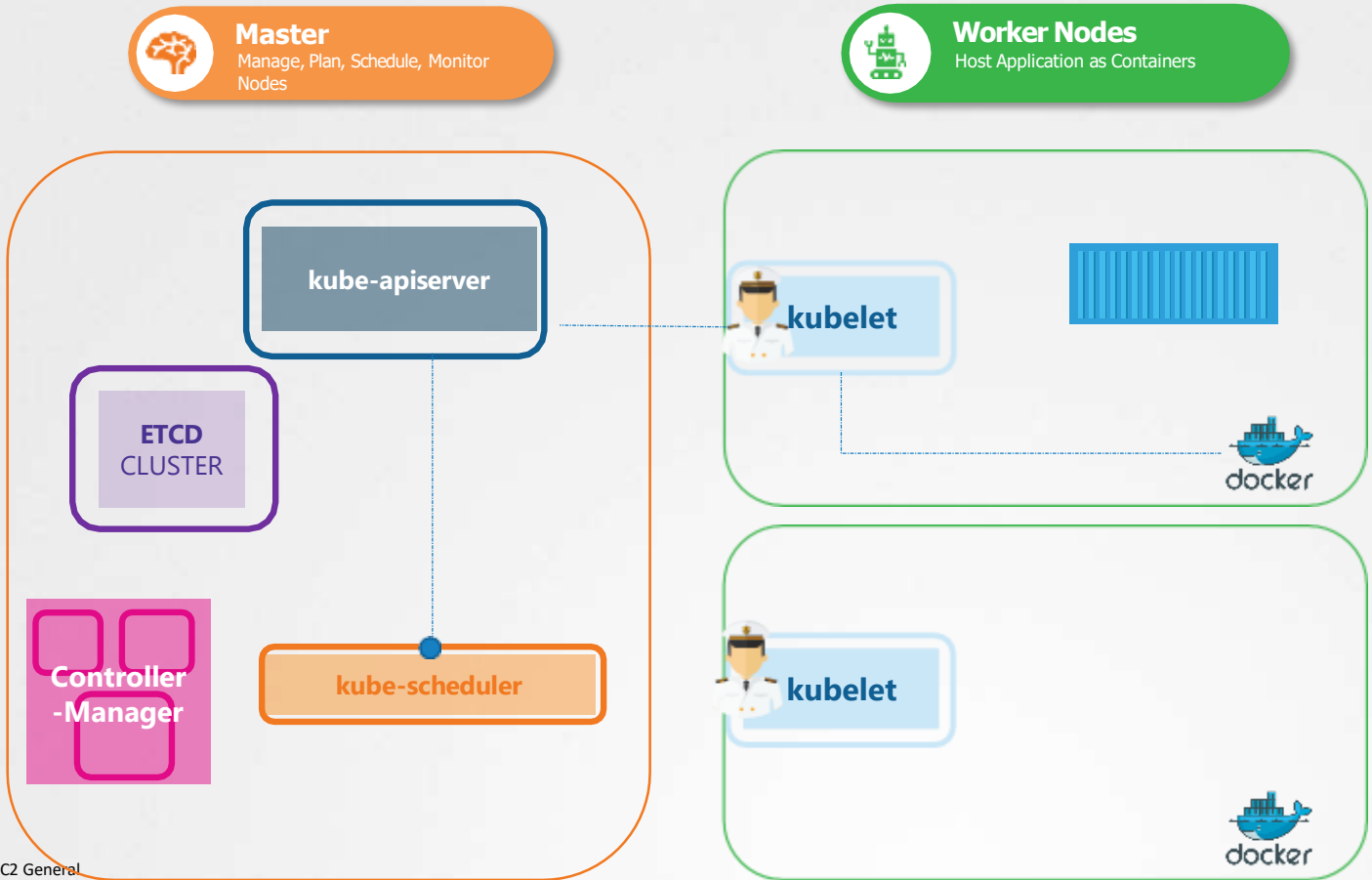
You can customize your own schedular by using the following features :

- Resource Requirements and Limits
- Taints and Tolerations
- Node Selectors/Affinity

Kubelet


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
kubelet
Worker Nodes
Host Application as Containers
docker
docker
Controller-Manager
D
CLUSTER
docker
e-scheduler

Kubernetes Architecture



Register Node

Create PODs

Monitor Node & PODs

C2 General

Installing kubelet

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kubelet
```

kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --config=/var/lib/kubelet/kubelet-config.yaml \\
  --container-runtime=remote \\
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \\
  --image-pull-progress-deadline=2m \\
  --kubeconfig=/var/lib/kubelet/kubeconfig \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```

- Kubelet not installed automatikly with kubeadm
  You have to install manually on all worker nodes

- It installed as a normal service



Kubeadm does not deploy Kubelets

View kubelet options

to check the service on minikube worker node

```
[root@master ~]# docker exec -it  minikube-m02 systemctl status kubelet
● kubelet.service - kubelet: The Kubernetes Node Agent
     Loaded: loaded (/lib/systemd/system/kubelet.service; enabled; vendor preset: enabled)
    Drop-In: /etc/systemd/system/kubelet.service.d
             └─10-kubeadm.conf
     Active: active (running) since Thu 2023-02-02 12:54:07 UTC; 3h 32min ago
       Docs: http://kubernetes.io/docs/
   Main PID: 942 (kubelet)
      Tasks: 15 (limit: 3399)
     Memory: 53.7M
        CPU: 1min 23.338s
     CGroup: /system.slice/kubelet.service
             └─942 /var/lib/minikube/binaries/v1.24.3/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --config=/var/lib/kubelet/config.yaml --
co
ntainer-runtime=remote --container-runtime-endpoint=/var/run/cri-dockerd.sock --hostname-override=minikube-m02 --image-service-endpoint=/var/run/cri-dockerd.sock --
kubeconfig=/etc/kubernetes/kubelet.conf --node-ip=192.168.49.3 --runtime-request-timeout=15m

Feb 02 15:39:07 minikube-m02 kubelet[942]: W0202 15:39:07.530667     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 15:44:07 minikube-m02 kubelet[942]: W0202 15:44:07.530656     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 15:49:07 minikube-m02 kubelet[942]: W0202 15:49:07.530322     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 15:54:07 minikube-m02 kubelet[942]: W0202 15:54:07.530972     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 15:59:07 minikube-m02 kubelet[942]: W0202 15:59:07.532154     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 16:04:07 minikube-m02 kubelet[942]: W0202 16:04:07.530920     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 16:09:07 minikube-m02 kubelet[942]: W0202 16:09:07.530828     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 16:14:07 minikube-m02 kubelet[942]: W0202 16:14:07.530152     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 16:19:07 minikube-m02 kubelet[942]: W0202 16:19:07.531732     942 machine.go:65] Cannot read vendor id correctly, set empty.
Feb 02 16:24:07 minikube-m02 kubelet[942]: W0202 16:24:07.530830     942 machine.go:65] Cannot read vendor id correctly, set empty.
[root@master ~]#
```

C2 General

Minikube



Kubectl and minikube local installation.

1) Create user in wheel group
$useradd –G wheel ahmed

2) Install docker
https://docs.docker.com/engine/install/centos/
3) Add your user to docker group
```
$sudo usermod -aG docker $USER && newgrp docker
```

4) minikube on Linux/Windows/Mac
https://minikube.sigs.k8s.io/docs/start/

5) Start minikube
$ minikube start

6) Install kubectl on Linux/Windows/Mac
https://kubernetes.io/docs/tasks/tools/#install-kubectl.

Lets Started ,

Install minikube



C2 General

Kubernetes Objects



Pods

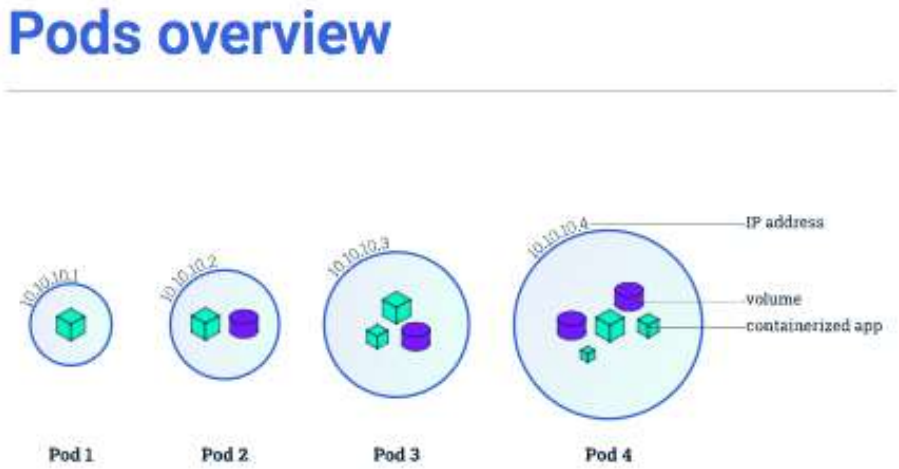


Kubernetes main objects

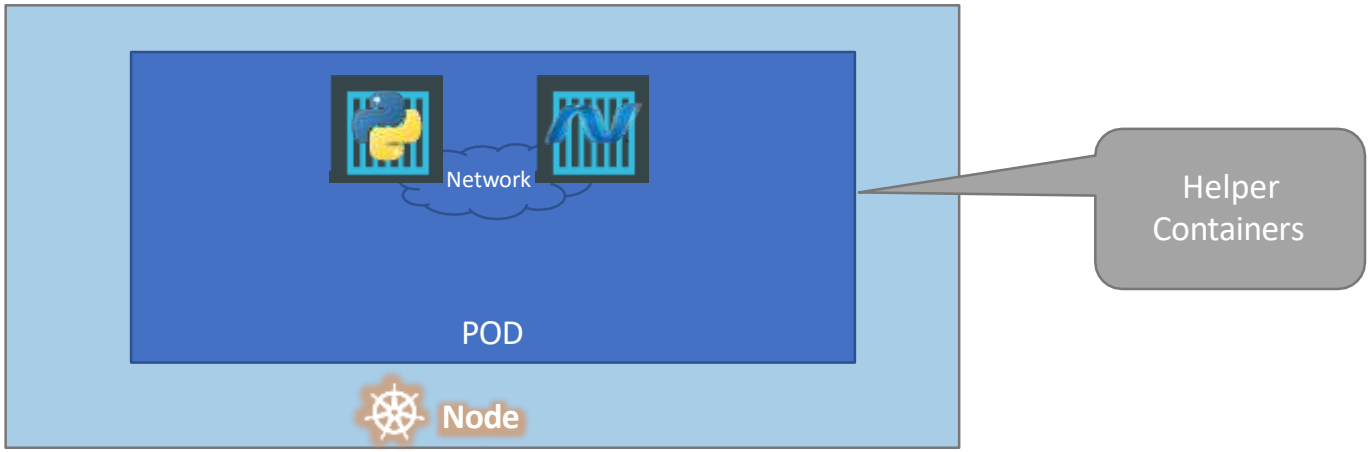
Pod

It represents a unit of deployment: a single instance of an application in Kubernetes, which might consist of either a single container or a small number of containers.

It encapsulates an application container (or, in some cases, multiple containers), storage resources, a unique network IP, and options that govern how the container(s) should run.



Multi-Container PODs



Kubernetes Resources creation



- We can create resources by either using YAML files (declarative) or imperative commands.

Imperative	**Declarative (YAML file) Kubectl create -f filename**
Easier	More complex
Faster	Take more time
Limited (not all specs can be written in an imperative command)	All the advanced specs can be written declaratively in YAML

Basic Format Yaml

```yaml
apiVersion: v1
kind: Pod
Metadata:
  name: myapp-rc
  labels:
      app: myapp

spec:
```

apiVersion → Version number of object

Can list all api versions :
$ kubectl api-versions

Can list all version with resources :
$ kubectl api-resources

Kind → Type of object

Metadata → information about the object such as name , labels ,annotations ,…

Spec → Contains pod configurations like container , image ,volumes ,…

Note: take care about the indentations

Example of pod creation using Declaritive :

- pod-name: myapp
- Image: nginx

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

```
replicationcontroller “myapp-rc” created
```

List applied pod :

```
> kubectl get pods
NAME            READY     STATUS    RESTARTS   AGE
myapp-rc-4lvk9  1/1       Running   0          20s
myapp-rc-mc2mf  1/1       Running   0          20s
myapp-rc-px9pz  1/1       Running   0          20s
```

Lets Play with Pods

1. Create pod in Declarative and imperative mode
2. List all pods in default namespace
3. Labels pods
4. Check status of pod
5. Delete pods
6. Describe pods
7. Edit pod by Changing image name of pod
8. Get image name of pod

C2 General

Pod Imperative Commands

- Creation of pod in certain namespace

kubectl run POD_NAME --image=IMAGE_NAME -n NAMESPACE_NAME

- Listing all pods in certain namespace

kubectl get pod -n NAMESPACE_NAME

- Listing all pods in all namespaces

kubectl get pod –all-namespaces

- Describing pod in certain namespace

kubectl describe pod POD_NAME -n NAMESPACE_NAME

- Getting logs of pod in certain namespace

kubectl logs POD_NAME -n NAMESPACE_NAME

- Getting logs of certain container inside multi-container pod in certain namespace

kubectl logs POD_NAME –c CONTAINER_NAME -n NAMESPACE_NAME

- Executing inside pod in certain namespace

kubectl exec -it POD_NAME bash

- Deleting pod in certain namespace

kubectl delete pod POD_NAME -n NAMESPACE_NAME

ReplicaSets

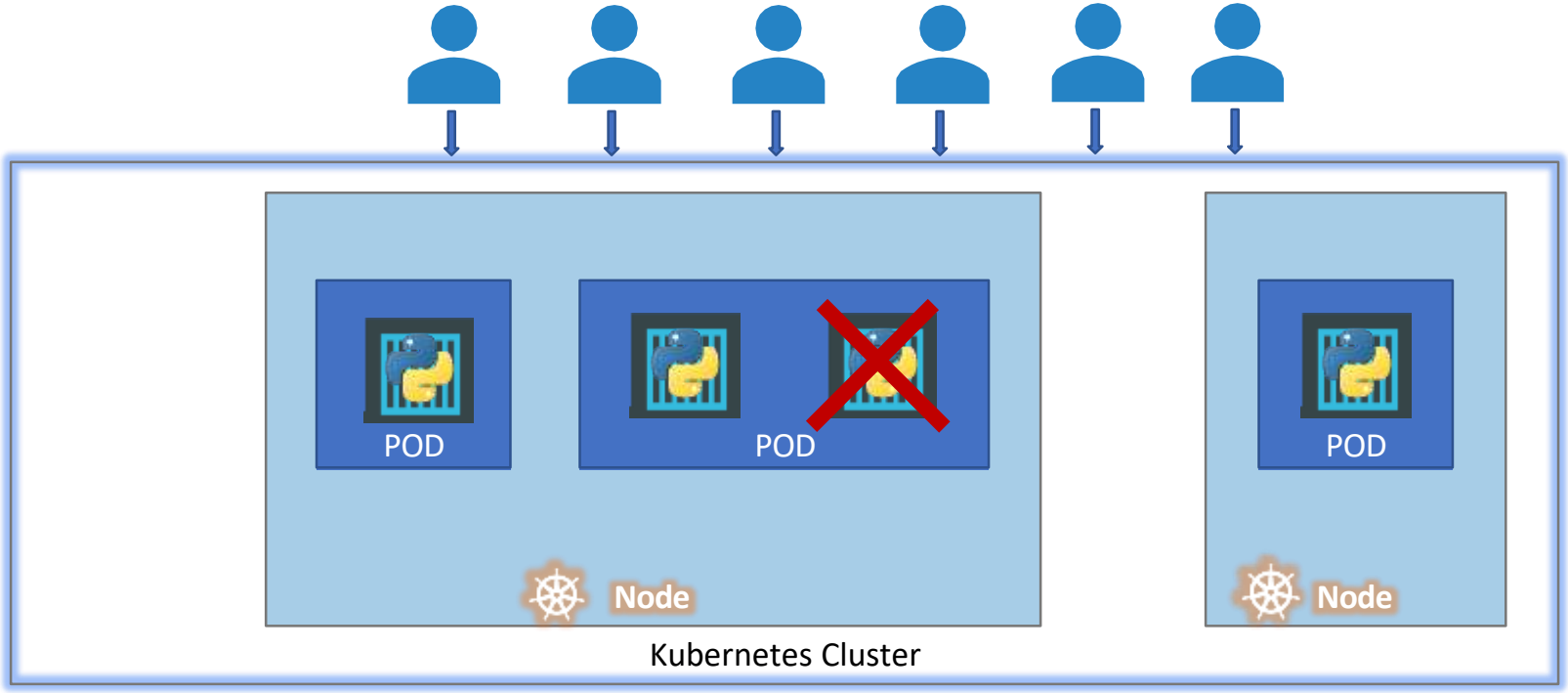


POD

What if we need to scale our app ?

Do we will add new container in the same pod , NO ✖

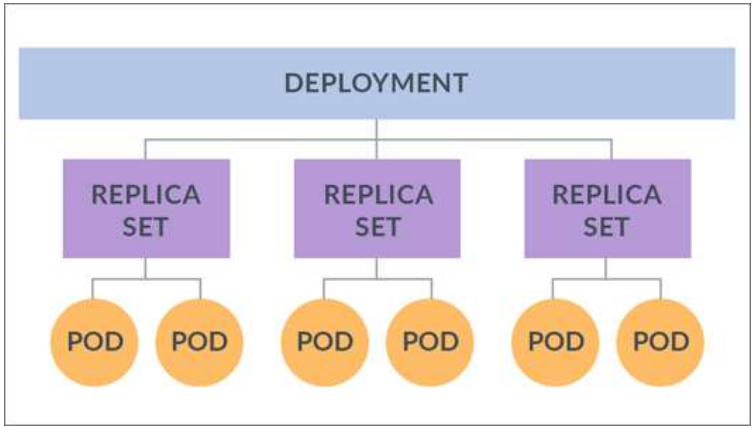
Will create another Pod and container inside it



Kubernetes main objects

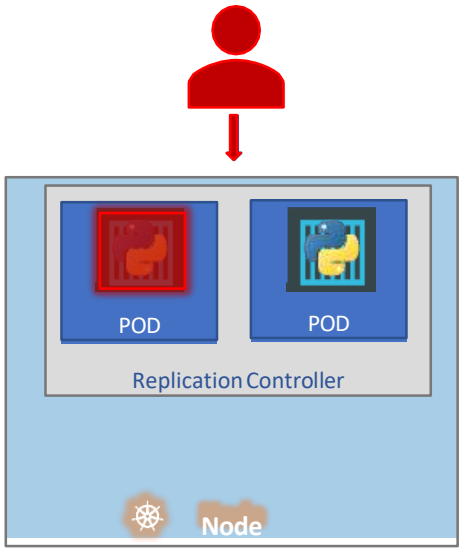
Replicasets:

- Allows you to have high availability of your application
  - More than replica for the same application
- Can have only one replica in replicaset
  - Allows you to maintain your app live
  - Restart pod if pod fails
- Create multiple pods to share the load between them

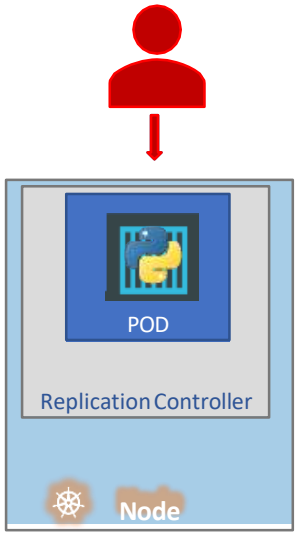



```
[saad@Master examples]$ kubectl get replicaset
NAME              DESIRED   CURRENT   READY   AGE
app-5b5d49f5bd    3         3         3       10d
mongo-7db964487   1         1         0       10d
```

High Availability



Can have more than instance for your app



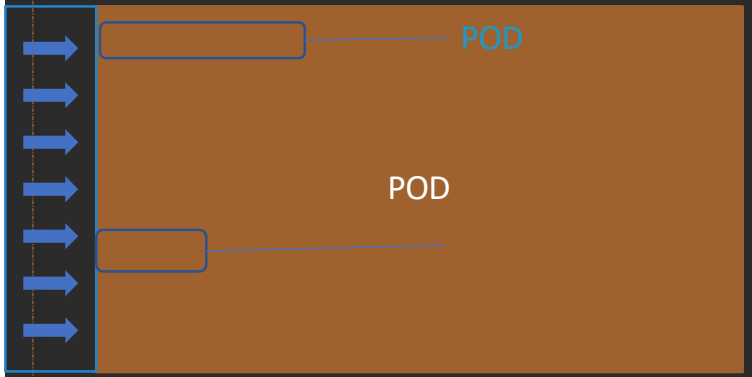
Can also have only one instance inside replicaset
Restart pod if fails‘

rc-definition.yml

```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
      app: myapp
      type: front-end
spec:
  template:
```

Replication Controller

Replication Controller




```yaml
  replicas: 3
```

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

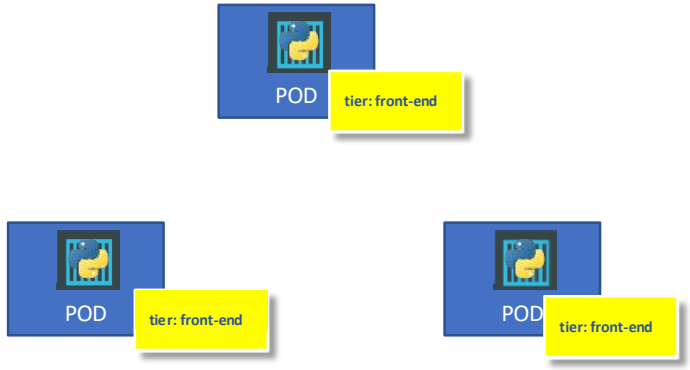
```
replicationcontroller "myapp-rc" created
```

```
> kubectl get replicationcontroller
NAME       DESIRED          CURRENT        READY          AGE
myapp-rc    3               3              3              19s
```

```
> kubectl get pods
NAME             READY      STATUS     RESTARTS   AGE
myapp-rc-4lvk9   1/1        Running    0          20s
myapp-rc-mc2mf   1/1        Running    0          20s
myapp-rc-px9pz   1/1        Running    0          20s
```

replicaset-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type:front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```



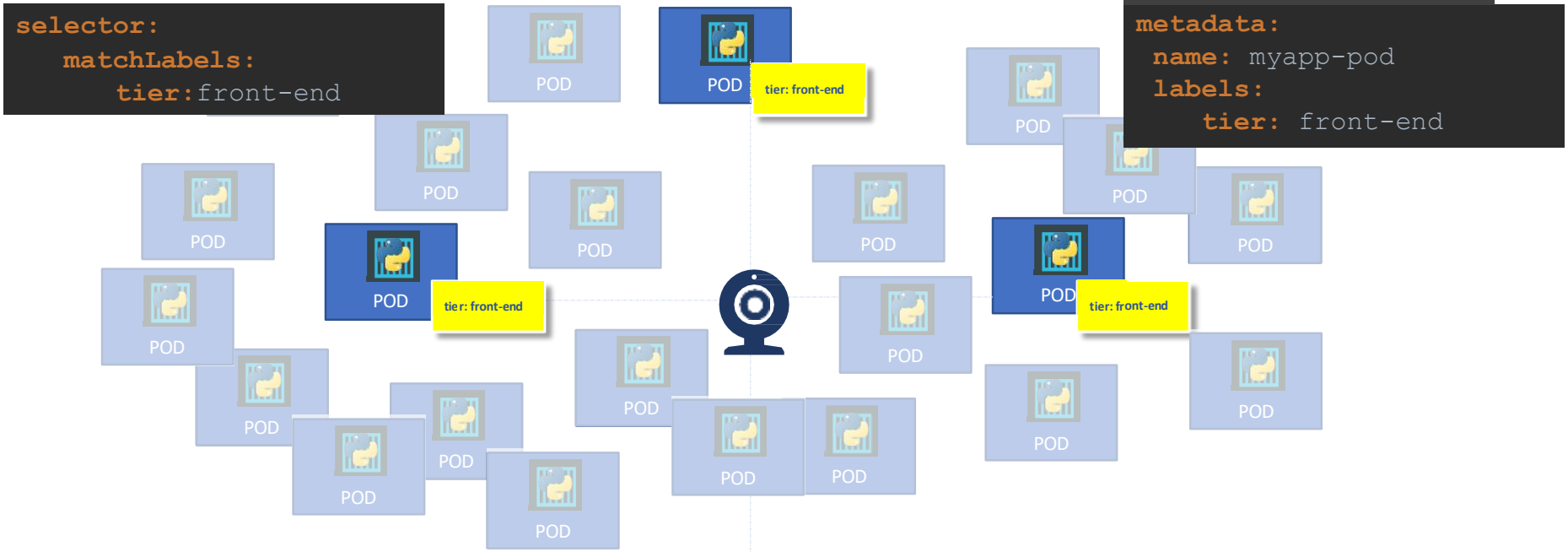
Labels and Selectors

replicaset-definition.yml

```
selector:
   matchLabels:
      tier:front-end
```

pod-definition.yml

```
metadata:
  name: myapp-pod
  labels:
     tier: front-end
```



Deployments



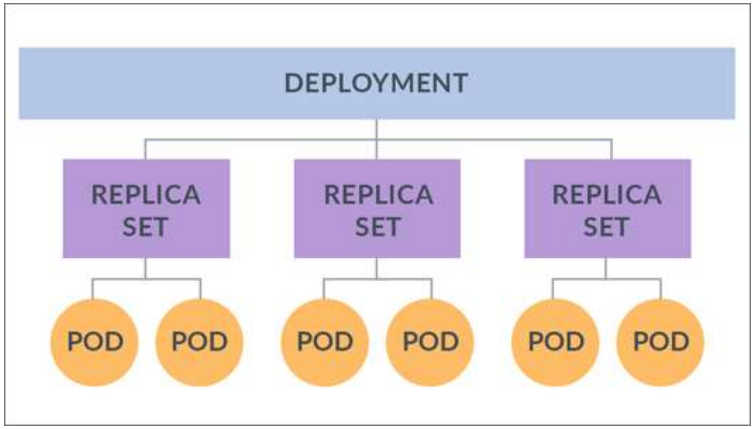
Kubernetes main objects

Deployment

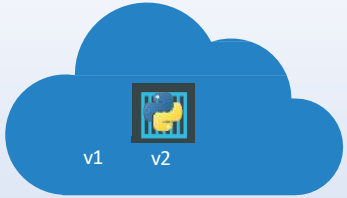
- It allows you to describe an application's life cycle, such as :

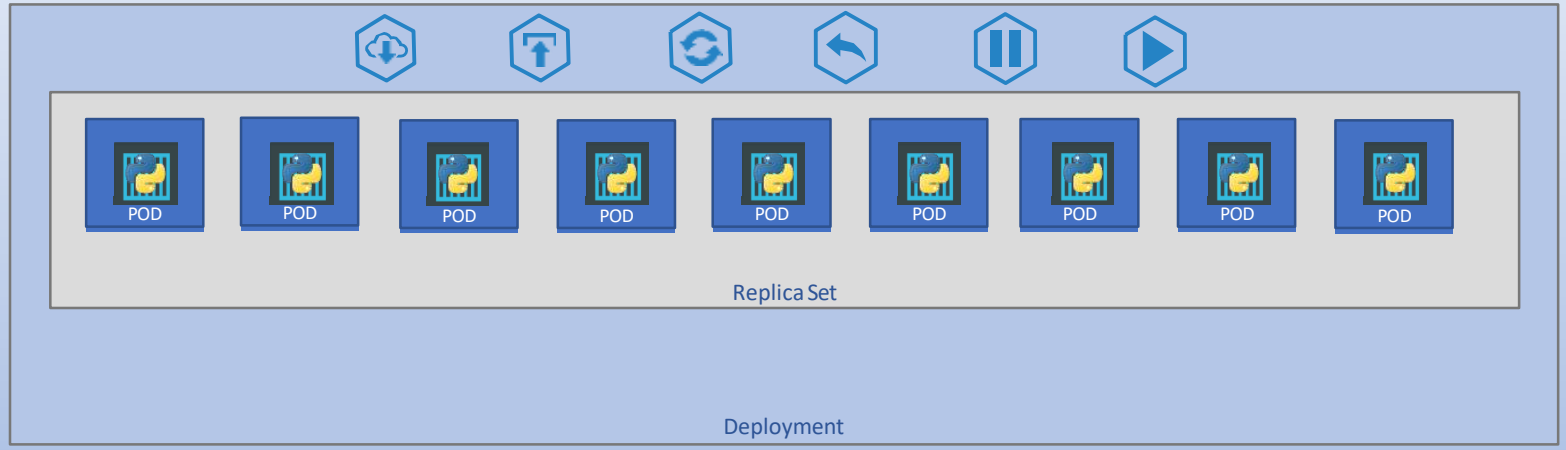
1. Which images to use for the app,
2. number of pods there should be
3. The way in which they should be updated.

- Deploy a replica set or pod
- Update pods and replica sets
- Rollback to previous deployment versions
- Scale a deployment
- Pause or continue a deployment



Deployment





Pod of Deployment are created inside Replicaset
Using Deployment you can rolling update and scale your application

Definition

The same definition file of ReplicaSet Except for Kind

```
> kubectl create -f deployment-definition.yml
deployment "myapp-deployment" created
```

```
> kubectl get deployments
NAME               DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
myapp-deployment   3         3         3            3           21s
```

```
> kubectl get replicaset
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-6795844b58   3         3         3       2m
```

```
> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myapp-deployment-6795844b58-5rbjl   1/1     Running   0          2m
myapp-deployment-6795844b58-h4w55   1/1     Running   0          2m
myapp-deployment-6795844b58-lfjhv   1/1     Running   0          2m
```

deployment-definition.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
      app: myapp
      type: front-end
spec:
  template
      name: myapp-pod
      labels:
          app: myapp
          type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx

  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

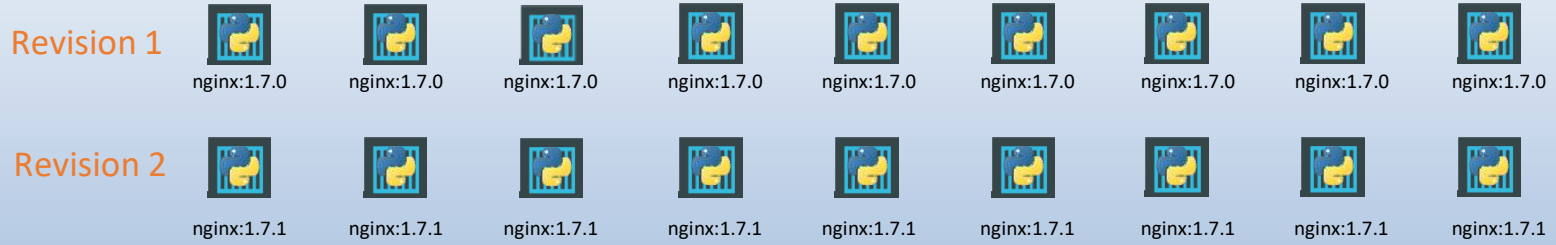
Lets Play with Deployments

1. Create Deployments in declarative and imperative way
2. Edit image name
3. Scale deployments

Rollout and Versioning

Change container image to nginx:1.7.1





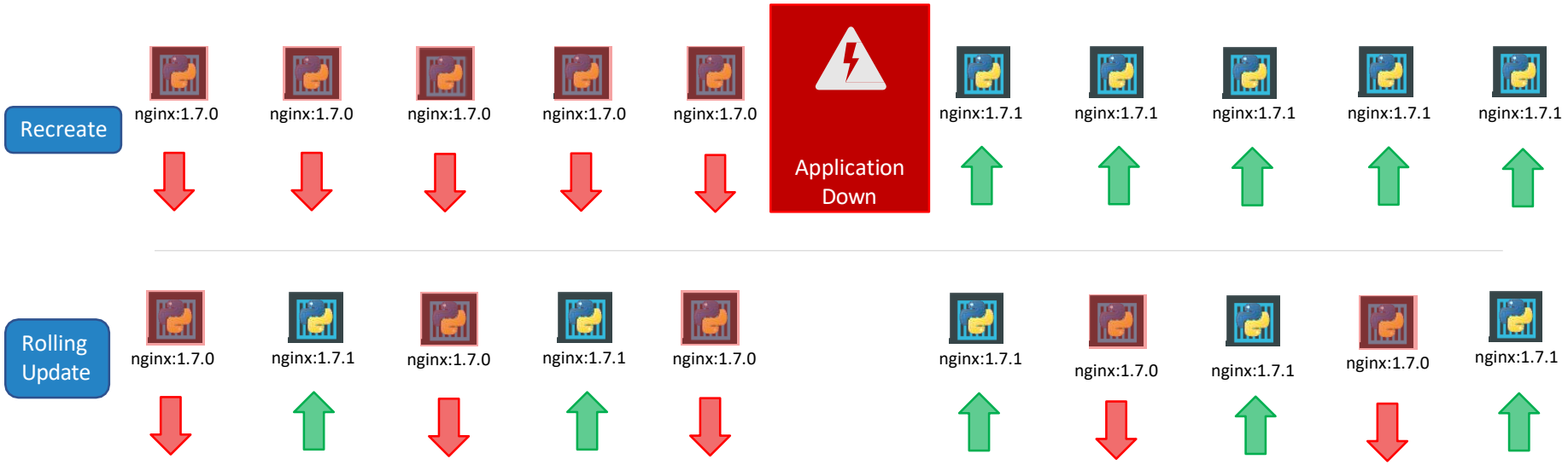

```
kubectl set image deployment myapp-deployment container-name=nginx:1.7.1 –record=true

kubectl rollout history deployment myapp-deployment

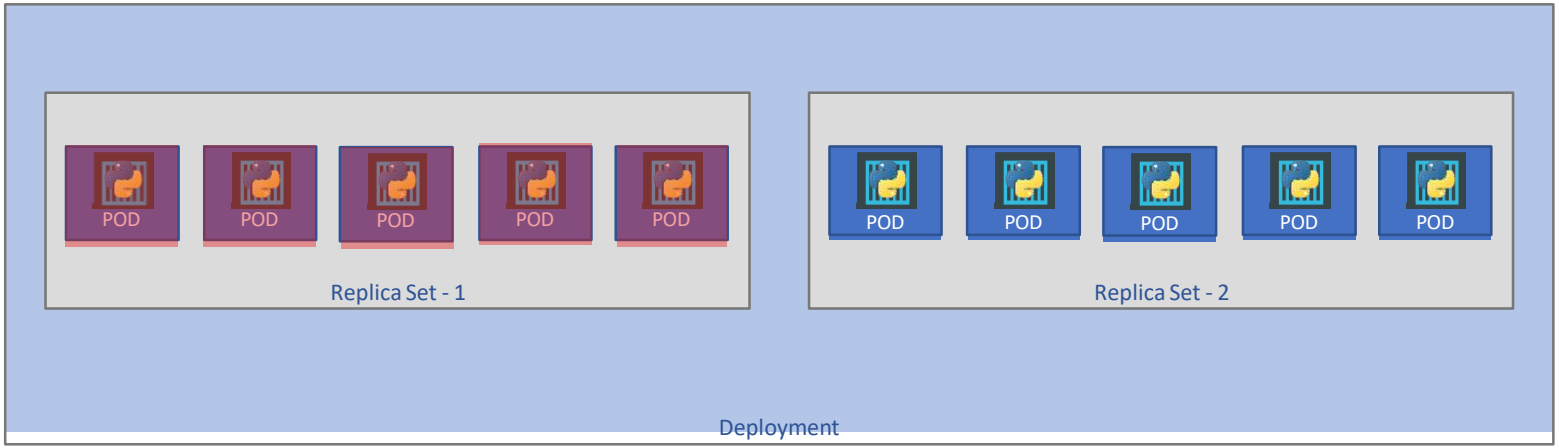
kubectl rollout undo deploy myapp-deployment

kubectl rollout undo deploy myapp-deployment --to-revision=3
```

Deployment Strategy



Upgrades

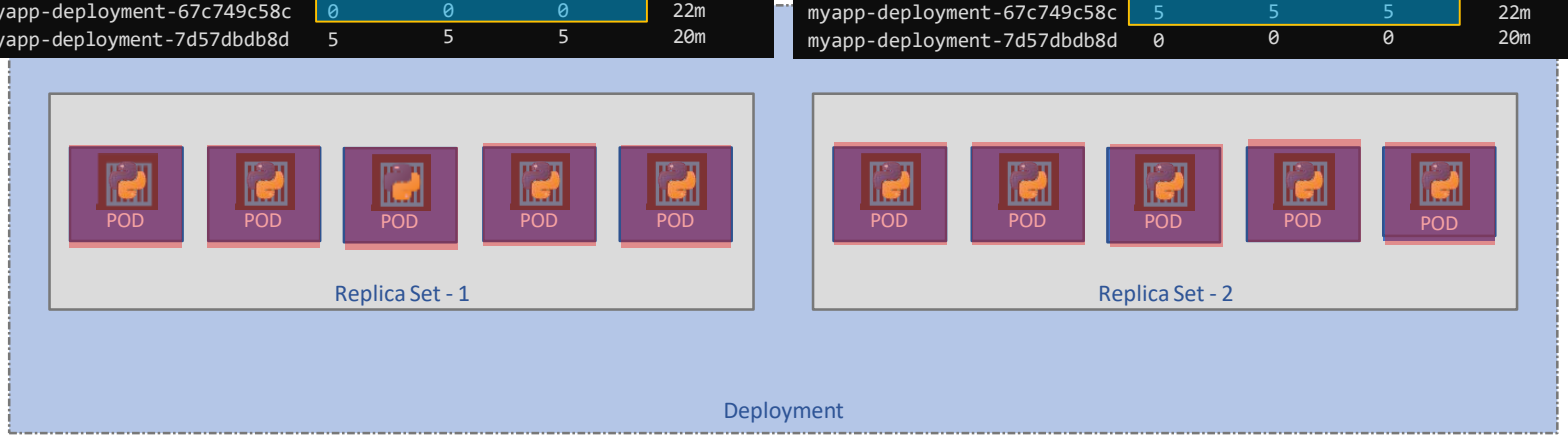



```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY     AGE
myapp-deployment-67c749c58c   0         0         0         22m
myapp-deployment-7d57dbdb8d   5         5         5         20m
```

Rollback

```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-67c749c58c   0         0         0       22m
myapp-deployment-7d57dbdb8d   5         5         5       20m
```

```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-67c749c58c   5         5         5       22m
myapp-deployment-7d57dbdb8d   0         0         0       20m
```




```
> kubectl rollout undo deployment/myapp-deployment
deployment “myapp-deployment” rolled back
```

Summarize Commands

Create

```
> kubectl create –f deployment-definition.yml
```

Get

```
> kubectl get deployments
```

Update

```
> kubectl apply –f deployment-definition.yml
```

```
> kubectl set image deployment/myapp-deployment nginx=nginx:1.9.1
```

Status

```
> kubectl rollout status deployment/myapp-deployment
```

```
> kubectl rollout history deployment/myapp-deployment
```

Rollback

```
> kubectl rollout undo deployment/myapp-deployment
```

Deployment Imperative Commands

- Creation of deployment in certain namespace

kubectl create deployment DEPLOYMENT_NAME --image=IMAGE_NAME -n NAMESPACE_NAME

- Listing all deployments in certain namespace

kubectl get deployment -n NAMESPACE_NAME

- Listing all deployments in all namespaces

kubectl get deployment –all-namespaces

- Scaling deployment with certain no.of replicas in certain namespace

kubectl scale deployment DEPLOYMENT_NAME --replicas=NO_OF_REPLICAS -n NAMESPACE_NAME

- Describing deployment in certain namespace

kubectl describe deployment DEPLOYMENT_NAME -n NAMESPACE_NAME

- Editing in deployment in certain namespace

kubectl edit deployment DEPLOYMENT_NAME -n NAMESPACE_NAME

N.B: In this case some values can edited and pods are restarted automatically with the new values and other values if edited a new file will be resulted and we will need to do ”kubectl apply -f newfile”

- Deleting deployment in certain namespace

kubectl delete deployment DEPLOYMENT_NAME -n NAMESPACE_NAME

Deployment Imperative Commands

- Changing image on the fly and record changes

kubectl set image deployment/DEPLOYMENT_NAME CONTAINER_NAME=IMAGE_NAME:TAG -n NAMESPACE_NAME --record

- Checking the rollout status

kubectl rollout status deployment/DEPLOYMENT_NAME -n NAMESPACE_NAME

- Checking the deployment history

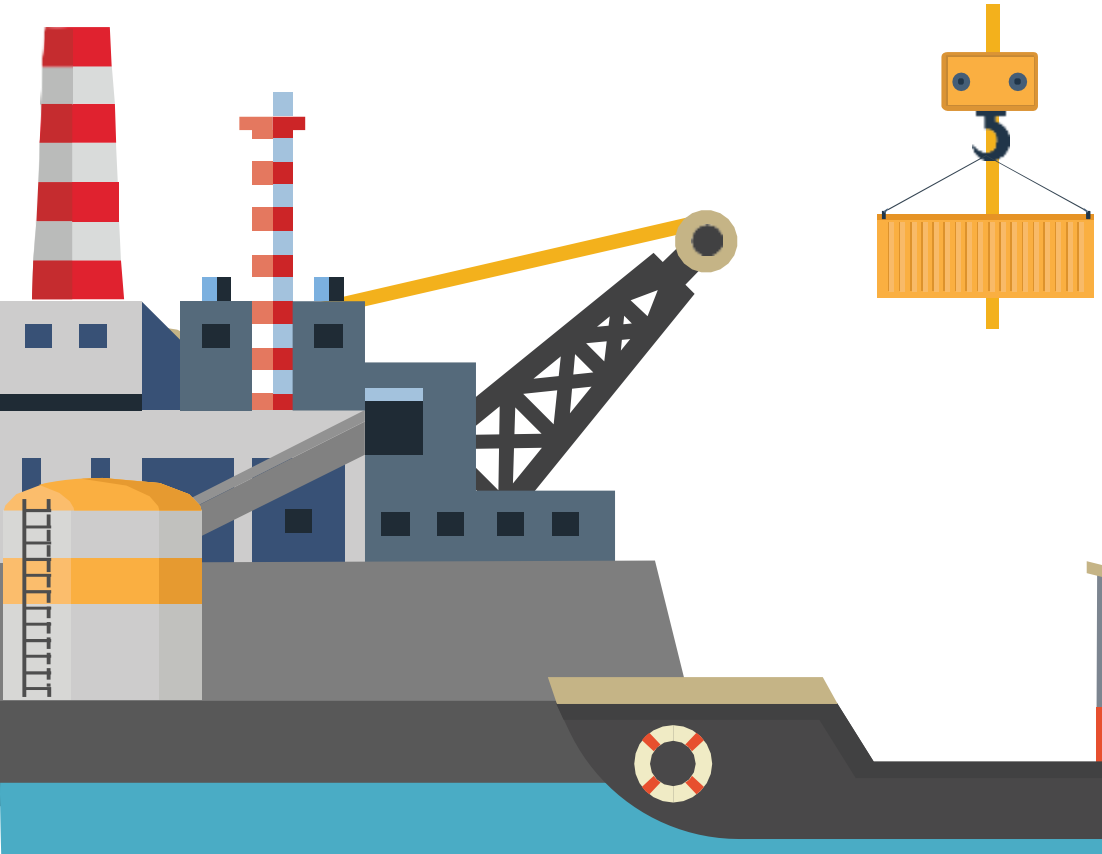
kubectl rollout history deployment/DEPLOYMENT_NAME -n NAMESPACE_NAME

- Rolling back to the latest version

kubectl rollout undo deployment/DEPLOYMENT_NAME -n NAMESPACE_NAME

- Rolling back to certain version

kubectl rollout undo deployment/DEPLOYMENT_NAME -n NAMESPACE_NAME --to-revision=n



DAY-2

Namespace



Kubernetes main objects

Namespace

- It is the pool where a number of K8s objects (pods, deployments, replica sets, services,…etc) are located in.
- A K8s cluster can contain many namespaces. Each namespace can have allocated resources as CPU and memory.
- Any cluster has two namespaces by default:

default The default namespace for objects with no other namespace

kube-system The namespace for objects created by the Kubernetes system

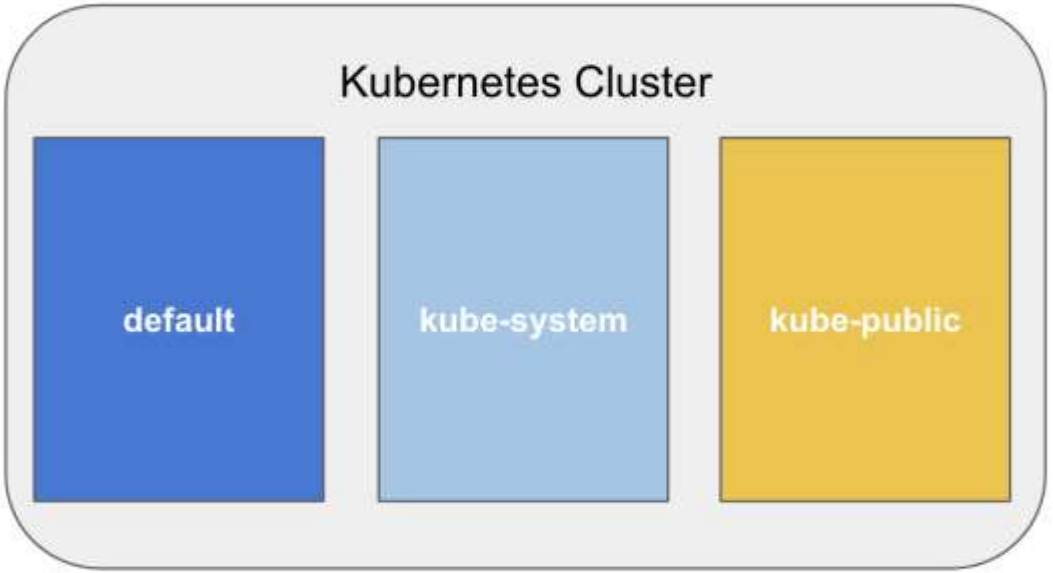
kube-public This namespace is created automatically and is readable by all users (including those not authenticated). This namespace is mostly reserved for cluster usage, in case that some resources should be visible and readable publicly throughout the whole cluster

Imperative commands:

Kubectl create ns NAMESPACE_NAME

Kubectl delete ns NAMESPACE_NAME

kubectl get namespace



Control Pod Resources

- When you specify a Pod, you can optionally specify how much of each resource a container needs. The most common resources to specify are CPU and memory (RAM); there are others.
- When you specify the resource request for containers in a Pod, the kube-scheduler uses this information to decide which node to place the Pod on.

```yaml
---
apiVersion: v1
kind: Pod
metadata:
  name: frontend
spec:
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Init container

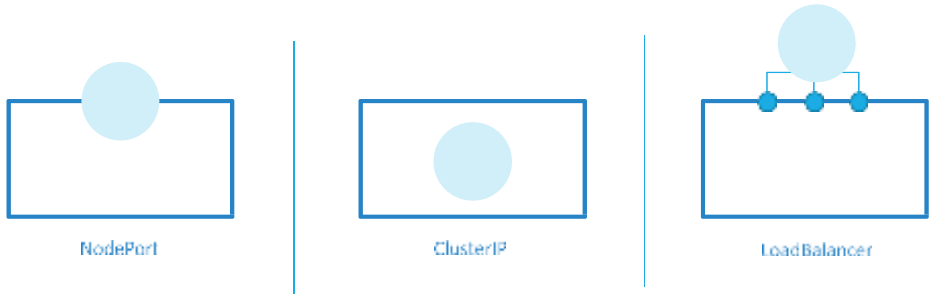
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app.kubernetes.io/name: MyApp
spec:
  containers:
  - name: myapp-container
    image: busybox:1.28
    command: ['sh', '-c', 'echo The app is running! && sleep 3600']
  initContainers:
  - name: init-myservice
    image: busybox:1.28
    command: ['sh', '-c', "until nslookup myservice.$(cat /var/run/secrets/kubernetes.io/serviceaccoun
```

It used to create a container to start before another container

Services



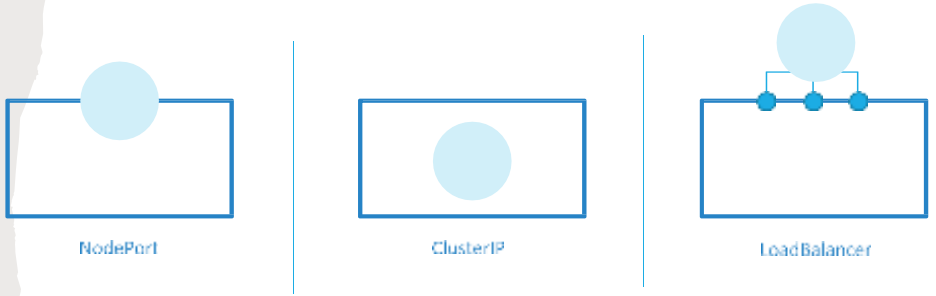
Services Types



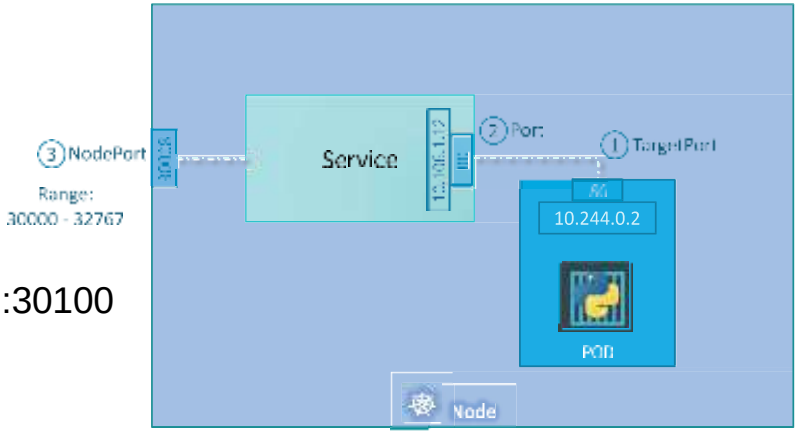
Kubernetes main objects

services

- Allow connectivity between pods
- For example, can create connections between backend and frontend
- We have 2 types of communications :
  - External ➔ end user
  - Internal ➔ between pods
- Types of services
  - NodePort
  - ClusterIP
  - Loadbalancer



Service - NodePort



192.168.10.100:30100

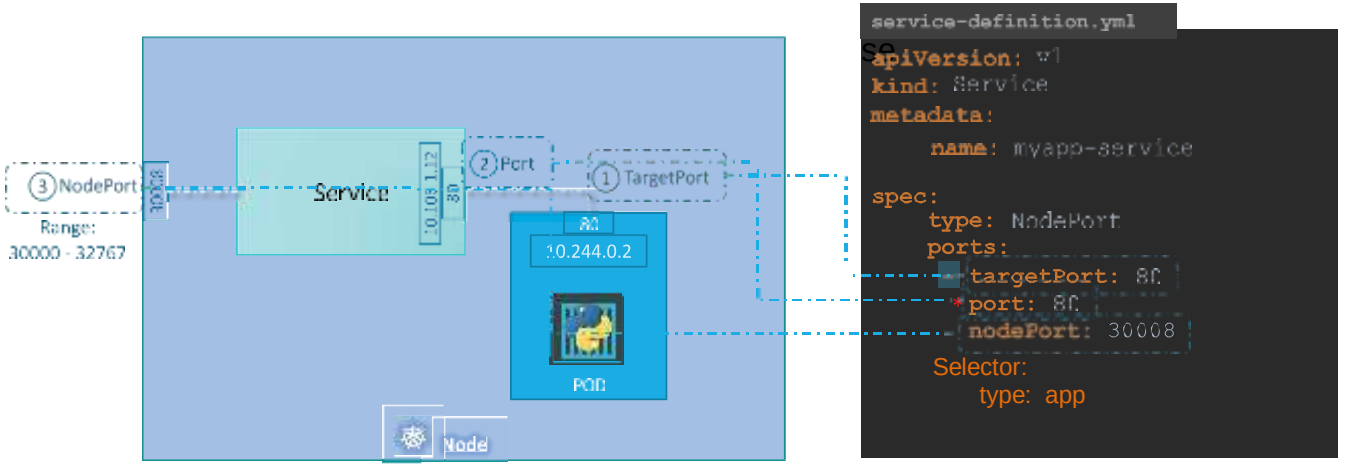
192.168.10.100

In NodPort contains 3 types of ports:

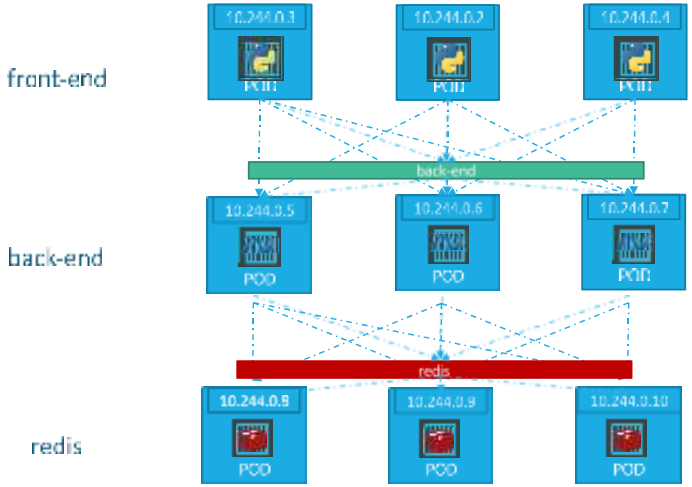
1. TargetPort: port on pod where actual server is running , where the service forward traffice to
2. Port : port on service itself
3. NodePort: port that can access pod externally through, Range from 30000:32767

- If you don't provide port will assume to be tha same of Target Port
- If you don't specifiy the NodePort will take it randomly from above range

Service - NodePort



ClusterIP


front-end
back-end
redis
back-end
redis
10.244.0.5
10.244.0.6
10.244.0.7
10.244.0.8
10.244.0.9
10.244.0.10
POD

ClusterIp configuration

service-definition.yml

```yaml
apiVersion: v1
kind: Service
metadata:
    name: back-end

spec:
    type: ClusterIP
    ports:
     - targetPort: 80
       port: 80

    selector:
```

pod-definition.yml

```
> kubectl create -f service-definition.yml
service "back-en." created

> kubectl get services
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP    7d.
back-en.     ClusterIP   10.106.127.133  <none>        80/TCP     3m
```

```yaml
    app: myapp
    type: back-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Service Imperative Commands

- Creation of service in certain namespace

kubectl create service clusterip SERVICE_NAME **--tcp**=port:targetport -n NAMESPACE_NAME

kubectl create service nodeport SERVICE_NAME **--tcp**=port:targetport **-**n NAMESPACE_NAME

kubectl create service loadbalancer SERVICE_NAME **--tcp**=port:targetport -n NAMESPACE_NAME

N.B: In this case we need to adjust manually labels and selectors of the pod and the service

N.B: If the service is NodePort, then a free port will be picked up by K8s randomly

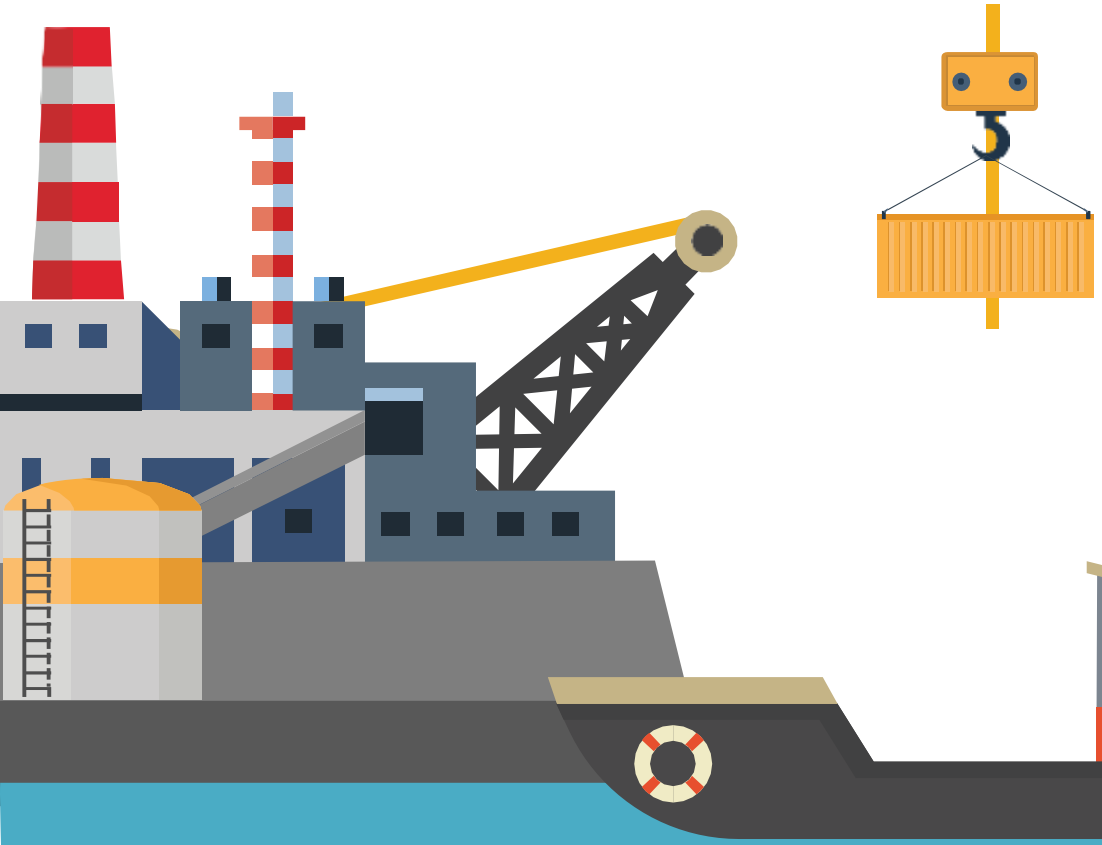
- Exposing of a service in certain namespace

kubectl expose deployment DEPLOYMENT_NAME **--port**=80 **--target-port**=8000 **[--type NodePort] -**n NAMESPACE_NAME

N.B: In this case labels and selectors of the pod and the service are adjusted automatically

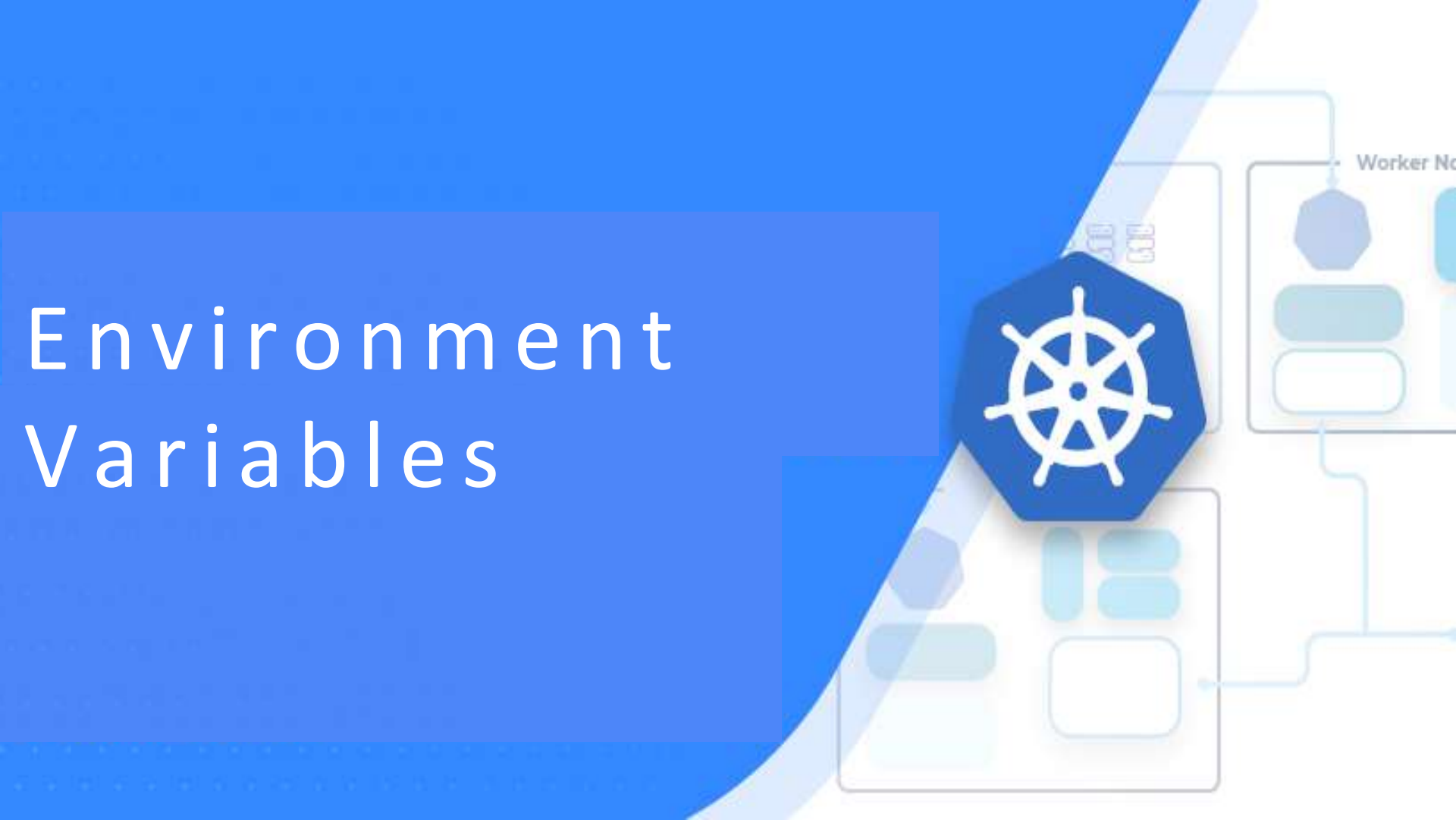
- Deleting certain service in certain namespace

kubectl delete service SERVICE_NAME -n NAMESPACE_NAME



DAY-3

Environment Variables



ENV Value Types

```
env:
  - name: APP_COLOR
    value: pink
```

1 Plain Key Value

```
env:
  - name: APP_COLOR
    valueFrom:
      configMapKeyRef:
```

2 ConfigMap

```
env:
  - name: APP_COLOR
    valueFrom:
      secretKeyRef:
```

3 Secrets

ENV Variables in Docker

```
docker run -e APP_COLOR=blue    simple-webapp-color
```




```
docker run  -e APP_COLOR=green simple-webapp-color
```




```
docker run  -e APP_COLOR=pink simple-webapp-color
```



ENV Variables in Kubernetes

```
docker run  -e APP_COLOR=pink simple-webapp-color
```

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080

    env:
      - name: APP_COLOR
        value: pink
```



ConfigMaps



ConfigMaps

ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```



Create ConfigMap



Inject into Pod

pod-definition.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080

    envFrom:
    - configMapRef:
        name: app-config
```

Create ConfigMaps



ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```

Imperative

```
kubectl create configmap
    <config-name> --from-literal=<key>=<value>
```

```
kubectl create configmap \
    app-config --from-literal=APP_COLOR=blue \
               --from-literal=APP_MOD=prod
```

Will create variable with the file name and its vaules is
The data inside file
Use it in nginx mount for example

```
kubectl create configmap
    <config-name> --from-file=<path-to-file>
```

```
kubectl create configmap \
    app-config --from-file=app_config.properties
```



```
saadalsayed@Saads-MacBook-Air test % cat index.html
hello from egypt
saadalsayed@Saads-MacBook-Air test % kubectl create configmap test-cm --from-file=index.html
```

To create more than key and its value using file use the following command :
$Kubectl create cm cm-name --from-env-file=values.txt

File must be in format:

$cat values.txt

```
NAME=saad
ROLE=Devops
ENV=SYS
```

Create ConfigMaps

ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```

Declarative

```
kubectl create -f
```

config-map.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```



Create ConfigMap

```
kubectl create -f config-map.yaml
```

View ConfigMaps

```
kubectl get configmaps
```

```
NAME         DATA      AGE
app-config   2         3s
```

```
kubectl describe configmaps
```

```
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_COLOR:
----
blue
APP_MODE:
----
prod
Events:  <none>
```

ConfigMap in Pods

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    envFrom:
      - configMapRef:
          name: app-config
```

config-map.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```

```
kubectl create -f pod-definition.yaml
```

ConfigMap in Pods

```
envFrom:
  - configMapRef:
      name: app-config
```

ENV

SINGLE ENV

```
env:
  - name: APP_COLOR
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_COLOR
```

```
volumes:
- name: app-config-volume
  configMap:
    name: app-config
```

VOLUME

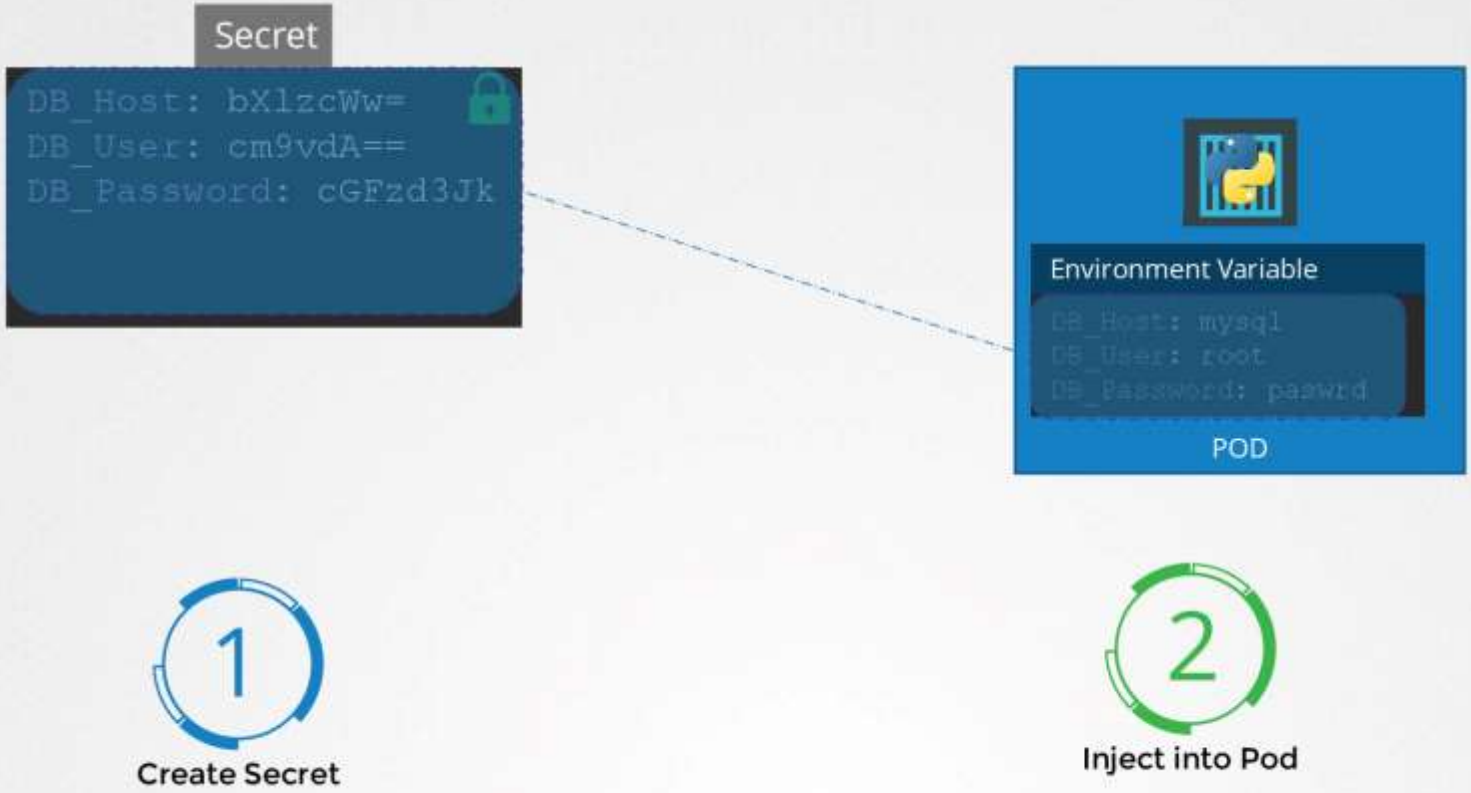
```
[saad@Master cm]$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
   name: pod
spec:
  containers:
  - name: cont
    image: nginx
    volumeMounts:
    - name:  app-config
      mountPath: /tmp

    env:
     - name: ROLE-STATIC
       valueFrom:
           configMapKeyRef:
             name: from-file-cm
             key: ROLE
    envFrom:
     - configMapRef:
         name: from-file-cm
  volumes:
  - name: app-config
    configMap:
       name: from-file-cm
[saad@Master cm]$
```



Kubernetes Secrets

Secret


Secret
DB_Host: bXlzcWw=
DB_User: cm9vdA==
DB_Password: cGFzd3Jk
Environment Variable
DB_Host: mysql
DB_User: root
DB_Password: paswrd
POD
1
Create Secret
2
Inject into Pod

Create Secrets

Secret

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```

Imperative

```
kubectl create secret generic
    <secret-name> --from-literal=<key>=<value>
```

```
kubectl create secret generic \
    app-secret --from-literal=DB_Host=mysql      \
               --from-literal=DB_User=root
               --from-literal=DB_Password=paswrd
```

```
kubectl create secret generic
    <secret-name> --from-file=<path-to-file>
```

```
kubectl create secret generic \
    app-secret --from-file=app_secret.properties
```



Create Secret

Create Secrets

Secret

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```



Create Secret

Declarative

```
kubectl create -f
```

secret-data.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_Host: bXlzcWw=
  DB_User: cm9vdA==
  DB_Password: cGFzd3Jk
```

```
kubectl create -f secret-data.yaml
```

Encode Secrets

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```



```
DB_Host: bXlzcWw=
DB_User: cm9vdA==
DB_Password: cGFzd3Jk
```

```
echo -n 'mysql' | base64
bXlzcWw=
```

```
echo -n 'root' | base64
cm9vdA==
```

```
echo -n 'paswrd' | base64
cGFzd3Jk
```

View Secrets

```
kubectl get secrets
```

```
NAME                  TYPE                                  DATA      AGE
app-secret            Opaque                                3         10m
default-token-mvtkv   kubernetes.io/service-account-token   3         2h
```

```
kubectl describe secrets
```

```
Name:         app-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
DB_Host:      10 bytes
DB_Password:  6 bytes
DB_User:      4 bytes
```

```
kubectl get secret app-secret -o yaml
```

```
apiVersion: v1
data:
  DB_Host: bX1zcWw=
  DB_Password: cGFzd3Jk
  DB_User: cm9vdA==
kind: Secret
metadata:
  creationTimestamp: 2018-10-18T10:01:12Z
  labels:
    name: app-secret
  name: app-secret
  namespace: default
uid: be96e989-d2bc-11e8-a545-080027931072
type: Opaque
```

Decode Secrets

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```



```
DB_Host: bXlzcWw=
DB_User: cm9vdA==
DB_Password: cGFzd3Jk
```

```
echo -n 'bXlzcWw=' | base64 --decode
mysql
```

```
echo -n 'cm9vdA==' | base64 --decode
root
```

```
echo -n 'cGFzd3Jk' | base64 --decode
paswrd
```

Secrets in Pods

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    envFrom:
      - secretRef:
          name: app-secret
```

secret-data.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_Host: bXlzcWw=
  DB_User: cm9vdA==
  DB_Password: cGFzd3Jk
```



Inject into Pod

```
kubectl create -f pod-definition.yaml
```

Secrets in Pods

```
envFrom:
  - secretRef:
      name: app-config
```

ENV

SINGLE ENV

```
env:
  - name: DB_Password
    valueFrom:
      secretKeyRef:
        name: app-secret
        key: DB_Password
```

```
volumes:
- name: app-secret-volume
  secret:
    secretName: app-secret
```

VOLUME

Secrets in Pods as Volumes

```
volumes:
- name: app-secret-volume
  secret:
    secretName: app-secret
```

VOLUME

```
ls /opt/app-secret-volumes
DB_Host      DB_Password  DB_User
```

```
cat /opt/app-secret-volumes/DB_Password
paswrd
```

Inside the Container

Kubernetes main objects

Secrets and ConfigMaps

- Both are used to inject configuration data into the containers.
- Both can be exposed inside a container as mounted files or volumes or environment variables.
- **The big difference between Secrets and ConfigMaps are that Secrets are obfuscated with a Base64 encoding.**
- Secret types

1. generic
2. tls
3. docker-registry



Secret & ConfigMap Imperative Commands

- Create generic secret

kubectl create secret generic SECRET_NAME **--from-file**=path/to/file

kubectl create secret generic SECRET_NAME **--from-literal**=key1=supersecret **--from-literal**=key2=topsecret

kubectl create secret generic SECRET_NAME **--from-env-file**=path/to/bar.env

- Delete secret

kubectl delete secret SECRET_NAME –n NAMESPACE_NAME

- Create configmap

kubectl create configmap CONFIGMAP_NAME **--from-file**=path/to/file –n NAMESPACE_NAME

kubectl create configmap CONFIGMAP_NAME **--from-literal**=key1=supersecret **--from-literal**=key2=topsecret –n NAMESPACE_NAME

kubectl create configmap CONFIGMAP_NAME **--from-env-file**=path/to/bar.env –n NAMESPACE_NAME

- Delete ConfigMap

kubectl delete configmap CONFIGMAP_NAME –n NAMESPACE_NAME

Volumes



Volumes

There are two types of volumes we can attach to the deployment according to the priority of persisting the data :

- Non-Persistent (emptydir)

An emptyDir volume is first created when a Pod is assigned to a node, and exists as long as that Pod is running on that node. As the name says, the emptyDir volume is initially empty. All containers in the Pod can read and write the same files in the emptyDir volume, though that volume can be mounted at the same or different paths in each container. When a Pod is removed from a node for any reason, the data in the emptyDir is deleted permanently.

- Persistent (pv, pvc)

In this case the data will be persistent since we create two important resources called persistent volume (pv) and persistent volume claim (pvc).

A PersistentVolume (PV) is a piece of storage in the cluster that has been provisioned by the administrator or dynamically provisioned using Storage Classes. This API object captures the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system. PV has three persistent volume reclaim policies (It is not a namespaced resource. It is Global on all namespaces)

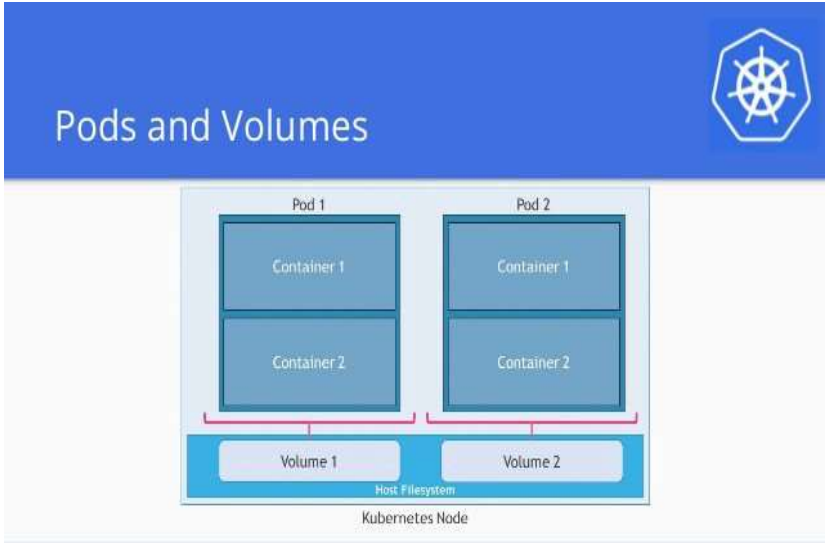
Retain: if pvc is deleted, pv won't be deleted and won't be reused

Reclaim: if pvc is deleted, pv will be reused

Delete: if pvc is deleted, pv is deleted

A PersistentVolumeClaim (PVC) is a request for storage by a user. PVCs consume PV resources (CPU and Memory). Claims can request specific size and access modes (e.g., they can be mounted ReadWriteOnce, ReadOnlyMany or ReadWriteMany, see AccessModes).

IMPORTANT: pv and pvc can related automatically without any Labels if the access modes are matching and pvc capacity is less than or equal to the pv



Volume



Data will be deleted if pod removed .
So, we need to create volume to be permanent
We have more solutions for this :

Volumes & Mounts

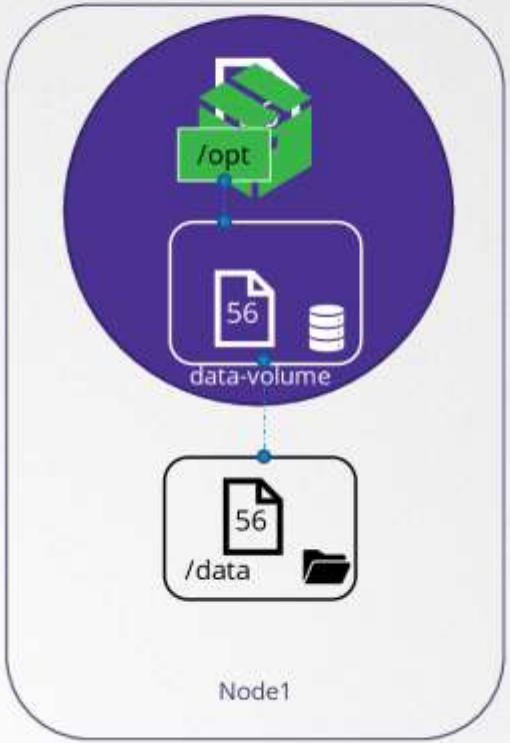
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
  - image: alpine
    name: alpine
    command: ["/bin/sh","-c"]
    args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
    volumeMounts:
    - mountPath: /opt
      name: data-volume

  volumes:
  - name: data-volume
    hostPath:
      path: /data
      type: Directory
```

Not recommended on multi node cluster

All files in /opt of pod will be saved to /data on only its node ,also when the pod deleted files still on node

Issue : /data directory must have the same value on all nodes , and this doesn't happen



Note :
In minikube
You need to ssh into minikube to see the data of hostpath

The issue with previous hostpath that you need to have the same path on all nodes, and this is difficult

To overcome you can use shared storage or external storage like AWS s3

Or use PV and pvc



Persistent Volume

The issue with previous Volume configurations !!?

- When we create volume, we create it on each pod configuration file manually
- So, each user must configure its storage in pod definition file .

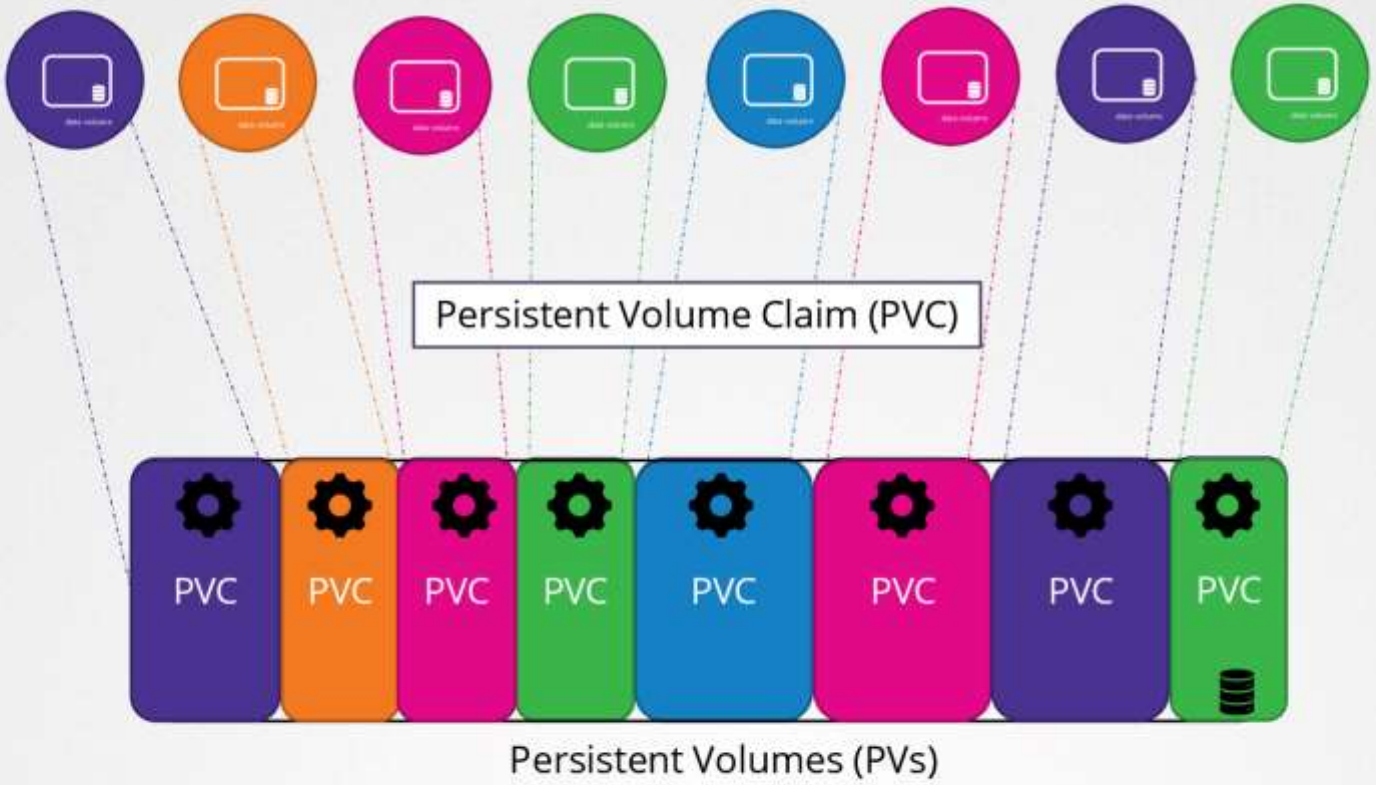
- When you have lots of pod ,users must configure storage on pod

And every time a change is made on storage the user must have to change on all pod.

So, we need more central solution for this !!!

The solution is the administrator have to create a large pool of storage and then users can get pieces from it as required

Persistent Volume



Persistent Volume

pv-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```

hostPath:
path: /data

Not Recommended on production

- ReadOnlyMany
- ReadWriteOnce
- ReadWriteMany



Persistent Volume (PV)

```
kubectl create -f pv-definition.yaml
```

```
kubectl get persistentvolume
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
pv-vol1	1Gi	RWO	Retain	Available				3m

Access Modes:

- **ReadOnlyMany** : read only by multiple nodes
- **ReadWriteMany:** read and write by multiple nodes
- **ReadWriteOnce**: read and write from single node

For Kubernetes cluster version 1.22+. :

- **ReadWriteOncePod:** the volume can be mounted as read-write by a single Pod.

In the CLI, the access modes are abbreviated to:

•RWO - ReadWriteOnce
•ROX - ReadOnlyMany
•RWX - ReadWriteMany
•RWOP - ReadWriteOncePod

Refernce:
https://kubernetes.io/docs/concepts/storage/persistent-volumes/#access-modes



Persistent Volume Claims

Persistent Volume Claim

pvc-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

```
kubectl create -f pvc-definition.yaml
```

pv-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```

persistentVolumeReclaimPolicy:

Retain: if pvc is deleted, pv won't be deleted and won't be reused
Reclaim: if pvc is deleted, pv will be reused
Delete: if pvc is deleted, pv is deleted

Note Each PVC is bounded to only one PV

Mount Persistent Volume Claim to POD:

```
Volumes:
  - name: test-volume
    persistentVolumeClaim:
      claimName:   pvc-name
```

How to use persistentvolume to use hostpath

You have to specify storageClassName to be hostPath

Pod

```
[saad@Master volume]$ cat shuffle-pod.yaml
apiVersion: v1
kind: Pod
metadata:
   name: shuflle
   labels:
      type: game
spec:
  containers:
  - name: con
    image: nginx
    command: ['/bin/sh','-c']
    args: ['shuf -i 0-100 -n 1 >/data/number.out;
sleep 3000']
    volumeMounts:
    - name: shuf
      mountPath: /data
  volumes:
  - name: shuf
    persistentVolumeClaim:
        claimName: shuffle-pvc
```

Persistent Volume

```
[saad@Master volume]$ cat pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
   name: shuffle-pvc
spec:
  storageClassName: hostpath
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
```

Persistent Volume Claim

```
[saad@Master volume]$ cat pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
   name: shuffle-pv
spec:
  storageClassName: hostpath
  accessModes:
    - ReadWriteOnce
  capacity:
     storage: 1Gi
  hostPath:
     path: /shuffle
```

Storage Classes



PV and PVCs

pv-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

pvc-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
  - image: alpine
    name: alpine
    command: ["/bin/sh","-c"]
    args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
    volumeMounts:
    - mountPath: /opt
      name: data-volume
  volumes:
  - name: data-volume
    persistentVolumeClaim:
      claimName: myclaim
```



Static Provisioning

```
gcloud beta compute disks create \
    --size 1GB
    --region us-east1
    pd-disk
```

pv-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

Disk have to be created on Cl
before provision it

Everytime you need to create
you need first to manually crea
on cloud

So it will be nice if it is provisio
application required it

SO we will use StorageClass

Dynamic Provisioning

pv-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

PV

sc-definition.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: google-storage
provisioner: kubernetes.io/gce-pd
```

Storage class for external storage will do the role of persistent volume ,
it will provision the volume on external source then create PV automatically then connect your PVC with it

SC

Dynamic Provisioning



sc-definition.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: google-storage
provisioner: kubernetes.io/gce-pd
```

pvc-definition.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: google-storage
  resources:
    requests:
      storage: 500Mi
```

pod-definition.yaml

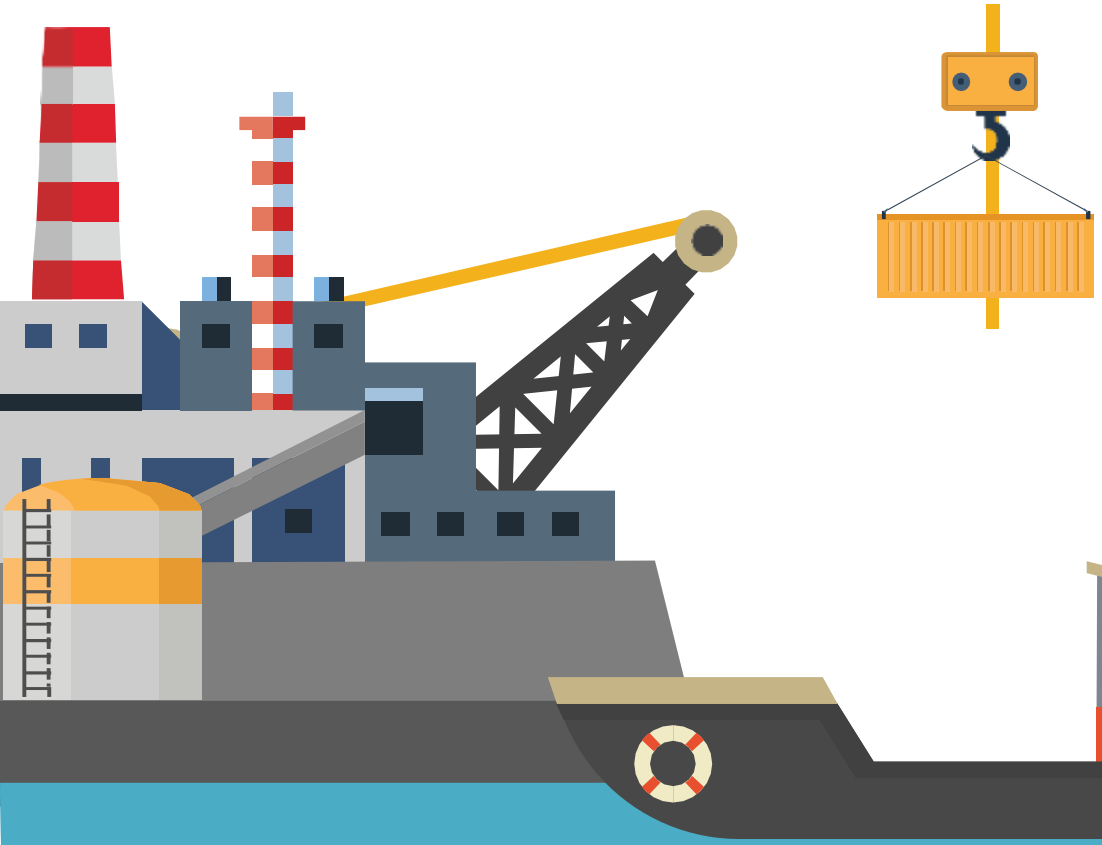
```
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
  - image: alpine
    name: alpine
    command: ["/bin/sh","-c"]
    args: ["shuf -i 0-100 -n 1 >> /opt/
    volumeMounts:
    - mountPath: /opt
      name: data-volume
  volumes:
  - name: data-volume
    persistentVolumeClaim:
      claimName: myclaim
```

SC

PV

PVC





DAY-4

Scheduling



DaemonSets

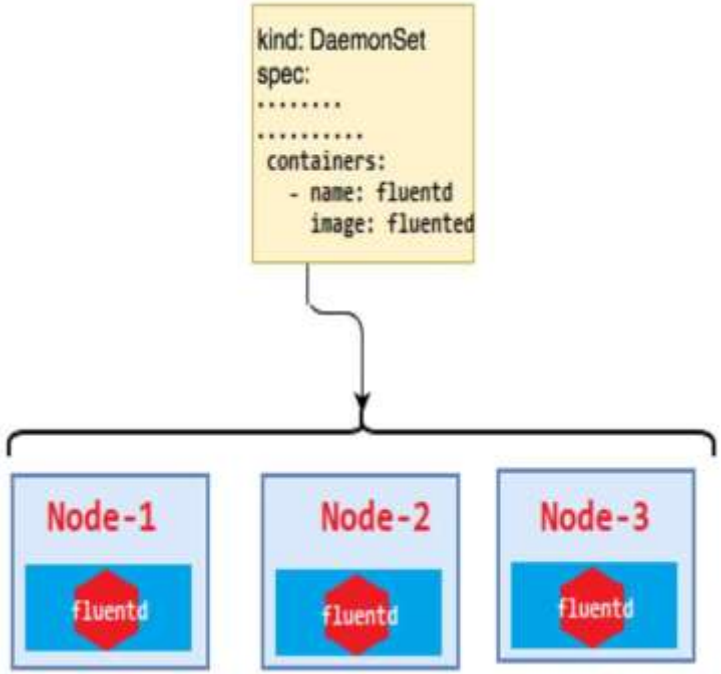


Kubernetes main objects

DaemonSet

- It ensures that all (or some) Nodes run a copy of a Pod. As nodes are added to the cluster, Pods are added to them. As nodes are removed from the cluster, those Pods are garbage collected. Deleting a DaemonSet will clean up the Pods it created.
- Some typical uses of a DaemonSet are:

1. running a logs collection daemon on every node
2. running a node monitoring daemon on every node



DaemonSet Definition

daemon-set-definition.yaml

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
      - name: monitoring-agent
        image: monitoring-agent
```

replicaset-definition.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
      - name: monitoring-agent
        image: monitoring-agent
```

```
kubectl create -f daemon-set-definition.yaml
```

```
daemon-set Created
```

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  labels:
     k8s-app: fluentd-elasticsearch
  name: fluentd-elasticsearch
spec:
  selector:
    matchLabels:
       k8s-app: fluentd-elasticsearch
  template:
    metadata:
      labels:
         k8s-app: fluentd-elasticsearch
    spec:
      containers:
      - image:
quay.io/fluentd_elasticsearch/fluentd:v2.5.2
        name: fluentd
        resources:
          limits:
            memory: 200Mi
          requests:
            cpu: 100m
            memory: 200Mi
        volumeMounts:
        - name: varlog
          mountPath: /var/log
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
```

This is example of creating fluentd container as daemonsets

Add new minikube node

$ minikube node add

Note:

* If you faced issue in initializing node regarding swap

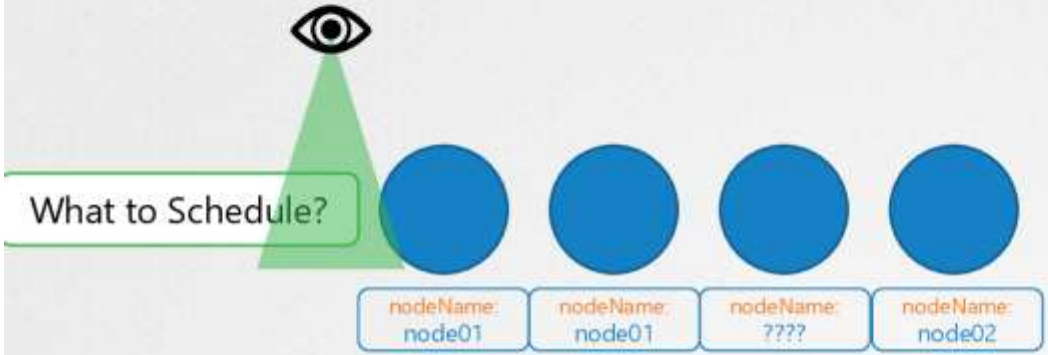
Have to make swap off ➔ swapoff -a

C2 General

Manual Scheduling



How scheduling works



Which node to schedule?

(Schedule)Bind Pod to Node

nodeName:
node02

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
      - containerPort: 8080

  nodeName: node02
```

KODEKLOUD

- By default, schedular use nodeName to specifiy the node name to run podname on it
- This will appear after you deploy the pod
- To manual schedual your pod you can specify this parameter nodeName on configuration

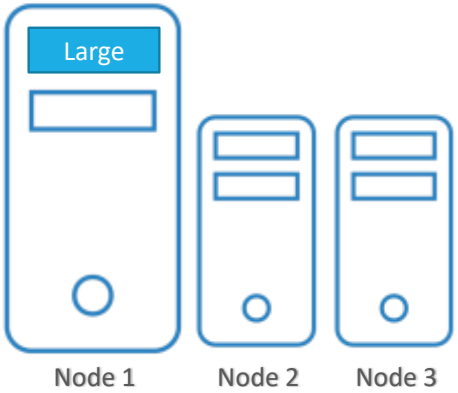
C2 General

Node Selectors



Label Nodes

```
kubectl label nodes <node-name> <label-key>=<label-value>
kubectl label nodes node-1 size=Large
kubectl label node minikube-m02 type-     To delete labels
```



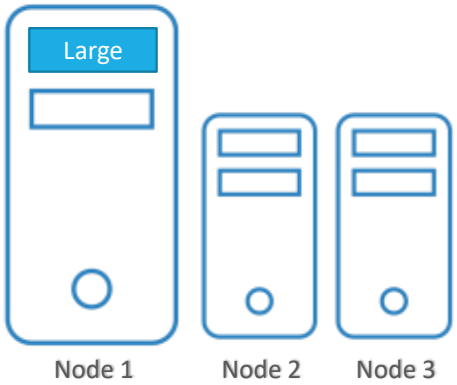
- You need to create your pod on specific group of nodes
- So you first labels your node with specific key=value
- Then when creating pod assign this labels to pod

Node Selectors

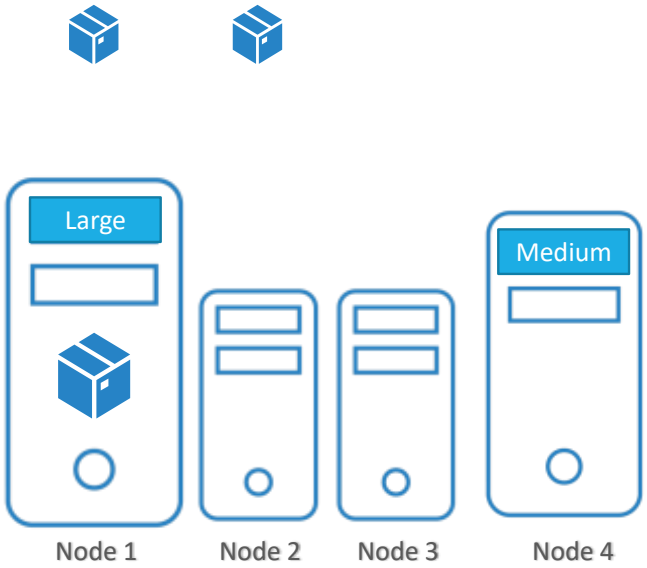
pod-definition.yml

```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor

  nodeSelector:
    size: Large
```



Node Selector - Limitations



- Large OR Medium?
- NOT Small

- What if you need to place your pod on nodes have labels of large or medium
- Or something like place the pods on the nodes that are not small
- This can't be done though node selector
- SO will use Affinity rules

Node Affinity



Node Selectors

pod-definition.yml

```yaml
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor
  nodeSelector:
    size: Large
```

- Large OR Medium?
- NOT Small

- Node affinity gives us power cababilites to limit pods creation on specific nodes
- With Great power comes greate complexitiy

Node Affinity

pod-definition.yml

```
apiVersion:
kind: Pod

metadata:
 name: myapp-pod
spec:

  containers:
  - name: data-processor
    image: data-processor

  nodeSelector:
    size: Large
```

pod-definition.yml

```
apiVersion:
kind:

metadata:
 name: myapp-pod
spec:

 containers:
  - name: data-processor
    image: data-processor

 affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: size
          operator: In
          values:
          - Large
```

Can have multiple values like :

- Large
- medium

Can have other operators like :

- In
- NotIn
- Exists

Node Affinity

pod-definition.yml

```
apiVersion:
kind:

metadata:
 name: myapp-pod
spec:

 containers:
  - name: data-processor
    image: data-processor
 affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:

        - key: size
          operator: Exists
```

When you use Exists operator

it will not compare the value

Node Affinity Types

Available:

`requiredDuringSchedulingIgnoredDuringExecution`

`preferredDuringSchedulingIgnoredDuringExecution`

Planned:

`requiredDuringSchedulingRequiredDuringExecution`

What if we forget to label node before creating pod ?

Node Affinity Types

Available:

requiredDuringSchedulingIgnoredDuringExecution

preferredDuringSchedulingIgnoredDuringExecution

	DuringScheduling	DuringExecution
Type 1	Required	Ignored
Type 2	Preferred	Ignored

Required DuringSchedulting: it is the state where the pod is not exist and created for the first time
preferedDuringSchedulting: if labels of node not found ignore this affinity role

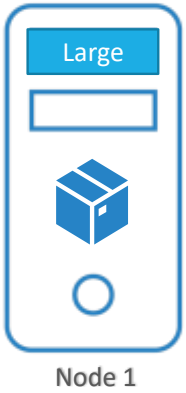
IgnoredDuring execution: this applies when the pod is already created and there is a change happen in environment such as changing pod labels , So pods will not impacted because it is ignored

Node Affinity Types

Planned:

requiredDuringSchedulingRequired**DuringExecution**

	DuringScheduling	DuringExecution
Type 1	Required	Ignored
Type 2	Preferred	Ignored
Type 3	Required	Required



```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: type
            operator: In
            values:
            - large
  containers:
  - name: with-node-affinity
    image: nginx
```

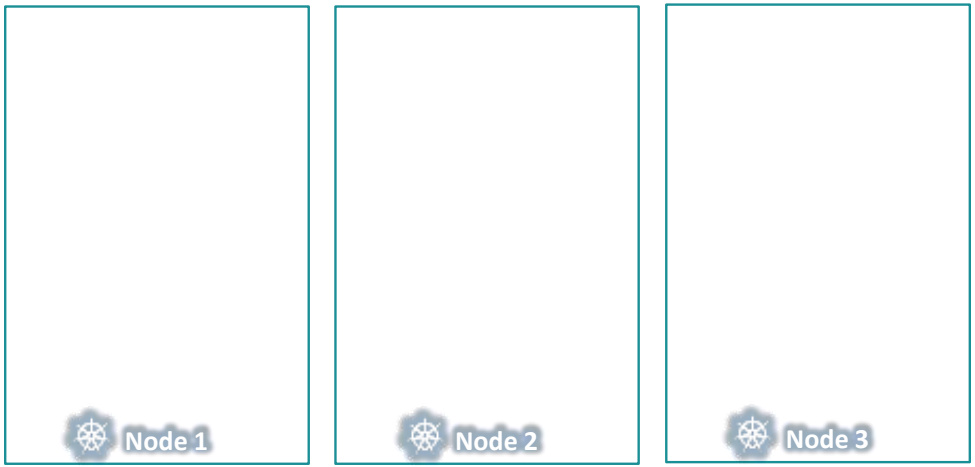
Practical example

Taints And Tolerations





- Taint and toleerations are used to set restirictions on what pods can be scheduled on a node
- For example, we need only specifica pods being created on speific node , no other pods will be created on this node

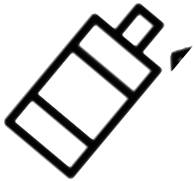


- Step 1 is to add taint to node
- Step 2 is to add toleration to pod needed to be applied on this node



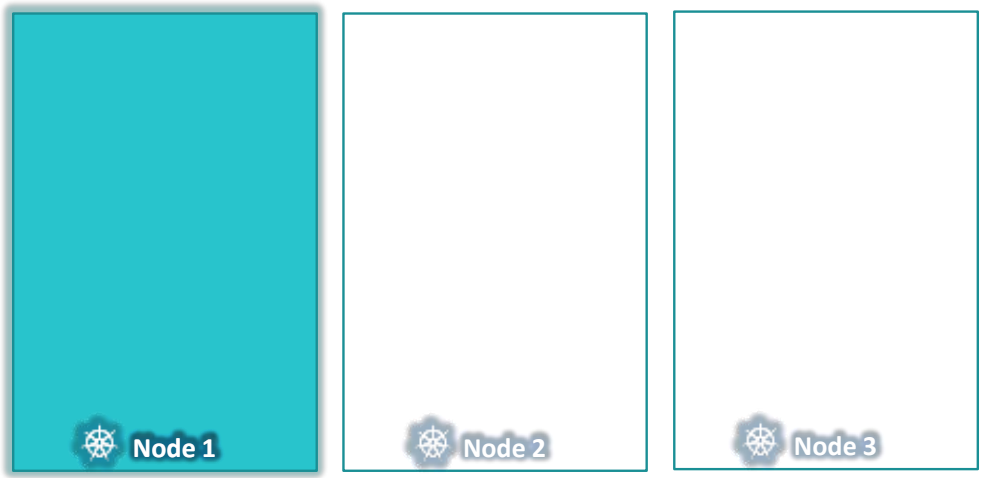


2



Taint=blue

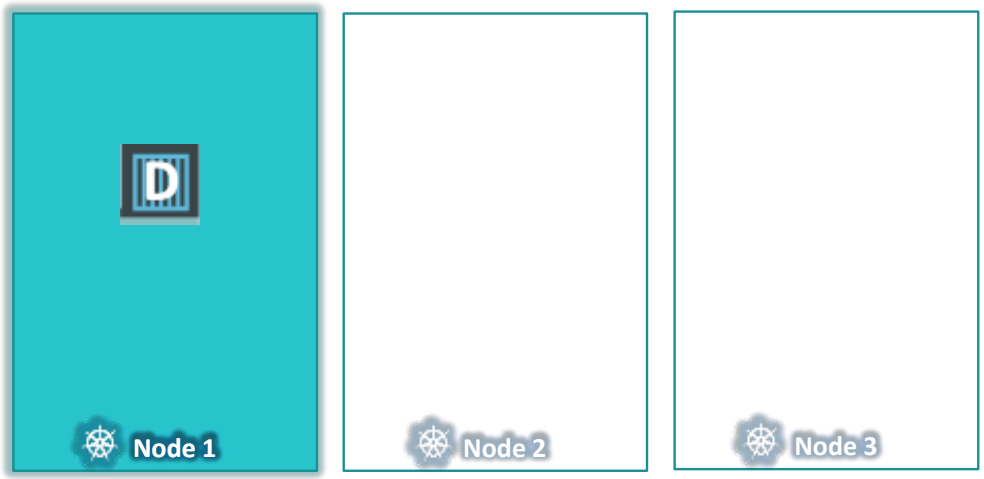
1



After toleration Only
Pod D will be deployed on
Node1



Taint=blue



Taints - Node

```
kubectl taint nodes node-name key=value:taint-effect
```

NoSchedule | PreferNoSchedule | NoExecute

What happens to PODs that do not tolerate this taint?

```
kubectl taint nodes node1 app=myapp:NoSchedule
```

```
kubectl taint nodes node1 key1=value1:NoSchedule-
```

Tolerations - PODs

```
kubectl taint nodes node1 app=myapp:NoSchedule
```

pod-definition.yml

```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: nginx-container
    image: nginx

  tolerations:
  - key:"    "
    operator:"Equal"
    value:"     "
    effect:"           "
```



Authentication and Authorisation

- Authentication: It is the act of validating that users are whom they claim to be.
- Authorization: it is the process of giving the user permission to access a specific resource or function.

C2 General

Secure Kubernetes

kube-apiserver

Kub-apiserver is the responsible for authentication

Who can access?

What can they do?

Accounts

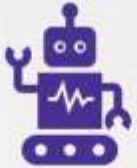


Admins



Developers

User



Bots



Prometheus

Service Accounts

Accounts



Admins



Developers



Bots

User

Service Accounts

```
kubectl create user user1
user1 Created
```



```
kubectl create serviceaccount sa1
Service Account sa1 Created
```



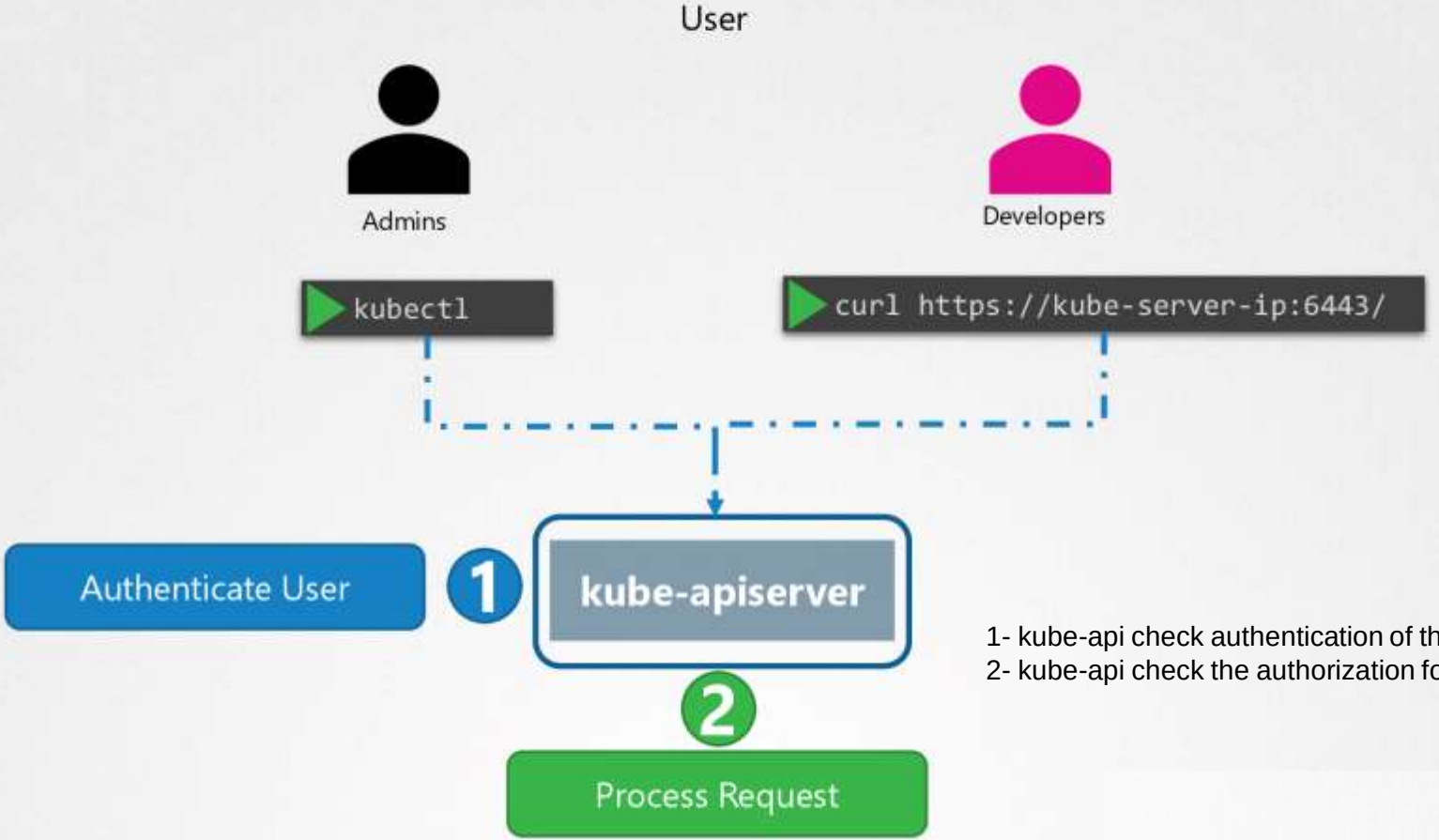
```
kubectl list users
Username
user1
user2
```



```
kubectl list serviceaccount
ServiceAccount
sa1
```



Accounts



1- kube-api check authentication of this user
2- kube-api check the authorization for this action

Authentication in Kubernetes

- Who can access the cluster?

1. Authenticated user
2. Authenticated group
3. Authenticated Service Account (used mainly by Applications Integrations)

- How can they access?

1. API call containing either

- Username and password
- Token
- CA (Certificate Authority) key, client certificate and client key

2. Kubectl and passing kubeconfig file containing all the required authentication data as CA key, client certificate and client key **(Commonly used)**

Ex: kubectl get po –n NAMESPACE --kubeconfig=CONFIG_FILE

N.B: certificate authority or certification authority (CA) is an entity that issues digital certificates.

N.B: kubeconfig also defines the cluster that should be accessed, who should access (user/group/service account) and the context which is a combination of the user@cluster.

Authentication

Who can access?

- Files – Username and Passwords Deprecated
- Files – Username and Tokens
- Certificates
- External Authentication providers - LDAP
- Service Accounts

Authorization

What can they do?

- RBAC Authorization
- ABAC Authorization
- Node Authorization
- Webhook Mode

Auth Mechanisms - Basic

kube-apiserver

```
--basic-auth-file=user-details.csv
```

user-details.csv

```
password123,user1,u0001
password123,user2,u0002
password123,user3,u0003
password123,user4,u0004
password123,user5,u0005
```

kube-apiserver.service

```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC \\
  --bind-address=0.0.0.0 \\
  --enable-swagger-ui=true \\
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --runtime-config=api/all \\
  --service-cluster-ip-range=10.32.0.0/24 \\
  --service-node-port-range=30000-32767 \\
  --v=2
```

Note: Showing fewer options for simplicity

Auth Mechanisms - Basic

Static Password File

user-details.csv

```
password123,user1,u0001,group1
password123,user2,u0002,group1
password123,user3,u0003,group2
password123,user4,u0004,group2
password123,user5,u0005,group2
```

Static Token File

user-token-details.csv

```
KpjCVbI7rCFAHYPkByTIzRb7gu1cUc4B,user10,u0010,group1
rJjncHmvtXHc6M1WQddhtvNyyhqTdxSC,user11,u0011,group1
mjpOFIEiFOkL9toikaRNtt59ePtczZSq,user12,u0012,group2
PG41lXhs7QjqwWkmBkvgGT9glOyUqZij,user13,u0013,group2
```

```
--token-auth-file=user-details.csv
```

```
curl -v -k https://master-node-ip:6443/api/v1/pods --header "Authorization: Bearer KpjCVbI7rCFAHYPkBzRb7gu1cUc4B"
```

Note: Showing fewer options for simplicity

Note

- This is not a recommended authentication mechanism
- Consider volume mount while providing the auth file in a kubeadm setup
- Setup Role Based Authorization for the new users

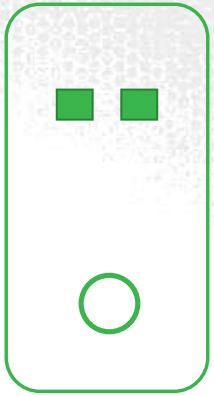
Authenticate Users Using Certificate



CERTIFICATE AUTHORITY (CA)



- User Create his/her private key
- Then crete Certificatesigningrequest (CSR)
- Send this CSR to kubeapiserver for approve this request
- After approval for csr it will generate a publickey to access the cluster
- Then User have public key which is certified now from CA of cluster
- User can use these pub/priv key now to access cluster



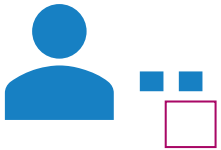


1. Create CertificateSigningRequest Object

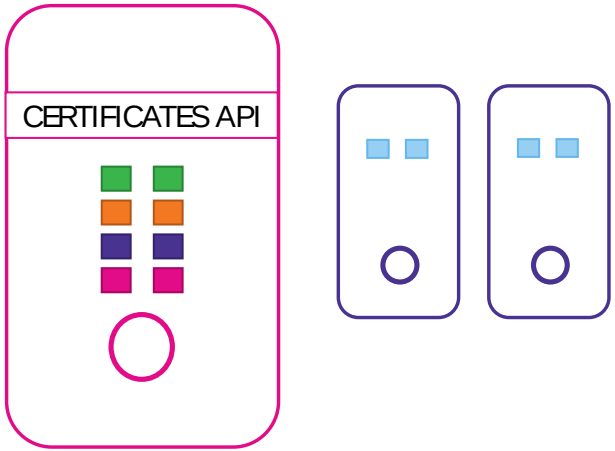
2. Review Requests

3. Approve Requests

4. Share Certs to Users







1- Create Private abd csr key :





```
openssl genrsa -out jane.key 2048
```

```
jane.key
```

```
openssl req -new -key jane.key -subj  "/CN=jane" -out jane.csr
```

```
jane.csr

-----BEGIN CERTIFICATE REQUEST-----
MIICWDCCAUACAQAwEzERMA8GA1UEAwwIbmV3LXVzZXIwggEiMA0GCSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQDO0WJW+DXsAJSIrjpNo5vRIBplnzg+6xc9+UVwkKi0
LfC27t+1eEnON5Muq99NevmMEOnrDUO/thyVqP2w2XNIDRXjYyF40FbmD+5zWyCK
9w0BAQsFAAOCAQEAS9iS6C1uxTuf5BBYSU7QFQHUzalNxAdYsaORRQNwHZwHqGi4
hOK4a2zyNyi44OOijyaD6tUW8DSxkr8BLK8Kg3srREtJql5rLZy9LRVrsJghD4gY
P9NL+aDRSxROVSqBaB2nWeYpM5cJ5TF53lesNSNMLQ2++RMnjDQJ7juPEic8/dhk
Wr2EUM6UawzykrdHImwTv2mlMY0R+DNtV1Yie+0H9/YElt+FSGjh5L5YUvI1Dqiy
4l3E/y3qL71WfAcuH3OsVpUUnQISMdQs0qWCsbE56CC5DhPGZIpUbnKUpAwka+8E
vwQ07jG+hpknxmuFAeXxgUwodALaJ7ju/TDIcw==
-----END CERTIFICATE REQUEST-----
```

1- Create CSR request :




```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
 name: jane-csr
spec:
  signerName: kubernetes.io/kube-apiserver-client
  usages:
  - client auth
  request:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBSRVFVRVNULS0tLS0KTUlJQ1ZEQ0NBVHd
DQVFBd0R6RU5NQXNHQTFVRUF3d0VjMkZoWkRDQ0FTSXdEUVlKS29aSWh2Y05BUU
VCQlFBRApnZ0VQQURDQ0FRb0NnZ0VCQUtudEdFZ1AvQWxibTZnVE8xbWVQaktDU
UNXNTZ6bExtUEVLdGZSbUc3N0VsQjMzCkxsT2hhUE5OclNvK2dXRE5vV0h3YWh0
bFk3OVVON3FKUmtrNnhqcTViR3A2c3JRVmZpZnZES3dzLzJvcUNsQ1gKSWJLRHR
5WHJ6d2ZySXJQNVpMaFFrWHJETFRjL2tycWJOelVsaERWb0xYSzArS3I4VDE1b1
ZSbEdJaUhaQ212RAptZXZiZXBiaFpGRXdmd0pFYk03Q2ZCYUpIMzJ5M2dSWlBkV
HBlWEJraisxZVgyVDA2Q2pROUhxQkYydklwWE82CnNNb284UTczUk1IbUFVNmxk
UlVDYnNHQOJvLytyMHRVRWJOYmtvRDVsV3BKS3l5MW9QS1JSTk8OY1JpKzZGUGc
KWXduTFYOcHRpSGZScnhqV3dOakIxSDhSRU9Yelpxc2o4dnMzT1RzQ0F3RUFBYU
FBTUEwRONTcUdTSWIzRFFFQgpDd1VBQTRJQkFRQXB6MTcvc3ZQSk9xUllZQVBse
WxSdU9zWjhJTnJZVkVjYWw4Zk4rUWhaQWpzb2J6YTMywlZWCmlsS2Nwd0FoOTJS
VE1lcDRNdVcyRXlIRHVtU01OZno4ZOszQWc3RFhvWjE5aFgzbzhWdGFzNDRoazR
NVkZVZGYKOVBhUnlsQmQxeDhteGhxZ2crcFdDSmhSUHMrV3ovdUcvcVM0aWpaMj
lpeHdvbHpIYWg1SjE1zlgxVG1BcFJ6RQpzV3FpMFVTWTJOQWxyekdtR0V3NVJvM
HdLYkV1bjVaSEsOYW9BL25LZGFJeEkrQUQ0ZTJ4SmhsVE5ROGhaOEJlCnJXdEdF
TjhCUkJzalRIdUhNaEYvdWVCVHJxUjRUanBKdVNZUDBvOEFDcVZDbVN4ZGZLMGR
aWmhjaGxHRi8ya2kKdUtReklISTVtSWZ3YUswdTVadjlnN2doS2VkemFDSzgKLS
0tLS1FTkQgQOVSVElGSUNBVEUgUkVRVUVTVC0tLS0tCg==
~
```

KODEKLOUD

```
kubectl get csr
```

```
NAME                  AGE       REQUESTOR              CONDITION
jane                  10m       admin@example.com         Pending
```

```
kubectl certificate approve jane
```

```
jane approved!
```

```
kubectl get csr jane -o yaml
```

```
apiVersion: certificates.k8s.io/v1beta1
kind: CertificateSigningRequest
metadata:
  creationTimestamp: 2019-02-13T16:36:43Z
  name: new-user
spec:
  groups:
  - system:masters
  - system:authenticated
usages:
  - digital signature
  - key encipherment
  - server auth
  username: kubernetes-admin
status:
  certificate:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURDakNDQWZLZ0F3SUJBZ0lVRmwy
Q2wxYXoxaWl5M3JNVisreFRYQUowU3dnd0RRWUpLb1pJaHZjTkFRRUwKQlFBd0ZURVRN
QkVHQTFVRUF4TUthM1ZpWlhKdVpYUmxjekFlRncweE9UQXlNVE14TmpNeU1EQmFGd1dn
Y0ZFeDl2ajNuSXY3eFdDS1NIRm5sU041c0t5Z0VxUkwzTFM5V29GelhHZDdWCmlEZ2FO
MVVRMFBXTVhjN09FVnVjSWc1Yk4weEVHTkVwRU5tdUlBNlZWeHVjS1h6aG9ldDY0MEd1
MGU0YXFKWVIKWmVMbjBvRTFCY3dod2xic0I1ND0KLS0tLS1FTkQgQ0VSVElGSUNBVEUt
LS0tLQo=
  conditions:
  - lastUpdateTime: 2019-02-13T16:37:21Z
    message: This CSR was approved by kubectl certificate approve.
    reason: KubectlApprove
    type: Approved
```

```
echo “LS0…Qo=” | base64 --decode
```

```
-----BEGIN CERTIFICATE -----
MIICWDCCAUACAQAwEzERMA8GA1UEAwwIbmV3LXVzZXIwgg
AQUAA4IBDwAwggEKAoIBAQDO0WJW+DXsAJSIrjpNo5vRIB
LfC27t+1eEnON5Muq99NevmMEOnrDUO/thyVqP2w2XNIDR
y3BihhB93MJ7Oql3UTvZ8TELqyaDknRl/jv/SxgXkok0AB
IF5nxAttMVkDPQ7NbeZRG43b+QWlVGR/z6DWOfJnbfezOt
EcCXAwqChjBLkz2BHPR4J89D6Xb8k39pu6jpyngV6uP0tI
j2qEL+hZEWkkFz80lNNtyT5LxMqENDCnIgwC4GZiRGbrAg
9w0BAQsFAAOCAQEAS9iS6C1uxTuf5BBYSU7QFQHUzalNxA
hOK4a2zyNyi44OOijyaD6tUW8DSxkr8BLK8Kg3srREtJql
P9NL+aDRSxROVSqBaB2nWeYpM5cJ5TF53lesNSNMLQ2++R
Wr2EUM6UawzykrdHImwTv2mlMY0R+DNtV1Yie+0H9/YElt
4l3E/y3qL71WfAcuH3OsVpUUnQISMdQs0qWCsbE56CC5Dh
vwQ07jG+hpknxmuFAeXxgUwodALaJ7ju/TDIcw==
-----END CERTIFICATE -----
```

Check communication to cluster :

```
saadalsayed@Saads-MacBook-Air users % kubectl get pod  \
--server https://192.168.49.2:8443 \
--client-key saad.key \
--client-certificate saad.pub \
--certificate-authority /Users/saadalsayed/.minikube/ca.crt
```

```
Error from server (Forbidden): pods is forbidden: User "saad" cannot list
resource "pods" in API group "" in the namespace "default"
```

RBAC

RBAC



- Can view PODs
- Can create PODs
- Can Delete PODs
- Can Create ConfigMaps

Developer

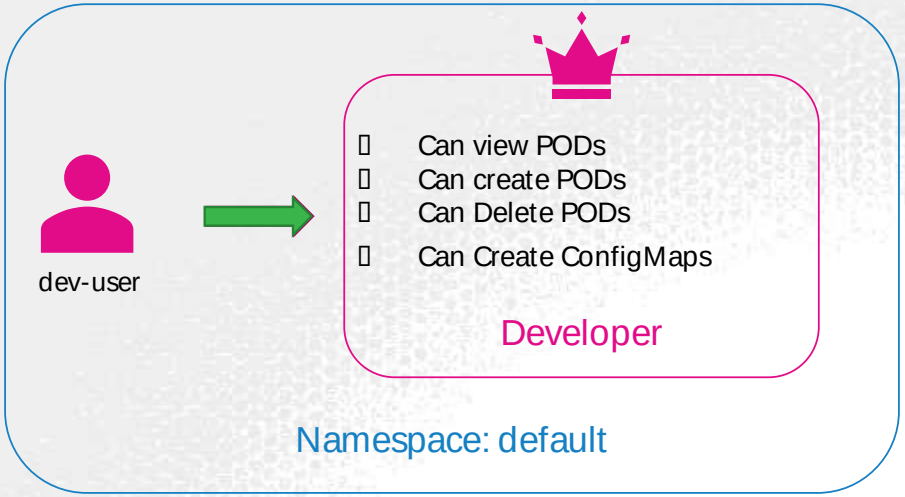
developer-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "delete"]

- apiGroups: [""]
  resources: ["ConfigMap"]
  verbs: ["create"]
```

```
kubectl create -f developer-role.yaml
```

RBAC




```
kubectl create -f devuser-developer-binding.yaml
```

developer-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "de
- apiGroups: [""]
  resources: ["ConfigMap"]
  verbs: ["create"]
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devuser-developer-binding
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

View RBAC

```
kubectl get roles
```

```
NAME        AGE
developer   4s
```

```
kubectl get rolebindings
```

```
NAME                        AGE
devuser-developer-binding   24s
```

```
kubectl describe role developer
```

```
Name:         developer
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  ---------  -----------------  --------------  -----
  ConfigMap  []                 []              [create]
  pods       []                 []              [get watch list create delete]
```

View RBAC

```
kubectl describe rolebinding devuser-developer-binding
```

```
Name:         devuser-developer-binding
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  Role
  Name:  developer
Subjects:
  Kind  Name      Namespace
  ----  ----      ---------
  User  dev-user
```

Check Access

```
kubectl auth can-i create deployments
yes
```

```
kubectl auth can-i delete nodes
no
```

```
kubectl auth can-i create deployments --as dev-user
no
```

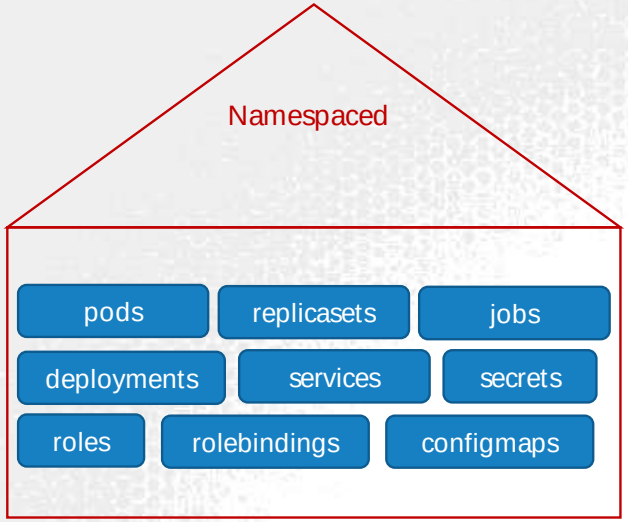
```
kubectl auth can-i create pods --as dev-user
yes
```

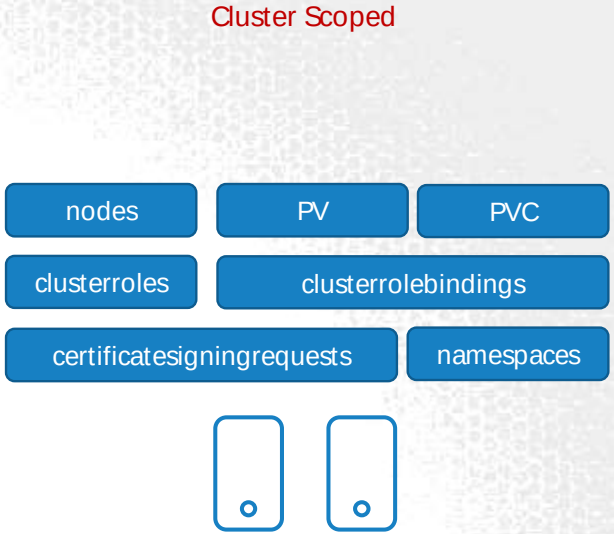
```
kubectl auth can-i create pods --as dev-user --namespace test
no
```

```
saadalsayed@Saads-MacBook-Air users % kubectl create role saad-role --resource=pod --verb=list,watch
role.rbac.authorization.k8s.io/saad-role created
saadalsayed@Saads-MacBook-Air users % kubectl create rolebinding  saad-rolebinding --user=saad --role=saad-role
rolebinding.rbac.authorization.k8s.io/saad-rolebinding created
saadalsayed@Saads-MacBook-Air users % kubectl get pod  \
--server https://127.0.0.1:50080 \
--client-key saad.key \
--client-certificate saad.pub \
--certificate-authority /Users/saadalsayed/.minikube/ca.crt
NAME                          READY   STATUS    RESTARTS   AGE
config-pod                    1/1     Running   0          30h
env-pod                       1/1     Running   0          31h
fluentd-elasticsearch-5qdjc   1/1     Running   0          3h8m
fluentd-elasticsearch-8mkdl   1/1     Running   0          3h8m
fluentd-elasticsearch-v4zwh   1/1     Running   0          3h8m
key-pod                       1/1     Running   0          30h
nginx-pod                     1/1     Running   0          29h
pod-pv                        1/1     Running   0          26h
secret-pod                    1/1     Running   0          29h
static-pod-minikube           1/1     Running   0          31h
test                          1/1     Running   0          5h30m
vol-pod                       1/1     Running   0          30h
with-affinity-anti-affinity   1/1     Running   0          119m
with-node-affinity            1/1     Running   0          96m
saadalsayed@Saads-MacBook-Air users % kubectl auth can-i create deployment --as=saad
no
saadalsayed@Saads-MacBook-Air users % kubectl auth can-i create pods --as=saad
no
saadalsayed@Saads-MacBook-Air users % kubectl auth can-i list pods --as=saad
yes
saadalsayed@Saads-MacBook-Air users %
```

Cluster Roles

Namespace

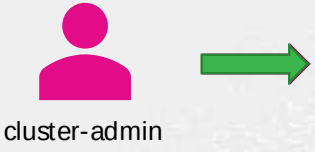


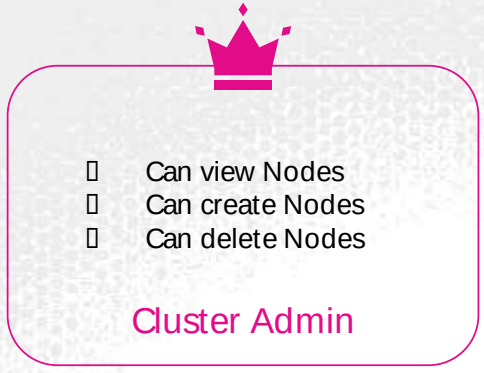



```
kubectl api-resources --namespaced=true
```

```
kubectl api-resources --namespaced=false
```

clusterrolebinding





cluster-admin-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-administrator
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["list", "get", "create", "delete"]
```

```
kubectl create -f cluster-admin-role-binding.yaml
```

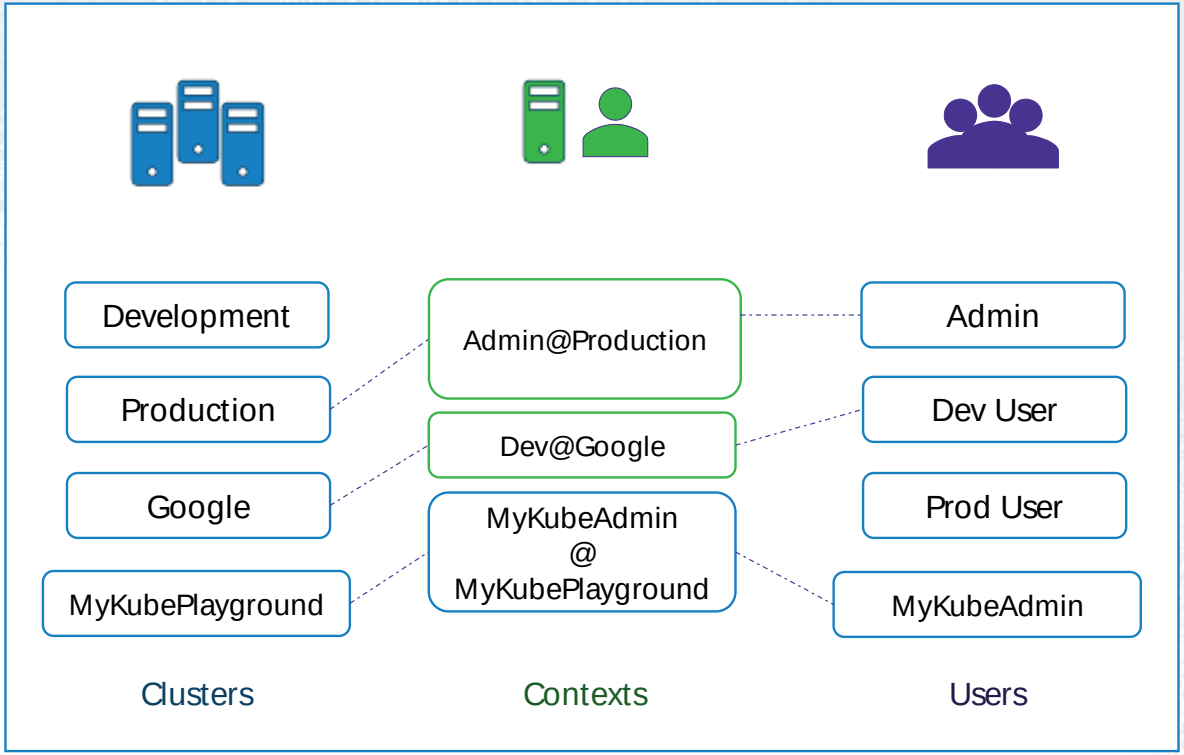
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin-role-binding
subjects:
- kind: User
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-administrator
  apiGroup: rbac.authorization.k8s.io
```

Security
KUBECONFIG

KubeConfig File

$HOME/.kube/config

```
--server my-kube-playground:6443
--client-key admin.key
--client-certificate admin.crt
--certificate-authority ca.crt
```



KubeConfig File

```yaml
apiVersion: v1
kind: Config

clusters:

- name: my-kube-playground
  cluster:
    certificate-authority: ca.crt
    server: https://my-kube-playground:6443


contexts:

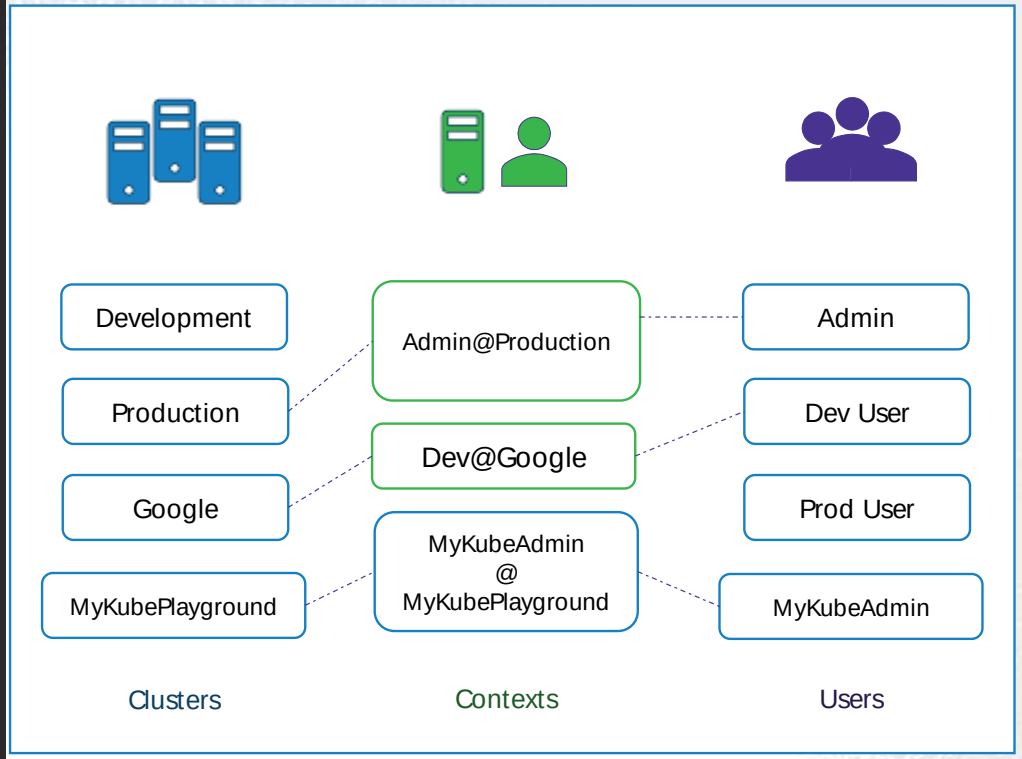
- name: my-kube-admin@my-kube-playground
  context:
    cluster:
    user:



users:

- name: my-kube-admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

$HOME/.kube/config



Kubectl config

kubectl config view

```
apiVersion: v1

kind: Config
current-context: kubernetes-admin@kubernetes

clusters:
- cluster:
    certificate-authority-data: REDACTED
    server: https://172.17.0.5:6443
  name: kubernetes

contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes


users:
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

kubectl config view –kubeconfig=my-custom-config

```
apiVersion: v1

kind: Config
current-context: my-kube-admin@my-kube-playground


clusters:
- name: my-kube-playground
- name: development
- name: production


contexts:
-  name: my-kube-admin@my-kube-playground
-  Name: prod-user@production


users:
- name: my-kube-admin
- name: prod-user
```

Add my certificate to kubeconfig :
#
$kubectl config set-credentials saad --client-key=saad.key --client-certificate=saad.pub --embed-certs=true

#add context
$kubectl config set-context saad --cluster=minikube --user=saad

#Change context
$Kubectl config use-context saad

K8s for plus:
1- ingress
2-networkpolicy
3-cluster maintenece
4-practise

==========

Main tools For DevOps :
1-Docker & k8s
2-cloud provider ➔ AWS , Azure , GCp ,….
3-automation ➔ bash script , ansible
  iac ➔ terraform ,..
4- CI/CD ➔ Jenkins. (automating app lifecycle)
5- linux
6- monitoring ➔ prometheus and grafana ,zabbix , ELK
7-version control ➔ github

DevOps Plus tool:
* DB basics .